

FROM C TO JAVASCRIPT · PRODUCTION-READY
REFERENCE

JavaScript 使用大全

面向 C 语言学习者的 14 讲系统进阶

覆盖 14 大主题:数据类型 → 运算符 → 流程控制 → 函数 → 内存管理 → 面向对象
→ 文件I/O → 网络 → 并发异常 → 泛型与元编程 → 反射 → 注解装饰器 → 模块系统 → 类型系统
进阶

第1章 数据类型与变量 · 第2章 运算符与表达式 · 第3章 流程控制 · 第4章 函数与方法
第5章 内存管理 · 第6章 面向对象 · 第7章 文件I/O · 第8章 网络通信
第9章 并发与异常 · 第10章 泛型与元编程 · 第11章 反射与动态特性
第12章 注解与装饰器 · 第13章 模块系统 · 第14章 类型系统进阶

知识导图 × C 语言对照 × 例题精讲 × 章末实验

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 JavaScript 的 8 种内建类型,弄清"C 里的 int 到 JS 里变成什么"这一核心迁移
- 理解 **number** 只有双精度浮点一种数值类型,没有 int,并掌握 BigInt 的适用场景
- 彻底分清 **undefined** 与 **null**、**typeof** 的著名坑、字符串的 UTF-16 编码模型
- 熟练使用 let/const 声明变量,理解 var 的提升(TDZ)与 const 的真实含义
- 掌握显式类型转换(Number/String/Boolean)与隐式转换的基本方向
- 会用数组与对象组织数据、用对象冻结模拟枚举(对应 C 的 enum)
- 理解"C 中怎么做→JS 中怎么做"的对照:编译期类型 → 运行时动态类型

💡 从 C 到 JavaScript:本章主题如何迁移

在 C 语言中,每一个变量在声明时就确定了它的类型与大小: `int x` 是 4 字节有符号整数, `char s[100]` 是 100 字节字符数组,类型是编译期的铁律, `sizeof` 随手可得。这些铁律保证了性能,但也带来了隐患——类型转换错误、越界、未初始化内存,一切都要程序员自己负责。

JavaScript 走的是完全相反的路: **类型属于值,不属于变量**。同一个变量可以先放整数、再放字符串、再放对象,完全合法。这被称为**动态类型**。代价是性能与"编译期帮你抓错"的能力,换来的是极高的表达力与灵活性。

你不需要担心内存:JS 有垃圾回收,没有 `malloc/free`,没有悬垂指针,没有数组越界写坏内存。你需要担心的变成了另一类问题:类型判断失误、隐式转换带来的诡异结果(如 `0 == ''` 为真)、`undefined` 满天飞。

生产中的 JS(Node.js 20+ 与浏览器)几乎都运行在 V8 引擎上,引擎内部会做 **隐藏类(hidden class)**与**即时编译(JIT)** 优化——你的变量在运行期被观察到"总是同一个类型"时,代码会跑得和静态类型语言一样快。所以"动态类型"不等于"随意类型":同一变量保持同一类型的写法,既正确又高效。

本章的目标是:把 C 的类型心智模型整体迁移到 JS。读完后,你应该能在心里快速回答"这个 C 类型在 JS 里对应什么、该怎么安全地写"。先立总纲:数字只有一个(加一个大整数),字符串是 UTF-16,布尔是真的,空值有两个,其余全是对象。

📦 前置知识自查

- C 的基本类型:int/char/float/double 的字节数与取值范围、有符号/无符号
- 变量的声明、初始化、作用域(块作用域与函数作用域)
- 字符串:char 数组、'\0' 结尾、strlen 与字节数
- enum 枚举、typedef 类型别名
- sizeof 运算符与类型大小的直觉

本章学习路线

1. **第一阶段(类型总览)**:读 §1.1 与 §1.2,先在脑中建立"8 种类型"的完整地图,把 C 类型逐一对号入座
2. **第二阶段(声明与转换)**:读 §1.3–§1.5,亲手在 Node REPL 里运行每个转换示例,观察输出
3. **第三阶段(字符串与复合类型)**:读 §1.6–§1.8,练习模板字符串与常用 API
4. **第四阶段(实验)**:独立完成章末实验"学生信息登记与成绩统计",用全部类型
5. **验收标准**:不看资料,能写出 typeof 对 8 种类型各自的输出,并说出 undefined 与 null 的 3 个区别

本章知识导图

- 第1章 数据类型与变量
 - §1.1 类型总览:C 类型 → JS 类型对照表
 - §1.2 number 与 BigInt:双精度浮点、整数安全范围
 - §1.3 变量声明:let/const/var、提升与 TDZ
 - §1.4 字面量与常量:数字/字符串/布尔/模板字符串、const 语义
 - §1.5 类型转换:显式转换与隐式转换的坑
 - §1.6 字符串:UTF-16、length 陷阱、常用 API
 - §1.7 复合类型入门:数组、对象、枚举模拟
 - §1.8 undefined 与 null 专题 + 类型别名(无)

§1.1 类型总览:C 的类型在 JS 里变成什么

直觉先行:一张表完成迁移

C 有 30 多种基本类型组合(int、unsigned long、float、char*.....);JS 只有 8 种,其中对象是一种"引用类型",其余 7 种是"原始类型"(primitive)。下表是本章最重要的对照表,建议反复看。

C 的写法	JS 的对应	说明
int / short / long	number	全部合并为双精度浮点,没有 int!
unsigned long long	bigint	超过 $2^{53}-1$ 的整数用 BigInt(字面量加 n)
float / double	number	double 是唯一浮点,float 不存在
_Bool(或 stdbool.h 的 bool)	boolean	C 没有真正的 bool,JS 有 true/false
char(字符)	string(单字符字符串)	JS 没有 char 类型,字符是长度为 1 的字符串
char[] / char*	string	字符串不可变,自带长度,无 '\0' 结尾
struct / union	object	对象,属性可动态增删
数组 int a[10]	Array(数组对象)	动态长度,可混合存放任意类型
指针 int* p	引用(值本身)	对象天然按引用共享,无指针运算
NULL 指针	null / undefined	两个空值,语义不同(见 §1.8)
enum	对象 + Object.freeze	用常量对象模拟,见 §1.7
typedef	(无)	JS 没有类型别名,类型跟随值

八种内建类型一览:**number**(双精度浮点)、**bigint**(任意精度整数)、**string**(UTF-16 字符串)、**boolean**、**undefined**、**null**、**symbol**(唯一标识,常用于对象键)、**object**(含数组、函数、日期等一切引用类型)。函数是对象的一种,但 `typeof` 函数 会返回 `'function'` ——这是历史遗留的特例。

⚠ 常见误区 1.1:typeof null 返回 'object'

这是语言自诞生就存在的 bug,无法修复(修复会破坏海量存量代码)。`typeof null === 'object'` 为 true,但 null 是原始类型。判断空值请直接用 `x === null`,不要依赖 `typeof`。

1.1.1 typeof 运算符:每个类型输出什么

```
// 在 Node.js 20+ 或浏览器控制台运行
console.log(typeof 42);           // 'number'
console.log(typeof 3.14);         // 'number' (没有 float,全是 number)
console.log(typeof 42n);          // 'bigint'
console.log(typeof 'hello');      // 'string'
console.log(typeof true);         // 'boolean'
console.log(typeof undefined);    // 'undefined'
console.log(typeof null);         // 'object' ← 著名的 bug,记住它!
console.log(typeof Symbol('id')); // 'symbol'
console.log(typeof {});           // 'object'
console.log(typeof []);           // 'object' ← 数组也是 object
console.log(typeof function (){}); // 'function' ← 函数特例
```

⚠ 常见误区 1.2:typeof 判不了"未声明的变量"就报错?

不会报错。`typeof undeclaredVar` 即使变量完全不存在也安全返回 `'undefined'`,这是 `typeof` 唯一的安全特性,生产代码里常用于特征检测,例如判断 `typeof process !== 'undefined'` 来区分 Node 与浏览器环境。

§1.2 number 与 BigInt:没有 int 的世界

C 的 `int` 加 1 溢出会回绕(有符号溢出是未定义行为);JS 的 `number` 加 1 溢出会变成 `Infinity`。因为 `number` 是 IEEE 754 双精度浮点,它能精确表示的最大整数是 $2^{53} - 1 = 9007199254740991$,即 `Number.MAX_SAFE_INTEGER`。超过这个范围的整数运算会悄悄丢失精度——这是生产事故的高发区(如订单号、ID、时间戳纳秒)。

```
// 双精度带来的"整数不精确"现象
console.log(0.1 + 0.2);           // 0.30000000000000004 ← 浮点误差,和 C 一样
console.log(9007199254740992);    // 9007199254740992
console.log(9007199254740992 + 1); // 9007199254740992 ← 精度丢了,+1 无效!
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
console.log(Number.isSafeInteger(9007199254740992)); // false

// 浮点比较的正确姿势:用极小差值(epsilon)
function nearlyEqual(a, b) {
  return Math.abs(a - b) < Number.EPSILON * Math.max(Math.abs(a), Math.abs(b));
}
console.log(nearlyEqual(0.1 + 0.2, 0.3)); // true

// 大整数:用 BigInt,字面量末尾加 n
const big = 9007199254740993n;
console.log(big + 1n);             // 9007199254740994n
console.log(typeof big);           // 'bigint'
```

⚠ 常见误区 1.3:BigInt 与 number 不能混算

`1n + 1` 直接抛 `TypeError: Cannot mix BigInt and other types`。必须显式转换: `Number(1n) + 1` 或 `1n + BigInt(1)`。另外 `BigInt` 不能用于 `Math` 对象方法,也不能与浮点字面量混用。生产建议:需要大整数的字段(如雪花 ID)全程用 `BigInt`,或统一转字符串传输(JSON 不支持 `BigInt`)。

1.2.1 特殊数值:NaN、Infinity 与 -0

JS 有四个"奇特"的 `number` 值,生产中必须认识:**NaN**(Not a Number,唯一的非自等值:NaN 不等于任何数包括它自己)、**Infinity**、**-Infinity** 与 **-0**。除以 0 不会崩溃,而是返回 `Infinity`——这与 C 的"除零是未定义行为/崩溃"截然不同,需要 **显式检查**。

```
console.log(1 / 0);                // Infinity
console.log(0 / 0);                // NaN
console.log(NaN === NaN);          // false ← 经典坑
console.log(Number.isNaN(NaN));    // true ← 正确的判断方式
console.log(isNaN('abc'));         // true ← 全局 isNaN 先转字符串,有坑!
console.log(Number.isNaN('abc'));  // false ← 正确的 Number.isNaN
console.log(0 === -0);              // true(=== 分不出正负零)
console.log(Object.is(0, -0));     // false ← Object.is 能区分
console.log(Number.isFinite(Infinity)); // false
```

§1.3 变量声明:let / const / var

C 里变量声明只分"有初始化/无初始化";JS 里声明要回答两个问题:可变吗(const vs let)与作用域(块作用域 vs 函数作用域)。现代生产代码的黄金法则:默认 `const`,需要重新赋值才用 `let`,永远不用 `var` (历史遗留代码除外)。

对比项	C	JS 的 let/const
未初始化变量	局部变量是垃圾值(UB)	值是 undefined,不会读到垃圾
作用域	块作用域(C99 起)	块作用域(if/for/{} 内)
作用域外访问	编译错误	运行时 ReferenceError
重复声明	同作用域内报错	let/const 重复声明报 SyntaxError
声明前使用	可以(值未定义)	TDZ,报 ReferenceError(见下)

1.3.1 声明前使用:TDZ(暂时性死区)

let/const 声明的变量在声明语句执行之前处于 **暂时性死区** (Temporal Dead Zone, TDZ), 任何读取都会抛 ReferenceError。这比 C 的"未初始化垃圾值"严格得多,本质是语言替你抓错。

```
// console.log(x); // ❌ 报错:ReferenceError: Cannot access 'x' before initialization
let x = 10;
console.log(x);    // 10

// var 则是"提升"到函数顶部,值为 undefined,不报错——这正是 var 的坑
console.log(y);    // undefined(不报错,因为 var 提升了声明)
var y = 20;
```

★ 重点结论:const 的"不变"是什么

const obj = { n: 1 } 里, const 锁死的是绑定(不能重新赋值),不是内容。obj.n = 2 合法, obj = {} 报错。类比:C 里 int *const p 锁死指针本身,不锁死指向的对象。需要连内容都锁死时用 Object.freeze (浅冻结)。

```
const URL_BASE = 'https://api.example.com';
// URL_BASE = 'x'; // ❌ TypeError: Assignment to constant variable

const config = { retries: 3 };
config.retries = 5;    // ✅ 合法:修改属性不违反 const
console.log(config.retries); // 5
// config = {};    // ❌ 重新赋值报错

const list = [1, 2, 3];
list.push(4);    // ✅ 合法:数组内容可改
console.log(list);    // [ 1, 2, 3, 4 ]
```

§1.4 字面量与常量

1.4.1 数字字面量

C 的进制写法(0x、0、0b 等)与后缀(L、U、f)在 JS 中大部分保留,但 **没有后缀字母**,大整数用 n 后缀。数字分隔符 `_` 是 ES2021 新特性,用于提高大数字可读性。

```
const hex = 0xFF;           // 255,十六进制
const oct = 0o17;           // 15,八进制(0o 前缀,注意不是 C 的 0 前缀)
const bin = 0b1010;         // 10,二进制
const sci = 1.5e3;           // 1500,科学计数法
const sep = 1_000_000;       // 1000000,数字分隔符,纯可读性
const big = 12345678901234567890n; // BigInt 字面量
// 注意:没有 float/double 之分,3.14 与 3 都是 number
console.log(hex, oct, bin, sci, sep, typeof big);
```

1.4.2 字符串字面量与模板字符串

C 只有双引号一种字符串;JS 有 **三种**:单引号、双引号(二者等价,生产规范通常统一其一)、**模板字符串**(反引号,支持换行与 `${表达式}` 插值,对应 C 的 `sprintf` 家族)。

```
const s1 = '单引号';        // 单引号
const s2 = "双引号";        // 双引号,等价
const s3 = `多行
字符串`;                    // 模板字符串:可换行

const name = '张伟';
const score = 92;
// 模板插值 ≈ C 的 sprintf("%s 得分 %d", name, score)
const report = `${name} 得分 ${score} 分`;
console.log(report);         // 张伟 得分 92 分

// 转义序列与 C 类似
const path = 'C:\\Users\\me'; // 双反斜杠
const line = '第一行\n第二行';
console.log(line);
```

工程建议 1.1:拼接字符串用模板字符串,不要用 +

大量 `'a' + b + 'c'` 拼接可读性差且容易踩隐式转换的坑。一律使用模板字符串;需要性能的批量拼接(如循环内拼 1 万次以上)改用数组 `push + join` 或 `Array.from` ——V8 对字符串拼接有优化,但模板字符串仍是最佳默认。

§1.5 类型转换:C 是"你说了算",JS 是"我说了算"

C 的转换是 **显式为主**: `(int)x` 一句话,编译器按规则截断;隐式转换(整型提升、`int→double`)规则清晰。JS 的 `number/string/boolean` 之间的隐式转换规则 **诡异且频繁触发** (运算符 `+`、`==` 等),所以生产铁律: **显式转换,不要依赖隐式**。隐式转换细节在第2章讲透,本章先掌握显式转换。

```
// 显式转换:Number() / String() / Boolean()
console.log(Number('42'));           // 42
console.log(Number(''));             // 0 ← 空串变 0,坑!
console.log(Number(' 12 '));         // 12,自动去首尾空格
console.log(Number('12abc'));        // NaN ← 不是 12!全串解析失败
console.log(Number(true));           // 1
console.log(Number(null));           // 0
console.log(Number(undefined));      // NaN

console.log(String(42));              // '42'
console.log(String(null));            // 'null'
console.log(String([1, 2, 3]));       // '1,2,3'

console.log(Boolean(0));              // false
console.log(Boolean(''));            // false
console.log(Boolean('0'));            // true ← '0' 是非空字符串!

// parseInt 家族:部分解析,与 Number 完全不同
console.log(parseInt('42abc'));       // 42,解析到非法字符为止
console.log(parseInt('0x1F'));        // 31,自动识别 0x
console.log(parseInt('10', 2));       // 2,第二个参数是基数(radix)
console.log(parseInt('08'));          // 8(老浏览器曾按八进制解析出 0)
console.log(parseFloat('3.14abc'));  // 3.14
```

⚠ 常见误区 1.4:parseInt 必须显式给基数

生产代码请永远写 `parseInt(s, 10)`。虽然现代引擎已不把 '08' 当八进制,但代码评审规范通常要求显式基数,防止历史行为差异。另外 `parseInt` 用于"用户输入→整数"时,记得先检查 `Number.isFinite` 结果,NaN 不等于任何值, `parseInt('abc') === NaN` 是 false,比不出结果。

§1.6 字符串:不可变与 UTF-16

C 的字符串是 `char[]` 加 `\0`, `strlen` 数是字节;JS 的字符串是 **不可变对象** (任何"修改"都产生新字符串),没有 `\0`,自带 `length`。编码模型是 **UTF-16**:`length` 数的是 16 位码元(code unit),不是字符个数,也不是字节数。中文、英文、数字在 BMP 平面内都是一个码元,`length` 与字符数一致;但 **emoji、生僻字(增补平面)** 占用两个码元,`length` 会"多算"。

```

const zh = '你好世界';
console.log(zh.length);           // 4, 中文每字一个码元
console.log('hello'.length);      // 5, ASCII 一个码元
const emoji = '🍌';                // U+1F600, 增补平面
console.log(emoji.length);        // 2 ← 坑! 一个字符占两个码元
console.log([...emoji].length);    // 1 ← 展开运算符按码点拆分, 正确计数
console.log(emoji.codePointAt(0).toString(16)); // 1f600, 码点

// 常见 API 速查(与 C 对照)
console.log('hello world'.indexOf('world')); // 6, 查找位置(类似 strstr)
console.log('hello'.slice(1, 3));             // 'el', 子串(C 没有原生对应)
console.log('a,b,c'.split(','));             // ['a','b','c'], 拆分
console.log(['a', 'b'].join('-'));           // 'a-b', 拼接
console.log(' x '.trim());                   // 'x', 去首尾空白
console.log('abc'.toUpperCase());           // 'ABC'
console.log('abc'.replace('a', 'x'));        // 'xbc', 只替换第一处
console.log('3'.padStart(3, '0'));           // '003', 补零
console.log('hello'.includes('ell'));        // true
console.log('file.txt'.endsWith('.txt'));    // true
console.log('abc'.charAt(1));                 // 'b', 等价 s[1]

```

⚠ 常见误区 1.5: 字符串不可变, 别学 C 改字符

`s[0] = 'x'` 在非严格模式下静默失败, 严格模式下抛错——总之 **改不了**。要“改”就构造新串: `s.slice(0, i) + newChar + s.slice(i + 1)`。另外拼接百万次字符串时, 用数组收集再 `join`, 避免反复创建大对象。

§1.7 复合类型入门: 数组、对象与枚举模拟

1.7.1 数组: C 的 `int a[10]` → 动态数组

```

const nums = [10, 20, 30];          // 数组字面量
nums.push(40);                      // 尾部追加(类似 C 里手动 realloc)
nums.pop();                          // 移除尾部
console.log(nums.length);            // 3, 长度是动态的
console.log(nums[1]);                // 20, 按下标访问
// 数组可以装任何类型(C 做不到)
const mixed = [1, 'two', { name: 'three' }, [4]];
console.log(mixed[1]);               // 'two'
console.log(Array.from('abc'));      // ['a','b','c'], 从类数组转数组

```

1.7.2 对象:struct 的动态版

```
// 对象字面量 ≈ 匿名 struct,但属性可随时增删
const student = {
  id: 1001,
  name: '李雷',
  scores: [88, 92, 76],
};
student.age = 18; // 运行时加属性(C 做不到)
delete student.age; // 删除属性
console.log(student.name); // '李雷',点号访问
console.log(student['id']); // 1001,方括号访问(键可以是变量)
const key = 'name';
console.log(student[key]); // '李雷',动态键名

// 遍历对象属性
for (const k of Object.keys(student)) {
  console.log(k, student[k]);
}
```

1.7.3 枚举模拟:C 的 enum → 冻结对象

C 的 enum 在 JS 中没有直接对应。生产惯例:用 `const` 对象 + `Object.freeze` 模拟,配合 `Object.freeze` 使其不可修改。ES2020+ 还有更正式的 **Symbol** 枚举写法,但冻结对象最常用。

```
// 模拟 C 的 enum Color { RED, GREEN, BLUE }
const Color = Object.freeze({
  RED: 'red',
  GREEN: 'green',
  BLUE: 'blue',
});
// Color.RED = 'x'; // 严格模式下抛 TypeError,对象被冻结

function describe(c) {
  switch (c) {
    case Color.RED: return '红色';
    case Color.GREEN: return '绿色';
    case Color.BLUE: return '蓝色';
    default: return '未知颜色';
  }
}
console.log(describe(Color.RED)); // 红色
```

§1.8 undefined 与 null 专题、类型别名

💡 直觉先行:两个空值怎么分

undefined 表示"系统给的默认空":变量声明未赋值、函数没 return、对象没有某属性,JS 自动给你 undefined。
null 表示"程序员主动说的空":C 里你写 `return NULL`,JS 里你写 `return null`,表达"这里故意没有值"。分工:C 的空全靠 NULL 一个,JS 拆成两个——undefined 是"语言层面的缺席",null 是"业务层面的空"。

对比项	undefined	null
产生来源	语言自动(未初始化、缺失属性)	程序员显式赋值
typeof 结果	'undefined'	'object'(bug)
布尔转换	false	false
相等性	== null 为 true;=== 不相等	== undefined 为 true;=== 不相等
JSON 序列化	属性被丢弃	序列化为 null
参数默认值触发	会触发默认值	不会触发(传 null 就是 null)

```

let a;           // 声明未赋值
console.log(a);  // undefined
function f() {}  // 没有 return
console.log(f()); // undefined
const obj = {};
console.log(obj.missing); // undefined ← 缺属性也是 undefined

// 判断"没有值"的惯用法:== null 同时匹配 undefined 和 null
function safe(x) {
  if (x == null) return '空';
  return '有值';
}
console.log(safe(undefined)); // 空
console.log(safe(null));      // 空
console.log(safe(0));          // 有值 ← 0 不是空
console.log(safe(''));         // 有值 ← 空串也不是空

```

类型别名:JS 没有 typedef,也不需要——类型属于值,一个函数接受任何类型。第14章会介绍 JSDoc 注释与 TypeScript 如何补上"给类型起名"的能力,生产大型项目通常引入 TypeScript 或 JSDoc。

工程建议 1.2:函数参数永远不要传 undefined 当"空"

API 边界统一用 null 表示"没有值": `updateUser(id, null)` 语义清晰,JSON 序列化也会保留 null;undefined 在 JSON 中会被静默丢弃,可能导致"字段神秘消失"的线上事故。数据库字段的"空"也建议显式 null。

本章速查表

主题	速查要点
类型总数	8 种:number、bigint、string、boolean、undefined、null、symbol、object
int 去哪了	没有 int!整数也是 number(双精度),安全整数范围 $\pm 2^{53}-1$
大整数	BigInt,字面量 10n,不能与 number 混算
布尔	true/false;C 没有真 bool,JS 有
字符	没有 char 类型,字符是长度为 1 的 string
typeof null	'object'(语言 bug),判空用 <code>x === null</code> 或 <code>x == null</code>
NaN	<code>NaN !== NaN</code> ;判断用 <code>Number.isNaN</code>
除零	1/0 是 Infinity,不崩溃;0/0 是 NaN
声明	默认 const,要改值用 let,不用 var
TDZ	let/const 声明前访问抛 ReferenceError
const 语义	锁绑定不锁内容;要锁内容用 <code>Object.freeze</code>
字符串编码	UTF-16;length 是码元数,emoji 占 2
字符串不可变	所有"修改"产生新串,用 slice/split/join 组合
数字转整数	<code>parseInt(s, 10)</code> 显式基数;Number(s) 全串解析
undefined vs null	undefined 语言默认, null 业务空;JSON 只保留 null
枚举	<code>Object.freeze({...})</code> 模拟
类型别名	无;大型项目用 TypeScript/JSDoc

最小必会清单

- 能说出 8 种内建类型,并写出 typeof 对每种类型的输出(含 null 的 bug)
- 能解释为什么 `0.1 + 0.2 !== 0.3`,以及 `Number.isSafeInteger` 的用途
- 能说出 const 锁绑定不锁内容,并演示 `Object.freeze`
- 能解释 TDZ,并说明 var 提升为什么危险
- 能写出模板字符串插值,替代 C 的 `sprintf`
- 能说出 `Number('12abc')` 与 `parseInt('12abc')` 的区别
- 能解释字符串 length 对 emoji 的"多算"问题,并用 [...s] 修正
- 能写出 undefined 与 null 的 3 个区别(来源/typeof/JSON)
- 能用 `Object.freeze` 模拟枚举,并配合 switch 使用
- 能写出数组 push/pop/length 与对象增删属性的代码

自测题

Q 1. 写出以下代码在 Node.js 20 中的输出:`console.log(typeof 3.14, typeof 'a', typeof NaN, typeof [])`

✅ 参考答案

输出: 'number' 'string' 'number' 'object'。3.14 是 number(无 float);'a' 是字符串(无 char);NaN 是 number;[] 是 object。

Q 2. `const a = 5;` 之后执行 `a = 6` 会发生什么?`const obj = {x:1};` 之后执行 `obj.x = 2` 呢?

✅ 参考答案

`a = 6` 抛 `TypeError`(常量不能重新赋值); `obj.x = 2` 合法,`const` 只锁绑定不锁内容。

Q 3. 说明 `undefined` 与 `null` 在 `typeof`、JSON 序列化、默认参数三个场景下的差异。

✅ 参考答案

`typeof:undefined` 返回 'undefined',`null` 返回 'object'(bug)。JSON.stringify({a: undefined, b: null}) 结果是 '{"b":null}',`undefined` 属性被丢弃。默认参数 `function f(x = 1){}` :`f(undefined)` 得 1,`f(null)` 得 null。

Q 4. 为什么 `9007199254740992 + 1 === 9007199254740992`?如何安全计算这类大整数?

✅ 参考答案

9007199254740992 已超过 $2^{53}-1$ 的精确整数范围,双精度浮点表示不下 +1 的变化,结果被舍入回原值。改用 `BigInt`: `9007199254740992n + 1n`。

Q 5. `'ab😊cd'` 的 `length` 是多少?如何正确数出可见字符个数?

✅ 参考答案

`length` 为 6('ab' 2 个码元 + 😊 2 个码元 + 'cd' 2 个码元)。正确计数: `[...'ab😊cd'].length` 得 4,或用 `Array.from`、`Intl.Segmenter` (按字素,更精确)。

Q 6. `Number('')`、`Number('12abc')`、`parseInt('12abc')`、`Boolean('0')` 分别是什么?

✅ 参考答案

0、NaN、12、true。空串转数字是 0;Number 全串解析失败给 NaN;parseInt 部分解析;'0' 是非空字符串,转布尔是 true。

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记与成绩统计

实验目标:用全章类型与转换知识,实现一个命令行学生信息登记与成绩统计程序。

实验要求:

- 任务 1(覆盖:对象、数组):定义 Student 对象(id/name/scores),用数组保存多名学生
- 任务 2(覆盖:枚举模拟、常量):用 Object.freeze 定义成绩等级枚举(A/B/C/D/F)
- 任务 3(覆盖:number、BigInt、浮点精度):统计平均分,处理 0.1+0.2 类浮点误差(用 toFixed 或 epsilon 比较);为学号使用大整数演示 BigInt
- 任务 4(覆盖:字符串 API、模板字符串):姓名按模板字符串格式化输出,支持按姓名模糊查找(用 includes)
- 任务 5(覆盖:显式转换):从字符串输入解析分数(parseInt/Number),非法输入给出提示
- 任务 6(覆盖:undefined/null、typeof、TDZ、const):演示 const 绑定不可改、未初始化变量为 undefined、用 == null 防御空值

实验环境:Node.js 20+,在项目目录执行 `node student.js`。

第一步 **建数据层:**先定义 Color 式冻结枚举 `Grade`,再构造 3 名学生对象放入数组

第二步 **现统计函数:**平均分用 reduce 求和后除以 length,用 `Math.round(x * 100) / 100` 处理浮点误差

第三步 **现查找与等级判定:**写 `findByName`(用 includes)、`gradeOf`(用 if/else 链,覆盖全等级)

第四步 **分项:**把成绩解析封装成 `safeParseScore`,返回 null 表示解析失败(练习 null 语义)

✅ 参考答案要点

核心思路:全部数据用 `const` 声明(对象内容可变),防御空值用 `== null`,输出统一模板字符串。代码约 100 行,覆盖本章所有知识点。

```
// student.js — Node.js 20+ 运行:node student.js
'use strict';

// 1. 枚举模拟(覆盖:Object.freeze、const 对象)
const Grade = Object.freeze({
  A: 'A',
  B: 'B',
  C: 'C',
  D: 'D',
  F: 'F',
});

// 2. 学生对象与数组(覆盖:对象、数组、BigInt 学号)
const students = [
  { id: 2025001n, name: '李雷', scores: [88, 92, 76] },
  { id: 2025002n, name: '韩梅梅', scores: [95, 89, 90] },
  { id: 2025003n, name: '王芳', scores: [60, 70, 65] },
];

// 3. 平均分(覆盖:number、浮点精度、模板字符串)
function average(scores) {
  if (scores == null || scores.length === 0) return 0;
  const sum = scores.reduce((acc, s) => acc + s, 0);
  return Math.round((sum / scores.length) * 100) / 100;
}

// 4. 等级判定(覆盖:if/else、枚举使用、Boolean 真值)
function gradeOf(avg) {
  if (avg >= 90) return Grade.A;
  if (avg >= 80) return Grade.B;
  if (avg >= 70) return Grade.C;
  if (avg >= 60) return Grade.D;
  return Grade.F;
}

// 5. 按姓名查找(覆盖:字符串 API includes、undefined)
function findByName(name) {
  return students.find((s) => s.name.includes(name));
}

// 6. 安全解析分数(覆盖:显式转换、null 语义)
function safeParseScore(text) {
  const n = Number(text);
  if (!Number.isFinite(n)) return null;
  if (n < 0 || n > 100) return null;
  return n;
}

// 7. 打印成绩单(覆盖:模板字符串、for...of、解构)
function printReport() {
  console.log('===== 成绩单 =====');
  for (const s of students) {
    const avg = average(s.scores);
    console.log(`学号 ${s.id} | ${s.name} | 平均 ${avg} 分 | 等级 ${gradeOf(avg)}`);
  }
}
```

```
}

// 8. 主流程(覆盖:const/let、TDZ 概念、== null 防御)
function main() {
  printReport();

  const hit = findByName('梅梅');
  if (hit == null) {
    console.log('未找到学生');
  } else {
    console.log(`找到:${hit.name},平均 ${average(hit.scores)} 分`);
  }

  const demo = safeParseScore('78.5');
  console.log(`解析 '78.5' -> ${demo},等级 ${gradeOf(demo)}`);
  const bad = safeParseScore('abc');
  console.log(`解析 'abc' -> ${bad}(null 表示非法输入)`);
}

main();
```

设计说明:avg 先用 `Math.round(x*100)/100` 把浮点误差收敛到两位小数;findByName 返回 undefined 时用 `== null` 同时拦截 undefined 与 null;学号用 `BigInt` 演示大整数;全程序除循环变量外全部 `const`,符合生产规范。

下一章预告:第2章 运算符与表达式 —— 掌握 `===` 与 `==` 的全部差异、短路运算符返回操作数的行为、位运算的 32 位限制,这些是阅读任何 JS 代码的第一道坎。

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 JS 算术运算与 C 的差异:双精度溢出、除法语义、负数取模、** 指数
- 理解位运算的 32 位有符号整数限制与无符号右移 >>>
- 讲透逻辑运算符返回 **操作数本身** 而非布尔值的机制,含 ?? 空值合并
- 彻底掌握 === 与 ==:真值表、隐式转换的著名坑(0=='0'、null>=0、[]==![])
- 熟练使用解构赋值、展开运算符、可选链 ?.
- 掌握运算符优先级表,理解求值顺序与 C 的差异
- 理解"C 中怎么做→JS 中怎么做":C 的表达式是纯数值运算,JS 的表达式经常是"值选择"

💡 从 C 到 JavaScript:本章主题如何迁移

运算符是两种语言最"像"的部分——+ - * / %、位运算、逻辑与比较,符号几乎完全一样,连优先级表都大同小异。但你若照搬 C 的心智模型,第 2 天就会在生产代码里踩坑。

第一个差异是 **除法**:C 里 5/2 是 2(截断),JS 里 5/2 是 2.5。第二个差异是 **逻辑运算符的返回值**:C 的 `a && b` 永远得到 0 或 1,JS 的 `a && b` 返回 a 或 b 的 **原值**。第三个差异是相等比较:JS 的 `==` 会做隐式类型转换,产生 `0 == ''` 为真这样的"鬼故事",因此生产规范几乎一律使用 `===`。

第四个差异是 **位运算只对 32 位整数有效**——`0xFFFFFFFF` 参与位运算会得到 -1,与 C 的 64 位直觉完全不同。

第五个差异是 JS 补了三个 C 没有的实用运算符:指数 `**`、可选链 `?.`、空值合并 `??`,它们让"安全取值"变得极其简洁。

读完本章,你应该能对任何一段 JS 表达式做两件事:① 判断它输出什么(值);② 说出它为什么(规则)。这两件事是对一切代码评审的基础。

📦 前置知识自查

- C 的算术/关系/逻辑/位/赋值运算符及优先级
- C 的短路求值:&& 与 || 只算必要的一边
- C 的整数除法截断与负数取模结果(C99 规定向零截断)
- C 的复合赋值 +=、-=、*=(注意 *= 与 ++ 的区别)
- C 的强制转换 (int) 与隐式整型提升

🚀 本章学习路线

1. **第一阶段(算术与位运算)**:读 §2.1–§2.3,重点建立"number 是浮点"的运算直觉
2. **第二阶段(逻辑与比较)**:读 §2.4–§2.5,亲手在 REPL 验证每个真值表条目,这是本章价值核心
3. **第三阶段(赋值与优先级)**:读 §2.6–§2.8,做优先级小测验
4. **第四阶段(实验)**:完成章末"表达式求值器"
5. **验收标准**:能不看资料说出 `0==''`、`null>=0`、`[]==![]` 三个表达式的值及原因

本章知识导图

- 第2章 运算符与表达式
 - §2.1 算术运算:除法语义、溢出、取模、指数 **
 - §2.2 自增自减:前置/后置
 - §2.3 位运算:32 位限制、>>>、标志位场景
 - §2.4 逻辑与短路:返回操作数、?? 空值合并
 - §2.5 比较:=== 与 == 真值表、NaN、Object.is
 - §2.6 赋值、解构与展开
 - §2.7 三目运算符与表达式风格
 - §2.8 优先级与求值顺序
 - §2.9 特色运算符:可选链 ?.、in、instanceof

§2.1 算术运算:五种与 C 的本质差异

运算	C 的行为	JS 的行为
<code>5 / 2</code>	2(int 除法截断)	2.5(永远是浮点除法!)
<code>5 % 2</code>	1	1(与 C 一致,向零取整)
<code>-5 % 2</code>	-1(C99 向零截断)	-1(符号随被除数)
溢出	有符号 UB,无符号回绕	变成 Infinity,永不崩溃
除 0	运行时错误/UB	<code>1/0 = Infinity</code> , <code>0/0 = NaN</code>
指数	用 <code>pow()</code> 函数	** 运算符与 <code>Math.pow</code>

```

console.log(5 / 2);           // 2.5 ← 没有整数除法!想取整用 Math.floor
console.log(Math.floor(5 / 2)); // 2
console.log(5 % 2);           // 1
console.log(-5 % 2);           // -1(结果符号跟随被除数,和 C 相同)
console.log(5 % -2);           // 1
console.log(2 ** 10);          // 1024,指数运算符(ES2016)
console.log(2 ** 0.5);         // 1.4142135623730951,支持浮点指数
console.log(1 / 0);            // Infinity
console.log(-1 / 0);           // -Infinity
console.log(0 / 0);            // NaN

// number 的精度:超过安全整数就失真
console.log(0.1 + 0.2);        // 0.30000000000000004(浮点误差,C 也一样)
console.log(Number.MAX_VALUE * 2); // Infinity,真正的溢出

```

2.1.1 取整三兄弟:floor / ceil / trunc

C 的 (int)x 是向零截断;JS 没有 (int),对应的是 `Math.trunc`。负数场景必须选对:

```

console.log(Math.floor(-1.5)); // -2 ← 向下取整(向负无穷)
console.log(Math.ceil(-1.5));  // -1 ← 向上取整
console.log(Math.trunc(-1.5)); // -1 ← 向零截断,等价于 C 的 (int)(-1.5)
console.log(Math.round(-1.5)); // -1 ← 四舍五入,注意 -1.5 向 -1 走!
console.log(Math.round(2.5));  // 3(正数 0.5 进位,负数有坑)

```

⚠ 常见误区 2.1:用 parseInt 代替 Math.floor

`parseInt(-0.5)` 得 -0(`parseInt` 只处理字符串/数字的整数部分,先转字符串再解析); `Math.floor(-0.5)` 得 -1。两者对负数的语义完全不同。生产规范:数值取整只用 `Math.floor/trunc/round`,`parseInt` 只用于"字符串转整数"。

§2.2 自增自减:与 C 完全一致

```

let i = 5;
console.log(i++); // 5(后置:先用后加)
console.log(i);   // 6
console.log(++i); // 7(前置:先加后用)
console.log(i);   // 7

// 注意:字符串参与算术运算会隐式转数字(第2.5节详细讲)
let s = '3';
console.log(s + 1); // '31' ← + 碰到字符串就变拼接!
console.log(s - 1); // 2    ← - * / 则会转成数字
console.log(+s);    // 3    ← 一元 + 是"转数字"的惯用法

```

⚠ 常见误区 2.2:+ 是"又加又拼"的变色龙

C 的 `+` 只做加法;JS 的 `+` 只要任一边是字符串,就变成字符串拼接: `1 + 2 + '3'` 是 '33'(先 `1+2=3` 再拼 '3'),而 `'1' + 2 + 3` 是 '123'。生产铁律:拼接用模板字符串,加法要保证两边都是 `number`,否则用 `Number()` 显式转换。

§2.3 位运算:32 位有符号整数的世界

JS 的所有位运算符(& | ^ ~ << >> >>>)都会把操作数先转成 32 位有符号整数再运算,结果也按 32 位有符号解释。这意味着:

```
console.log(1 << 31);           // -2147483648 ← 溢出成负数了!
console.log((1 << 31) >>> 0);    // 2147483648 ← 无符号右移修正
console.log(0xFFFFFFFF);        // 4294967295(字面量是普通 number)
console.log(0xFFFFFFFF | 0);     // -1 ← 位运算后变成 32 位有符号!
console.log(~0);                 // -1(取反,和 C 一样)
console.log(7 & 3);              // 3(按位与)
console.log(7 | 8);              // 15(按位或)
console.log(7 ^ 3);              // 4(按位异或)
console.log(8 >> 1);             // 4(有符号右移,补符号位)
console.log(-8 >> 1);            // -4
console.log(-8 >>> 1);           // 2147483644(无符号右移,高位补 0)
```

★ 重点结论:JS 没有 >>> 之外,位运算世界 = 32 位

参与位运算的 number 先做 `ToInt32`:超过 32 位范围的数值会被截断。所以位运算不能用于大整数 (ID 字段、时间戳),那是 `BigInt` 的领域(`10n & 3n` 是合法的 `BigInt` 位运算)。32 位限制的最大实用价值:位运算很快,常用于标志位组合与状态压缩。

2.3.1 生产场景:权限标志位(与 C 的 enum 位标志对照)

```
// C: enum Perm { READ=1, WRITE=2, EXEC=4 }; int p = READ|WRITE;
const PERM = Object.freeze({
  READ: 1, // 0b001
  WRITE: 2, // 0b010
  EXEC: 4, // 0b100
});
let perm = PERM.READ | PERM.WRITE; // 3 = 0b011,组合权限
console.log(perm & PERM.READ);     // 1,非零即有读权限
console.log(perm & PERM.EXEC);     // 0,无执行权限
perm |= PERM.EXEC;                 // 7 = 0b111,追加权限
console.log(perm);                 // 7
perm &= ~PERM.WRITE;               // 5 = 0b101,撤销写权限
console.log(perm);                 // 5
console.log((perm & PERM.WRITE) !== 0); // false,确认写权限已撤销
```

⚠ 常见误区 2.3:用 && 判断位标志

`perm && PERM.READ` 逻辑错误——&& 判断的是真值不是位!判断位必须用 `(perm & PERM.READ) !== 0`,不要用 `perm && x` 这种"巧写",它会把 0 以外的任何值当"有"。位运算与逻辑运算的混用在代码评审中很常见,务必分清。

§2.4 逻辑与短路:返回的是操作数,不是布尔

这是 JS 与 C 最大的运算符差异:&& 与 || 返回两个操作数之一,不强制转布尔。C 里 `a && b` 是 0/1;JS 里:

```
// || :返回第一个"真值"操作数;都假则返回最后一个
console.log(0 || '默认值'); // '默认值' ← 经典默认值写法
console.log('' || 'fallback'); // 'fallback'(空串是假值)
console.log(null || 42); // 42
console.log(undefined || 1); // 1
console.log(3 || 99); // 3 ← 左边真就直接返回,右边不评估
console.log(0 || false); // false ← 都假,返回最后一个

// && :返回第一个"假值"操作数;都真则返回最后一个
console.log(1 && 2); // 2 ← 都真返回最后一个
console.log(0 && 2); // 0
console.log('' && 'x'); // ''
console.log(null && 'x'); // null
console.log(true && 'ok'); // 'ok'

// 短路:右边不执行
let called = 0;
false && (called = 1); // 右边不执行
console.log(called); // 0
true || (called = 2); // 右边不执行
console.log(called); // 0

// !! 双取反:转成真正的布尔
console.log(!!'abc'); // true
console.log(!!0); // false
```

2.4.1 假值全集:只有 6 个

JS 的真值转换只有 6 个假值: false、0、-0、""(空串)、null、undefined、NaN。其余一切都是真值,包括 '0'、'false'、[]、{}。这与 C 的"0 假非 0 真"完全不同——C 里任何非零整数都是真,JS 里 '0' 是真、[] 也是真。

```
const falsy = [false, 0, -0, '', null, undefined, NaN];
for (const v of falsy) {
  console.log(String(v).padEnd(9), '->', Boolean(v)); // 全部 false
}
console.log(Boolean('0')); // true ← 字符串 '0' 不是假值!
console.log(Boolean([])); // true
console.log(Boolean({})); // true
console.log(Boolean('false')); // true
console.log(Boolean(Infinity)); // true
```

2.4.2 空值合并 ?? :只有 null/undefined 才触发

?? 与 || 的区别: || 对所有假值生效,?? 只对 null/undefined 生效。处理"数值 0 也是合法配置"的场景必须用 ??:

```
const retries = 0;
console.log(retries || 3);    // 3 ← 坑!0 被 || 吞掉了
console.log(retries ?? 3);    // 0 ← 正确,0 是合法值

const name = '';
console.log(name || '匿名');  // '匿名'
console.log(name ?? '匿名');  // '' ← 空串也是合法值

// 注意:?? 不能与 && || 混用不括号
// console.log(1 ?? 2 || 3); // ❌ SyntaxError,必须 (1 ?? 2) || 3
```

§2.5 比较:=== 与 == 的完整真相

2.5.1 严格相等 ===:生产唯一选择

=== 不做类型转换,类型不同直接不相等。与 C 的 **==** 直觉一致,只是把"类型必须匹配"提到了第一位。判空、判枚举、判结果一律用 **===**。两个坑:NaN 永远不等于任何值(含自身);对象比较的是引用不是内容(和 C 的结构体按位比较不同)。

```
console.log(1 === 1);          // true
console.log(1 === '1');        // false ← 类型不同
console.log('1' === '1');      // true
console.log(NaN === NaN);       // false ← 唯一的不自等值
console.log(0 === -0);          // true(=== 不区分正负零)
console.log({} === {});         // false ← 不同对象,引用不同
const a = { n: 1 };
const b = a;
console.log(a === b);           // true ← 同一引用
console.log(null === undefined); // false
```

2.5.2 宽松相等 ==:你真要用它,先把这张表背下来

== 会先做类型转换再比较。它存在完全是因为历史原因,生产代码评审几乎一律禁止使用。但读老代码时必须能看懂,所以列出核心规则与著名例子:

```
// 数字 vs 字符串:字符串转数字后比较
console.log(0 == '0');           // true ← 著名坑
console.log(1 == '1');           // true
console.log('' == 0);            // true ← 空串转 0

// 布尔参与:先转数字(true=1, false=0)
console.log(true == 1);          // true
console.log(false == 0);         // true
console.log(true == 2);          // false
console.log('true' == true);     // false ← 字符串 'true' 不是数字,转 NaN

// null 与 undefined:只与彼此相等
console.log(null == undefined); // true
console.log(null == 0);          // false ← 著名坑,null 不转 0!
console.log(undefined == 0);     // false

// 对象与原始值:对象先转原始值(toString/valueOf)
console.log([] == '');           // true ← [] 转成 ''
console.log([] == 0);            // true
console.log([1] == 1);           // true
console.log([1, 2] == '1,2');    // true
console.log([] == ![]);          // true ← 最著名的鬼故事!
console.log('' == false);        // true

// 关系比较的坑:null >= 0 为 true 而 null > 0 为 false
console.log(null > 0);            // false(null 转 0,0 > 0 为假)
console.log(null == 0);          // false(== 特殊规则,不转换)
console.log(null >= 0);          // true ← 无解之谜!>= 里 null 转 0
console.log(undefined >= 0);     // false(undefined 转 NaN)
console.log(NaN >= 0);           // false(NaN 与任何比较都是 false)
```

★ 必须记住的结论

① 全部用 ===,除非你有意做 `x == null` 的判空(它同时匹配 null 与 undefined,这是 == 唯一的生产用法);② 关系比较(> < >= <=)只对 number 使用,字符串比较是字典序;③ NaN 永远不比;④ 判相等永远不用 == 处理对象。

2.5.3 Object.is:比 === 更严格

```
console.log(Object.is(NaN, NaN)); // true ← 唯一能判 NaN 相等的方法(其实 Number.isNaN 也行)
console.log(Object.is(0, -0));    // false ← 区分正负零
console.log(Object.is(1, '1'));   // false
console.log(Object.is({}, {}));    // false
```

§2.6 赋值、解构与展开

2.6.1 复合赋值

```
let x = 10;
x += 5;    // 15
x -= 3;    // 12
x *= 2;    // 24
x /= 4;    // 6
x %= 4;    // 2
x **= 3;   // 8,指数复合赋值
x <<= 1;   // 16,位移复合赋值
x >>= 2;   // 4
x &= 3;    // 0(按位与)
x |= 8;    // 8(按位或)
console.log(x);
```

2.6.2 解构赋值:C 没有对应物

```
// 数组解构:一次性取出多个值
const point = [3, 7];
const [px, py] = point;
console.log(px, py);           // 3 7

// 交换变量(不需要临时变量,没有 C 的 tmp 写法)
let a2 = 1, b2 = 2;
[a2, b2] = [b2, a2];
console.log(a2, b2);           // 2 1

// 对象解构:按属性名取值
const user = { id: 1, name: '韩梅梅', age: 17 };
const { name, age } = user;
console.log(name, age);        // 韩梅梅 17

// 重命名 + 默认值
const { name: userName = '匿名', city = '北京' } = user;
console.log(userName, city);    // 韩梅梅 北京

// 剩余元素 ...
const [first, ...rest] = [10, 20, 30, 40];
console.log(first, rest);       // 10 [ 20, 30, 40 ]
```

2.6.3 展开运算符 ...

```
// 数组展开:合并数组(替代 C 的 memcpy 手写循环)
const arr1 = [1, 2];
const arr2 = [3, 4];
const merged = [...arr1, ...arr2];
console.log(merged);           // [ 1, 2, 3, 4 ]

// 数组拷贝(浅拷贝)
const copy = [...merged];
console.log(copy === merged);  // false, 是新的数组

// 对象展开:合并/覆盖属性(浅拷贝)
const base = { host: 'localhost', port: 3000 };
const cfg = { ...base, port: 8080, debug: true };
console.log(cfg);              // { host: 'localhost', port: 8080, debug: true }

// 函数调用展开
console.log(Math.max(...[3, 9, 5])); // 9, 替代 apply
```

⚠ 常见误区 2.4:展开是浅拷贝

`{...obj}` 只拷贝一层,嵌套对象仍然是共享引用。修改拷贝后的嵌套对象会同时影响原对象——与 C 的 `memcpy` 结构体完全不同的语义。深拷贝的正确姿势见第14章(`structuredClone`)。

§2.7 三目运算符 ?:

C 的三目在 JS 中完全保留,且因为表达式风格流行,使用频率远高于 C。生产风格建议:单层三目可读性很好,嵌套三目禁止(可读性灾难,改用函数或 if)。

```
const score = 85;
const level = score >= 60 ? '及格' : '不及格';
console.log(level);           // 及格

// 三目是"表达式",可以返回值参与后续运算(与 C 一致)
const price = 100;
const pay = price > 50 ? price * 0.9 : price;
console.log(pay);             // 90

// ❌ 禁止嵌套三目:score>90?'A':score>80?'B': 'C'
// 应改写为:
function levelOf(s) {
  if (s > 90) return 'A';
  if (s > 80) return 'B';
  return 'C';
}
console.log(levelOf(85));      // B
```

§2.8 优先级与求值顺序

JS 的优先级表与 C 高度相似,只需记住几个差异点。下面按优先级从高到低列出常用运算符(完整表见 MDN):

优先级	运算符	与 C 的差异
最高	() 分组、[] 下标、. 属性、?. 可选链、new	新增 ?.
2	++ -- 一元 - + ! ~ typeof	一元 + 是转数字
3	** 指数	C 没有;注意它优先于负号
4	* / %	同 C
5	+ -	+ 可能是拼接!
6	<< >> >>>	新增 >>>
7	< <= > >= in instanceof	新增 in/instanceof
8	== != === !==	新增 === !==
9	&	同 C
10	^	同 C
11		同 C
12	&&	返回操作数
13		返回操作数
14	??	C 没有,不能与 12/13 混用不括号
15	? :	同 C
16	= += -= ...	同 C
最低	, 逗号	同 C

★ 求值顺序:C 的 UB,JS 的规定

C 的 `f() + g()` 谁先调用是未定义行为;JS 规定 从左到右。另外 JS 没有序列点问题, `i = i++` 的后果是确定的(虽然仍然是无意义代码)。生产建议:与 C 相同,一行表达式不要写两个副作用。

```
// 优先级实战
console.log(2 ** 3 ** 2);    // 512 ← ** 是右结合!3**2=9,2**9=512
console.log(-2 ** 2);        // SyntaxError ← 必须写 -(2**2)
console.log(-(2 ** 2));      // -4
console.log(3 + 4 * 2);      // 11,乘法先
console.log((3 + 4) * 2);    // 14,括号
console.log(1 < 2 == true);   // true:1<2 是 true,true==true
console.log(0 & 1 || 2);      // 2:位运算优先于逻辑运算
// 口诀:括号永远是最可靠的自文档—不要靠背表
```

§2.9 特色运算符:可选链 ?. 与 in/instanceof

2.9.1 可选链 ?.:C 没有的安全访问

```
const resp = { data: { user: { name: '李雷' } } };
```

// C 写法:每层都要判 NULL(或调用方保证非空)
// JS 老写法:
const n1 = resp && resp.data && resp.data.user && resp.data.user.name;
// JS 现代写法:
const n2 = resp?.data?.user?.name;
console.log(n2); // 李雷
console.log(resp?.data?.list?.[0]?.id); // undefined,中间断链不报错
console.log(user?.name ?? '匿名'); // 与 ?? 配合是生产标配

// 注意:?. 只短路"属性访问",不短路后面的方法调用
// resp.missing.hello() ← 报错;resp.missing?.hello() ← undefined

2.9.2 in 与 instanceof

```
const obj = { a: 1, b: undefined };  
console.log('a' in obj); // true  
console.log('b' in obj); // true ← 属性存在但值是 undefined  
console.log('c' in obj); // false  
console.log(obj.c === undefined); // true,in 能区分"属性不存在"与"属性为 undefined"
```



```
console.log([] instanceof Array); // true  
console.log({} instanceof Object); // true  
console.log(/re/ instanceof RegExp); // true  
// 跨 realm(iframe/worker)时 instanceof 会失效,生产用 Array.isArray  
console.log(Array.isArray([])); // true
```

工程建议 2.1:表达式也要自文档化

三条规定:① 逻辑与位运算混写必须加括号;② 比较运算的操作数先显式转 number,杜绝隐式转换混入表达式;③ 复杂表达式拆成带名字的局部变量(`const isAdmin = user.role === 'admin'`),让代码评审者一眼看懂意图。

工程建议 2.2:判空统一风格

团队里统一三种判空:期望 null/undefined 都没有时用 `x == null`;期望"有实际值"时用 `x !== null && x !== undefined` 或 `x != null`;只关心真值(如布尔开关)时用 `if (x)`。ESLint 配置 `eqeqeq` 规则可以强制 `===`,把 `==` 限制在判空场景。

本章速查表

主题	速查要点
除法	/ 永远是浮点除法:5/2=2.5;整数除法用 Math.floor(a/b)
取模	结果符号跟随被除数,与 C99 一致;-5%2=-1
溢出	number 溢出得 Infinity,不崩溃;精确整数限 $\pm 2^{53}-1$
除零	1/0=Infinity,0/0=NaN,不报错,需自行检查
指数	2 ** 10 = 1024;右结合;-2**2 需写 -(2**2)
位运算	先转 32 位有符号整数:0xFFFFFFFF 0 = -1;大整数用 BigInt
>>>	无符号右移,高位补 0;C 没有
假值	仅 6 个:false 0 -0 '' null undefined NaN
&& /	返回操作数原值; 取第一个真值, 都假取最后;短路
??	只对 null/undefined 生效;不能与 && 混用无括号
===	无转换,生产唯一选择;NaN 不自等;对象比引用
== 的坑	0=='0'、''==0、[]==![]、null==0 为 false、null>=0 为 true
判空	x == null 同时匹配 null/undefined,唯一推荐的 == 用法
解构	const [a,b]=arr; const {name}=obj; 可交换变量、取默认值
展开	[...arr] { ...obj } 都是浅拷贝;合并数组/对象最常用
可选链	a?.b?.c 安全取值,与 ?? 搭配是生产标配
求值顺序	严格从左到右(与 C 的 UB 不同)

最小必会清单

- 能说出 5/2、5%2、-5%2、0/0、1/0 的值
- 能说出 + 在字符串与数字混用时变成拼接的规则,并写出规避写法
- 能说出 0xFFFFFFFF|0 的结果并解释 32 位机制
- 能说出 && || 的返回值规则,并写出"默认值"与"条件执行"两种惯用法
- 能列出全部 6 个假值,并解释为什么 '0' 与 [] 是真值
- 能说出 0=='0'、null>=0、[]==![]、null==undefined 四个结果与原因
- 能用 ?? 处理"0 是合法配置"的场景
- 能写出数组/对象解构、展开合并、剩余参数三种代码
- 能说出 ** 的右结合性并写出正确的负底数写法
- 能用可选链与 ?? 重写 5 层嵌套的对象取值

自测题

Q 1. 写出输出:console.log(5/2, 5%2, -5%2, 2**3**2)

✅ 参考答案

2.5、1、-1、512。5/2 是浮点除法;取模符号随被除数;** 右结合:3**2=9 再 2**9=512。

Q 2. 写出输出:console.log(0 && 'a', 1 && 'a', 0 || 'b', 1 || 'b', null ?? 'c', 0 ?? 'c')

✅ 参考答案

0、'a'、'b'、1、'c'、0。&& 返回第一个假值或最后一个值;|| 返回第一个真值或最后一个值;?? 只对 null/undefined 触发。

Q 3. 下面每个表达式的值是什么:0=='0';null>=0;[]==![];null==undefined;'true'==true

✅ 参考答案

true、true、true、true、false。0 与 '0' 比较时字符串转数字;null>=0 中 null 转 0 得 true(而 null==0 是 false);[] 转 "" 后 ![] 转 false,"-==0 两边都转 0;null 与 undefined 互相相等;'true' 转数字是 NaN,与 1 不等。

Q 4. 为什么 const config = {retries: 0}; config.retries || 3 得到 3?正确写法是什么?

✅ 参考答案

0 是假值,|| 返回 3。正确: config.retries ?? 3,?? 只对 null/undefined 触发,保留 0。

Q 5. 用解构与展开把 [1,2] 与 [3,4] 合并并取出第一个元素,再把 {a:1,b:2} 与 {b:9,c:3} 合并。

✅ 参考答案

const [first, ...rest] = [...[1,2], ...[3,4]] (first=1, rest=[2,3,4]); const merged = {...{a:1,b:2}, ...{b:9,c:3}} 得 {a:1,b:9,c:3},后面的 b 覆盖前面的。

Q 6. 判断以下代码的非法点:2 ** -1;console.log(1 ?? 2 || 3);-(2**2)

✅ 参考答案

2 ** -1 合法,得 0.5(指数可为负);1 ?? 2 || 3 抛 SyntaxError,?? 不能与 || 混用不括号; -(2**2) 合法得 -4。

章末实验题:表达式求值器

实验 2:简易表达式求值器

实验目标:实现一个支持加、减、乘、除、取模、幂、括号与一元负号的表达式计算器,用运算符优先级与短路求值实现。

实验要求:

- 任务 1(覆盖:算术运算符、优先级):实现 $+$ $-$ $*$ $/$ $\%$ $**$ 的优先级递归下降解析
- 任务 2(覆盖:括号与一元负号、求值顺序):支持 $($ 与 $)$ 与 $-x$ 一元运算,注意 $**$ 右结合
- 任务 3(覆盖:解构、展开、数组方法):用数组栈管理数字与运算符,或函数递归管理
- 任务 4(覆盖:除法/取模语义、NaN/Infinity): $5/2$ 输出 2.5 ;除零输出 Infinity ;非法输入输出错误提示
- 任务 5(覆盖:逻辑短路、比较、 $??$):空输入用 $??$ 给默认表达式;结果用 $===$ 判断合法性
- 任务 6(覆盖:位运算与自增):附加命令 `bit 7 3` 输出 $7\&3$ 、 $7|3$ 、 7^3 、 $7<<1$ 并演示 32 位溢出($1<<31$ 为负数)

实验环境:Node.js 20+,命令行参数输入表达式: `node calc.js "2 + 3 * 4"`。

第一步 写词法拆分:用正则或手写扫描把字符串拆成 token(数字、运算符、括号)

第二步 写递归下降:`parseExpr`(处理 $+$ $-$)、`parseTerm`(处理 $*$ $/$ $\%$)、`parseFactor`(处理 $**$ 与一元 $-$ 与括号),按优先级逐层调用

第三步 理右结合:幂运算写成右递归或循环内向左收集

第四步 加演示:解析 `bit` 命令并逐行打印位运算结果,直观看到 32 位截断

✅ 参考答案要点

核心:token 数组 + 递归下降。parseExpr→parseTerm→parsePower→parseAtom 四层,幂层用右结合循环。共约 90 行,覆盖本章全部运算符。

```
// calc.js — Node.js 20+ 运行:node calc.js "2 + 3 * 4" 或 node calc.js "bit 7 3"
'use strict';

// 1. 词法分析:拆成 token(覆盖:正则、字符串 API、模板字符串)
function tokenize(src) {
  const tokens = [];
  const re = /\s*([0-9]+(?:\.[0-9]+)?|[\+\-\*\/%()]\s*\s*)/g;
  let m;
  while ((m = re.exec(src)) !== null) {
    tokens.push(m[1]);
  }
  if (tokens.length === 0) return null;
  return tokens;
}

// 2. 递归下降解析器(覆盖:优先级、右结合、一元负号、括号)
class Parser {
  constructor(tokens) {
    this.tokens = tokens;
    this.pos = 0;
  }
  peek() { return this.tokens[this.pos]; }
  next() { return this.tokens[this.pos++]; }

  // 表达式:加/减(最低优先级)
  parseExpr() {
    let left = this.parseTerm();
    while (this.peek() === '+' || this.peek() === '-') {
      const op = this.next();
      const right = this.parseTerm();
      left = { op, left, right };
    }
    return left;
  }

  // 项:乘/除/取模
  parseTerm() {
    let left = this.parsePower();
    while (this.peek() === '*' || this.peek() === '/' || this.peek() === '%') {
      const op = this.next();
      const right = this.parsePower();
      left = { op, left, right };
    }
    return left;
  }

  // 幂:右结合(2**3**2 = 2**(3**2))
  parsePower() {
    const base = this.parseAtom();
    if (this.peek() === '**') {
      this.next();
      const exp = this.parsePower(); // 右递归实现右结合
      return { op: '**', left: base, right: exp };
    }
    return base;
  }
}
```

```

}

// 原子:数字、括号、一元负号
parseAtom() {
  const t = this.peek();
  if (t === '(') {
    this.next();
    const e = this.parseExpr();
    this.next(); // 吃掉 ')'
    return e;
  }
  if (t === '-') {
    this.next();
    return { op: '-', left: { num: 0 }, right: this.parseAtom() };
  }
  this.next();
  return { num: Number(t) };
}
}

```

```

// 3. 求值(覆盖:NaN/Infinity、===、Number.isFinite)
function evaluate(node) {
  if ('num' in node) return node.num;
  const l = evaluate(node.left);
  const r = evaluate(node.right);
  switch (node.op) {
    case '+': return l + r;
    case '-': return l - r;
    case '*': return l * r;
    case '/': return l / r; // 浮点除法,5/2=2.5
    case '%': return l % r; // 符号随被除数
    case '**': return l ** r; // 右结合已在解析阶段处理
    default: return NaN;
  }
}
}

```

```

// 4. 主流程(覆盖:?? 默认值、process.argv、模板字符串、Infinity 提示)
function main() {
  const input = process.argv[2] ?? '1 + 2 * 3';
  if (input.startsWith('bit')) {
    const [_, a, b] = input.split(/\s+/);
    const x = Number(a), y = Number(b);
    console.log(`${x} & ${y} = ${x & y}`);
    console.log(`${x} | ${y} = ${x | y}`);
    console.log(`${x} ^ ${y} = ${x ^ y}`);
    console.log(`1 << 31 = ${1 << 31}(32 位溢出成负数!)`);
    console.log(`(1 << 31) >>> 0 = ${(1 << 31) >>> 0}`);
    return;
  }
  const tokens = tokenize(input);
  if (tokens === null) {
    console.log('错误:空表达式');
    return;
  }
  const ast = new Parser(tokens).parseExpr();
}

```

```
const result = evaluate(ast);
if (Number.isFinite(result)) {
  console.log(`${input} = ${result}`);
} else if (result === Infinity) {
  console.log(`${input} = Infinity(除零)`);
} else {
  console.log(`${input} = NaN(非法运算)`);
}
}

main();
```

设计说明:递归下降天然体现优先级:函数调用层次 = 优先级层次,parsePower 的右递归实现 ** 右结合;一元负号用 0 - x 表示;除零在求值层产生 Infinity,主流程用 Number.isFinite 显式提示,避免静默传播。运行示例: `node calc.js "2 + 3 * 4"` 输出 14, `node calc.js "(2 ** 3) ** 2"` 输出 64, `node calc.js "2 ** 3 ** 2"` 输出 512。

下一章预告:第3章 流程控制 —— 真值判断的 if、严格 === 的 switch、for...of 与 for...in 的选择,以及循环里 var 的经典闭包 bug。

第3章 流程控制

建议学时:2-4 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 if 条件的"真值"规则:JS 不要求布尔,6 个假值之外都是真
- 掌握 switch 与 C 的差异:严格 ===、无默认 fallthrough、共享 case
- 熟练使用四种循环:for、while、do-while、for...of,并分清 for...in 的用途
- 掌握带标签的 break/continue,理解它是 C 的 goto 的安全替代
- 彻底理解循环里 var 的经典闭包 bug 与 let 的块作用域修复
- 掌握数组遍历的现代写法:for...of、forEach、map/filter 的选择
- 理解"C 中怎么做→JS 中怎么做":循环变量作用域与迭代方式

💡 从 C 到 JavaScript:本章主题如何迁移

流程控制是两种语言最接近的部分:if/else、for、while、do-while、switch、break、continue 一应俱全,大括号风格都一致。但也藏着三处暗礁。

第一处暗礁是 **条件不要求是布尔**:C 里 `if (p)` 检查指针非空,JS 里 `if (x)` 检查"真值"——但空字符串、0、NaN、null、undefined 都是假,这在 C 里只有 0 是假。写 `if (name)` 判断"名字非空"在生产里是常见 bug 源:'0'、'false' 都是真值,空串才触发分支。

第二处暗礁是 **switch 的语义**:C 的 switch 只支持整型并默认 fallthrough;JS 的 switch 用 === 比较,支持任意类型,且不自动穿透(每个 case 结尾的 break 是惯例而非必需)。但 JS 保留了"共享 case"能力: `case 1: case 2:` 叠写仍合法。

第三处暗礁是 **循环与闭包**:C 的 `for (int i = 0; i < 3; i++)` 里 i 是块变量;JS 的 var 是函数作用域,循环结束时 i 只剩下最终值,回调里读 i 全部读到同一个值——这就是闻名天下的"循环里 var 的闭包 bug"。现代 JS 用 let 声明循环变量,彻底解决。

JS 还提供了 C 没有的迭代器遍历:for...of 按顺序取每个元素,不需要下标;数组还有 map/filter/reduce 等函数式遍历。读数组代码时,先判断它用哪种风格,再逐段理解。

📦 前置知识自查

- C 的 if/else 与悬挂 else(else 与最近的 if 结合)
- C 的 switch 语义:整型、fallthrough、break
- C 的 for/while/do-while 与 break/continue
- C 的 goto 与标签
- C 的循环变量作用域(块作用域)

本章学习路线

1. **第一阶段(if 与 switch):**读 §3.1–§3.2,重点记真值表与 switch 的 === 语义
2. **第二阶段(循环):**读 §3.3–§3.4,把每个循环例子在 REPL 跑一遍
3. **第三阶段(闭包 bug 与数组遍历):**读 §3.5–§3.7,亲手复现 var 闭包 bug 并修复
4. **第四阶段(实验):**完成章末"文本菜单 + 素数筛选 + 统计"
5. **验收标准:**能独立写出循环里用 let 的正确回调代码,并解释 var 为什么错

本章知识导图

- 第3章 流程控制
 - §3.1 if/else 与真值判断、悬挂 else
 - §3.2 switch:严格 ===、共享 case、与 C 对照
 - §3.3 循环:for / while / do-while / for...of / for...in
 - §3.4 break/continue 与标签
 - §3.5 goto 的替代:提前返回与标签 break
 - §3.6 循环变量捕获:var 闭包 bug 与 let 修复
 - §3.7 数组遍历三风格与现代写法、无限循环对照

§3.1 if / else:条件按真值判断

```
const score = 85;
if (score >= 90) {
  console.log('优秀');
} else if (score >= 60) {
  console.log('及格');
} else {
  console.log('不及格');
}

// 真值判断:JS 不要求条件为布尔
const name = '';
if (name) { // '' 是假值,不进分支
  console.log('有名字');
} else {
  console.log('名字为空');
}

const count = 0;
if (count) { // 0 是假值!这里常常是 bug
  console.log('有计数');
} else {
  console.log('计数为 0'); // 走到这里
}

// 生产建议:明确意图
if (count > 0) { // 明确:大于 0
  console.log('有计数');
}
```

⚠ 常见误区 3.1:if (arr.length) 与 if (arr) 完全不同

`if (arr)` 判断的是"arr 是否存在"(非 null/undefined),空数组 `[]` 是真值,恒进分支; `if (arr.length)` 判断"元素个数大于 0",空数组是 0(假)不进分支。两个意图都常见,写错就是线上 bug。同理 `if (obj.name)` 在 name 为 0 或 "" 时误判。

3.1.1 悬挂 else:C 的规则保留

与 C 相同,else 与最近的未配对 if 结合。生产规范建议任何 else 嵌套都加大括号,可读性优先。JS 没有语法问题,只有风格问题。

```
let x = 1, y = 2;
// 悬挂 else:else 属于内层 if(与 C 一致)
if (x === 1)
  if (y === 2) console.log('内层');
  else console.log('外层?不,还是内层的 else'); // 属于内层 if
// 上面代码等价于(不推荐,仅为解释规则):
if (x === 1) {
  if (y === 2) {
    console.log('内层');
  } else {
    console.log('还是内层的 else');
  }
}
```

§3.2 switch:严格 ===,没有隐式 fallthrough

对比项	C 的 switch	JS 的 switch
条件类型	整型/字符	任意值,用 === 严格比较
比较语义	按整数值	===(不转换类型)
fallthrough	默认穿透,漏 break 是经典 bug	默认不穿透(空 case 可共享)
default	可选,位置任意	同 C,习惯放最后
break 可省	不可省(除非故意穿透)	每 case 结尾 break 是惯例

```

function describe(color) {
  switch (color) {
    case 'red':
      return '红色';          // return 直接退出,不用 break
    case 'green':
      return '绿色';
    case 'blue':
      return '蓝色';
    default:
      return '未知颜色';
  }
}

console.log(describe('red'));    // 红色

// 共享 case:多个 case 执行同一段(与 C 的故意 fallthrough 相同)
function daysOf(month) {
  switch (month) {
    case 2:
      return 28;
    case 4:
    case 6:
    case 9:
    case 11:
      return 30;              // 4/6/9/11 共享
    default:
      return 31;
  }
}

console.log(daysOf(4));          // 30
console.log(daysOf(1));          // 31

// === 语义的坑:switch (x) 不转换类型
const n = '1';
switch (n) {
  case 1: console.log('数字 1'); break;    // 不会命中!n 是字符串
  case '1': console.log('字符串 1'); break; // 命中这里
  default: console.log('其他');
}

```

⚠ 常见误区 3.2:switch 忘了 break 会静默穿透吗?

JS 里忘写 break 确实会继续执行下一个 case 的语句(穿透),但每个 case 通常以 return/break 结束,且由于语言不强制,穿透在 JS 中比 C 更罕见。用 ESLint 的 no-fallthrough 规则可以静态拦截。生产建议:case 体内语句多时,用大括号包裹,声明局部变量时避免与穿透混淆。

工程建议 3.1:现代代码用"对象查找"替代 switch 长链

10 个 case 以上的 switch 建议改成对象查找(映射表),可读性更高、天然支持动态键:

```
const COLOR_CN = Object.freeze({
  red: '红色',
  green: '绿色',
  blue: '蓝色',
});
console.log(COLOR_CN['red'] ?? '未知颜色'); // 红色
```

§3.3 循环:四种形态与两种迭代

3.3.1 经典三循环:与 C 几乎相同

```
// for:与 C 完全一致
for (let i = 0; i < 5; i++) {
  process.stdout.write(i + ' ');
}
console.log(); // 0 1 2 3 4

// while
let j = 0;
while (j < 3) {
  process.stdout.write(j + ' ');
  j++;
}
console.log(); // 0 1 2

// do-while:至少执行一次(与 C 相同)
let k = 10;
do {
  console.log('执行一次:', k);
  k++;
} while (k < 3);
```

3.3.2 数组遍历的现代写法:for...of(ES2015)

```
const fruits = ['苹果', '香蕉', '橙子'];

// C 风格:下标遍历(JS 也支持)
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}

// 现代风格:for...of,直接取元素,无需下标
for (const f of fruits) {
  console.log(f);
}

// 需要下标时:用 entries()
for (const [i, f] of fruits.entries()) {
  console.log(`${i}: ${f}`);
}

// for...of 也能遍历字符串(按码元)与 Map/Set
for (const ch of 'abc') {
  console.log(ch); // a b c
}

const map = new Map([[ 'a', 1], [ 'b', 2]]);
for (const [key, value] of map) {
  console.log(key, value);
}
```

⚠ 常见误区 3.3:for...in 遍历数组的陷阱

for...in 遍历的是可枚举属性名(字符串),不是元素;它遍历数组得到的是 '0'、'1' 字符串下标,且会把数组上自定义的属性也遍历出来。生产规则:数组用 for...of,对象用 Object.keys/entries,for...in 基本不用(除非遍历普通对象的自身属性并接受原型链风险)。

```
const arr = ['a', 'b'];
arr.custom = 'extra';
for (const k in arr) {
  console.log(k); // 0 1 custom ← 把自定义属性也遍历了
}
for (const v of arr) {
  console.log(v); // a b ← 干净
}
```

§3.4 break / continue 与带标签跳转

C 的 break 只能跳出最近一层循环,JS 相同;但 JS 支持带标签的 break/continue,可以跳出任意多层——这是 C 的 goto 在现代代码里的安全替身(ESLint 规则也推荐标签而非 goto 风格跳转)。

```
// 双层循环找目标:跳出两层
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) {
      break outer;          // 直接跳出两层
    }
    console.log(`${i},${j}`);
  }
}
console.log('跳出两层');

// continue 标签:跳过外层循环的当前迭代
inner: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) {
      continue inner;       // 跳到外层下一轮,i 会 +1
    }
    console.log(`${i},${j}`);
  }
}

// continue 的常规用法:过滤
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) continue; // 跳过偶数
  process.stdout.write(i + ' ');
}
console.log(); // 1 3 5 7 9
```

§3.5 goto:JS 没有,但你有更好的工具

C 的 goto 用于:① 统一错误出口;② 跳出深层循环;③ 状态机跳转。JS 没有 goto 关键字,但三种场景都有现代替代:

C 的 goto 场景	JS 的替代
错误统一出口(err: cleanup; return)	try/finally(第9章)或提前 return
跳出深层循环	带标签 break / 提取成函数后 return
状态机跳转	while + switch 状态机模式,或状态对象

```
// 提取函数 + 提前 return,替代 goto 的"找出口"
function findIndex(matrix, target) {
  for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[i].length; j++) {
      if (matrix[i][j] === target) {
        return [i, j];          // 直接返回,天然跳出两层
      }
    }
  }
  return [-1, -1];             // 未找到
}

const m = [[1, 2], [3, 4]];
console.log(findIndex(m, 3)); // [ 1, 0 ]

// 状态机:while + switch(替代 goto 跳转)
function smallStateMachine() {
  let state = 'start';
  while (state !== 'end') {
    switch (state) {
      case 'start':
        console.log('开始');
        state = 'work';
        break;
      case 'work':
        console.log('工作中');
        state = 'end';
        break;
    }
  }
}

smallStateMachine();
```

§3.6 循环变量捕获:var 的经典闭包 bug

这是 JS 面试与生产事故的"老朋友"。C 里 `for (int i = 0; ...)` 的 `i` 每轮都是独立的块变量;JS 的 `var` 是函数作用域,整个循环只有一个 `i`,循环结束后 `i` 保留最终值。于是回调函数(如 `setTimeout`)读取 `i` 时,读到的是同一个最终值。

```
// ❌ 经典 bug:全部输出 3
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var:', i), 10);
}
// 输出:3 3 3

// ✅ 修复一:用 let(每轮迭代独立绑定)
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log('let:', j), 10);
}
// 输出:0 1 2

// ✅ 修复二:立即执行函数捕获(旧代码风格)
for (var k = 0; k < 3; k++) {
  (function (n) {
    setTimeout(() => console.log('IIFE:', n), 10);
  })(k);
}
// 输出:0 1 2
```

★ 重点结论:let 的循环语义是"每轮新绑定"

for 循环里 `let i` 每次迭代都会创建一个新的绑定,回调捕获的是那一轮的 `i`——这是规范明确规定的语义(与 C 的块变量直觉一致)。生产铁律:循环变量一律 `let`,回调里捕获循环变量时检查是否为每轮值。异步回调细节见第9章事件循环。

§3.7 数组遍历三风格与无限循环对照

3.7.1 三风格总览:下标 / 迭代 / 函数式

```
const nums = [1, 2, 3, 4, 5];

// 风格一:for 下标(需要 index 或提前退出时)
for (let i = 0; i < nums.length; i++) {
  if (nums[i] === 3) break;
}

// 风格二:for...of(只读遍历的首选)
let sum = 0;
for (const n of nums) sum += n;
console.log(sum);           // 15

// 风格三:函数式(map/filter/reduce,第4章详讲)
const doubled = nums.map((n) => n * 2);
const evens = nums.filter((n) => n % 2 === 0);
const total = nums.reduce((acc, n) => acc + n, 0);
console.log(doubled, evens, total);

// forEach:副作用遍历
nums.forEach((n) => process.stdout.write(n + ' '));
console.log();

// 生产选型:需要 break/continue/await → for 系列;纯遍历 → for...of;
// 转换/过滤/聚合 → map/filter/reduce;其他副作用 → forEach
```

3.7.2 无限循环写法对照

```
// C 经典:for(;;) 或 while(1)
// JS 对应:
let running = true;
let steps = 0;
while (true) {
  steps++;
  if (steps >= 3) break;    // 必须要有退出条件!
}
console.log(steps);        // 3

// Node 服务器主循环常写成:事件循环驱动,不需要自己写死循环(见第9章)
```

3.7.3 遍历中修改数组:JS 没有"迭代器失效",但有新坑

C++ 在 vector 遍历中 erase 会使迭代器失效;JS 的数组没有这个问题,但 for...of 遍历中 push 会无限循环,删元素会跳过元素——**生产规则:遍历中不要增删原数组**,先过滤出新数组再遍历。

```
// 安全做法:过滤出新数组
const raw = [1, 2, 3, 4, 5, 6];
const kept = raw.filter((n) => n % 2 === 1); // 保留奇数
console.log(kept); // [ 1, 3, 5 ]
console.log(raw); // [ 1, 2, 3, 4, 5, 6 ] 原数组不变

// ❌ 错误示范:遍历中 splice
// for (let i = 0; i < raw.length; i++) {
//   if (raw[i] % 2 === 0) raw.splice(i, 1); // 下标错乱,跳过元素
// }
```

🚀 工程建议 3.2:循环里的大对象操作注意性能

循环体内避免:重复拼接字符串(用数组收集)、重复访问深层属性(先取局部变量)、每次迭代创建闭包(提出来)。10 万级循环里, `const t = obj.a.b` 提出来能省一半时间;字符串收集后 `join` 比 `+=` 快一个数量级。用 Node 的 `--prof` 或浏览器 Performance 面板定位热点后再优化,不要盲目微优化。

本章速查表

主题	速查要点
if 条件	按真值:仅 6 个假值(false 0 -0 " null undefined NaN)
if (arr)	判断"数组存在";if (arr.length) 判断"有元素",别混
悬挂 else	与 C 相同:else 归最近的 if;生产全加大括号
switch 比较	用 ===,不转类型:'1' 不命中 case 1
switch fallthrough	默认不穿透;case 4: case 5: 共享合法
for	与 C 相同;循环变量用 let
do-while	至少执行一次,与 C 相同
for...of	数组/字符串/Map/Set 的值遍历,首选
for...in	遍历可枚举属性名;数组别用;生产基本不用
break/continue 标签	outer: for(...) 后 break outer;替代 goto 跳层
goto	没有;提前 return / 标签 break / 状态机替代
var 闭包 bug	var 是函数作用域,回调全读到最终值;let 每轮新绑定
遍历风格	转换 map;筛选 filter;聚合 reduce;副作用 forEach;要 break 用 for
遍历中改数组	禁止;先 filter 出新数组
无限循环	while(true) + break;事件循环由运行时驱动

最小必会清单

- 能列出 6 个假值,并说出 if (count) 与 if (count > 0) 的语义差异
- 能写出 switch 的 === 语义案例,并解释 '1' 不命中 case 1

- 能写出共享 case 的 switch(4/6/9/11 月共享 30 天)
- 能说出 for...of 与 for...in 的区别与各自适用场景
- 能写出带标签的 break 跳出双层循环
- 能说出 JS 没有 goto 后三种替代方案
- 能复现 var 闭包 bug 并用 let 修复,并解释原因
- 能写出 for...of、entries()、forEach、map/filter/reduce 五种遍历
- 能解释为什么遍历中 splice 会跳过元素,并给出安全写法
- 能用 while(true) + break 写出带退出条件的循环

自测题

Q 1. 写出输出:`const a = []; if (a) console.log('A'); if (a.length) console.log('B')`

✓ 参考答案

只输出 A。[] 是真值(对象都是真值),a.length 是 0(假值)。

Q 2. 写出输出:`const x = '2'; switch (x) { case 2: console.log('数字'); break; case '2': console.log('字符串'); break; default: console.log('其他'); }`

✓ 参考答案

输出"字符串"。switch 用 ===,字符串 '2' 不等于数字 2。

Q 3. 下面的代码输出什么?为什么?`for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); }`

✓ 参考答案

输出 3 3 3。var 是函数作用域,只有一个 i,循环结束后 i=3,回调执行时全部读到 3。改成 let 输出 0 1 2。

Q 4. 如何用带标签的 break 从双层循环中跳出?请给出代码。

✓ 参考答案

`outer: for (...) { for (...) { if (条件) break outer; } },break outer 直接结束外层循环。`

Q 5. for...of 遍历数组 arr 时,arr.custom = 1 会被遍历到吗?for...in 呢?

✓ 参考答案

for...of 不会(只取元素);for...in 会(遍历所有可枚举字符串属性名,包括 custom)。

Q 6. 写一个 do-while 循环,打印 1 到 3,并说明它和 while 的区别。

✓ 参考答案

`let i = 1; do { console.log(i); i++; } while (i <= 3);`。do-while 先执行循环体再判断,至少执行一次。

章末实验题:文本菜单与素数筛选统计器

实验 3:文本菜单与素数筛选统计器

实验目标:实现一个命令行交互工具:菜单驱动(1 筛选素数 / 2 统计数组 / 3 矩阵查找 / 0 退出),强制覆盖全章流程控制知识点。

实验要求:

- 任务 1(覆盖:if/else、真值判断):菜单输入用真值规则防御空输入,选项用 if 链分发
- 任务 2(覆盖:switch、共享 case):选项处理改用 switch,支持 '1'/'2'/'3' 数字与 'q' 共享退出路径
- 任务 3(覆盖:for、while、break/continue):素数筛选用双重 for + continue;读取输入用 while 循环
- 任务 4(覆盖:do-while):统计子菜单用 do-while 保证至少执行一次
- 任务 5(覆盖:带标签 break、for...of、entries):矩阵查找用标签 break 提前退出;遍历统计用 for...of 与 entries
- 任务 6(覆盖:var 闭包 bug 与 let):在工具内演示 setTimeout 捕获循环变量时 var 与 let 的差异
- 任务 7(覆盖:对象查找替代 switch 长链):把等级映射表用对象查找实现

实验环境:Node.js 20+,交互式运行 `node menu.js`;或把输入重定向为文件。

第一步 写纯函数:isPrime(n)、filterPrimes(list)、matrixFind(matrix, target)、statList(list),全部与 UI 解耦

第二步 写菜单循环:while(true) + switch(choice),'q' 退出;输入用 readline 的 question 封装成 await

第三步 后写演示区:展示标签 break、var/let 闭包差异、对象查找

✅ 参考答案要点

核心:纯函数(素数/统计/查找)+ while(true) 菜单循环 + switch 分发 + readline 异步输入。约 120 行,覆盖全章全部知识点。

```

// menu.js — Node.js 20+ 运行:node menu.js
'use strict';
const readline = require('node:readline/promises');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

// 1. 素数判断:for + continue(覆盖:for、continue、真值)
function isPrime(n) {
  if (n < 2) return false;
  if (n === 2) return true;
  if (n % 2 === 0) return false;
  for (let d = 3; d * d <= n; d += 2) {
    if (n % d === 0) return false;
  }
  return true;
}

// 2. 筛选素数:for...of + 数组 push(覆盖:for...of、数组)
function filterPrimes(list) {
  const out = [];
  for (const n of list) {
    if (isPrime(n)) out.push(n);
  }
  return out;
}

// 3. 矩阵查找:带标签 break(覆盖:标签、break)
function matrixFind(matrix, target) {
  outer: for (let i = 0; i < matrix.length; i++) {
    for (let j = 0; j < matrix[i].length; j++) {
      if (matrix[i][j] === target) {
        return [i, j];
      }
    }
  }
  return [-1, -1];
}

// 4. 统计:do-while + while(覆盖:do-while、while、break)
function statList(list) {
  let sum = 0;
  let max = list.length > 0 ? list[0] : 0;
  let count = 0;
  let i = 0;
  if (list.length > 0) {
    do {
      const v = list[i];
      sum += v;
      if (v > max) max = v;
      count++;
      i++;
    } while (i < list.length);
  }
}

```

```
    return { sum, max, count, avg: count === 0 ? 0 : Math.round((sum / count) * 100) / 100 };
}
```

// 5. 对象查找替代 switch 长链(覆盖:对象映射、??、真值)

```
const GradeCN = Object.freeze({
  A: '优秀',
  B: '良好',
  C: '及格',
  D: '不及格',
});
function gradeOf(avg) {
  if (avg >= 90) return GradeCN.A ?? '优秀';
  if (avg >= 75) return GradeCN.B;
  if (avg >= 60) return GradeCN.C;
  return GradeCN.D;
}
```

// 6. 演示 var 闭包 bug 与 let 修复(覆盖:闭包、let/var)

```
function closureDemo() {
  const results = [];
  for (var v = 0; v < 3; v++) {
    results.push(() => v); // 三个函数共享同一个 v
  }
  for (let l = 0; l < 3; l++) {
    results.push(() => l); // 每轮独立绑定
  }
  const nums = results.map((f) => f());
  console.log('var 段全为 3,let 段 0..2:', nums.join(','));
}
```

// 7. 菜单主循环:while + switch(覆盖:switch、break、while、真值)

```
async function main() {
  const data = [10, 23, 47, 51, 97, 4, 13];
  const matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
  console.log('==== 流程控制实验菜单 =====');
  while (true) {
    const choice = (await rl.question(
      '[1]素数筛选 [2]数组统计 [3]矩阵查找 [4]闭包演示 [q]退出\n> '
    )).trim().toLowerCase();
    switch (choice) {
      case '1': {
        console.log('素数:', filterPrimes(data).join(' '));
        break;
      }
      case '2': {
        const st = statList(data);
        console.log(`和=${st.sum} 最大=${st.max} 个数=${st.count} 平均=${st.avg}`);
        console.log(`平均等级:${gradeOf(st.avg)}`);
        break;
      }
      case '3': {
        const pos = matrixFind(matrix, 6);
        if (pos[0] === -1) {
          console.log('未找到');
        } else {

```

```

        console.log(`找到 6 在 (${pos[0]},${pos[1]})`);
    }
    break;
}
case '4':
    closureDemo();
    break;
case 'q':          // 共享退出路径
case 'Q':
    console.log('再见');
    rl.close();
    return;
default:
    console.log('未知选项,重新输入');
}
}
}

main().catch((err) => {
    console.error('程序异常:', err);
    rl.close();
});

```

设计说明:纯函数与 UI 分离便于测试;菜单用 while(true) + switch(choice) 结构,case 内大括号包裹声明局部变量避免穿透歧义;输入 trim().toLowerCase() 后用真值检查防御空输入;闭包演示把 var/let 差异可视化;gradeOf 用对象查找 + ?? 默认值。删除注释中的 `for (var v ...)` 用 let 即可观察修复效果。

下一章预告:第4章 函数与方法 —— 函数是一等公民、this 绑定四规则、闭包与高阶函数,理解这些才能真正读懂 JS 代码。

第4章 函数与方法

建议学时:4-6 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解"函数是一等公民":函数可赋值、传参、返回,远超 C 的函数指针
- 掌握参数传递语义:值传递与"对象按引用共享"的区别,函数内修改是否影响外部
- 彻底掌握 this 的四种绑定规则(默认/隐式/显式/new)与箭头函数的特殊规则
- 熟练使用 call/apply/bind 并理解三者差异
- 理解闭包的机制与生产用途,以及它与 C 函数指针的本质区别
- 掌握高阶函数 map/filter/reduce 与回调风格
- 理解"C 中怎么做→JS 中怎么做":main 入口、参数与返回的差异

💡 从 C 到 JavaScript:本章主题如何迁移

C 里函数是"代码段":有地址,有调用约定,有函数指针可以传来传去。JS 里函数更进一步,是 **第一等公民**:函数就是对象,可以存进数组、当参数传给另一个函数、被另一个函数返回、还能"记住"定义时的环境——后者正是闭包,是 C 的函数指针做梦都做不到的能力。

C 的函数指针用 `int (*cmp)(const void*, const void*)` 给 `qsort` 传比较器;JS 用 `arr.sort((a, b) => a - b)`,语法简洁一个数量级,而且回调能捕获外层变量(闭包),C 只能靠额外参数传递上下文。

C 里没有 this:结构体成员函数用 `obj.method(&obj, ...)` 显式传自己。JS 的 this 是"调用时决定"的隐式参数,灵活但反直觉——四种绑定规则必须逐个记牢,这是本章最重要的部分。

参数传递也完全不同:C 全部传值,想改外部就传指针;JS 里原始类型传值(改不动外部),对象传"引用的副本"(能改到外部对象的属性,但重新赋值不影响外部)。这个"对象共享"语义是生产 bug 高发区,必须建立肌肉记忆。

最后,JS 没有 C 那样的 main 入口:Node 程序从模块顶层开始执行,浏览器从事件驱动开始。但模块可以导出一个 `main()` 函数并调用它,风格上向 C 靠拢,可读性更好。

📦 前置知识自查

- C 的函数定义、声明与调用,形参实参、return
- C 的传值与传地址(`int* p`),修改外部变量的两种姿势
- C 的函数指针:typedef 函数指针类型、回调函数
- C 的作用域:局部变量在函数退出后销毁
- C 的 static 局部变量(生命周期延长)——JS 闭包与之形似神不同

本章学习路线

1. **第一阶段(语法与传参)**:读 §4.1–§4.3,把所有示例在 REPL 跑通
2. **第二阶段(this,本章核心)**:读 §4.4–§4.6,自己写"四种绑定"的最小例子,务必亲手验证
3. **第三阶段(闭包与高阶函数)**:读 §4.7–§4.8,理解闭包"捕获环境"的机制
4. **第四阶段(实验)**:完成章末"可配置统计管线"
5. **验收标准**:不看资料,能说出四种 this 绑定场景与箭头函数的例外,能写出一个闭包计数器

本章知识导图

- 第4章 函数与方法
 - §4.1 定义与调用:声明式/表达式/箭头函数
 - §4.2 参数传递:值传递与对象共享、默认参数、rest、arguments
 - §4.3 返回值与多返回值(解构)
 - §4.4 this 绑定四规则:默认/隐式/显式/new
 - §4.5 call/apply/bind 详解
 - §4.6 箭头函数:this/arguments 的词法绑定
 - §4.7 闭包:捕获环境、计数器/工厂/私有状态
 - §4.8 高阶函数:map/filter/reduce、回调
 - §4.9 递归、入口与组织

§4.1 定义与调用:三种写法

```
// 写法一:函数声明(可提升,与 C 的函数声明类似)
function add(a, b) {
  return a + b;
}
console.log(add(1, 2));           // 3

// 写法二:函数表达式(不可提升,必须定义后使用)
const sub = function (a, b) {
  return a - b;
};
console.log(sub(5, 2));           // 3

// 写法三:箭头函数(ES2015,无 this/arguments,见 §4.6)
const mul = (a, b) => a * b;
console.log(mul(3, 4));           // 12

// 函数是对象:可以当属性、存数组、传参
const ops = [add, sub, mul];
console.log(ops[2](2, 5));        // 10,从数组里取函数调用
console.log(typeof add);          // 'function'
console.log(add.name);            // 'add',函数自带名字属性
```

⚠ 常见误区 4.1:函数声明会被提升,函数表达式不会

与 C 的"函数必须先声明后使用"不同,JS 的函数声明会在整个作用域顶部生效,先调用后声明也合法;但 `const` 声明的函数表达式严格遵守 TDZ,定义前调用抛 `TypeError`。生产规范:一律用 `const` + 函数表达式或箭头函数,行为可预测,也避免命名冲突。

§4.2 参数传递:值传递 + 对象共享

💡 直觉先行:传参永远"传副本",但副本可能是引用

C 里传结构体是整块拷贝,传指针是拷贝指针本身;JS 里只有一种传参方式:传值的副本。区别在于:原始类型 (`number/string/bool`)的副本是值本身,函数内修改不影响外部;对象(含数组、函数)的副本是"引用",引用指向同一个对象,函数内修改属性会反映到外部,但给参数重新赋值不会影响外部变量。

场景	C	JS
改外部 int	传 <code>int*</code> 再 <code>*p = x</code>	传对象包一层,或函数返回新值
改结构体成员	传 <code>struct*</code>	传对象,直接改属性即可
整个数组传参	退化为指针,配合长度参数	传引用,自带 <code>length</code>
形参重新赋值	不影响实参	不影响实参(引用副本)

```

// 原始类型:改不动外部
function tryChange(n) {
  n = 999;
}
let x = 1;
tryChange(x);
console.log(x);           // 1 ← 不受影响

// 对象:改属性影响外部
function addScore(student, score) {
  student.scores.push(score); // 通过引用改到了外部对象
}
const s = { name: '李雷', scores: [80] };
addScore(s, 95);
console.log(s.scores);    // [ 80, 95 ] ← 外部被改了

// 但重新赋值参数不影响外部
function rebind(obj) {
  obj = { name: '全新对象' }; // 只是让形参指向新对象
}
const o = { name: '原对象' };
rebind(o);
console.log(o.name);      // 原对象 ← 外部不受影响

// C 的"输出参数"写法在 JS 里改用什么?→ 返回新值
function pushClone(arr, v) {
  return [...arr, v];      // 返回新数组(不可变风格)
}
const nums = [1, 2];
const nums2 = pushClone(nums, 3);
console.log(nums, nums2);  // [1,2] [1,2,3]

```

4.2.1 默认参数与 rest 参数

```
// 默认参数:C 没有(需手动判断);JS 在形参表直接给
function greet(name = '匿名', level = '同学') {
  return `你好,${level} ${name}`;
}
console.log(greet());           // 你好,同学 匿名
console.log(greet('李雷'));     // 你好,同学 李雷
console.log(greet('韩梅梅', '老师')); // 你好,老师 韩梅梅
// 注意:传 undefined 触发默认值,传 null 不触发
console.log(greet(undefined, null)); // 你好,null 匿名

// rest 参数:收集多余参数为数组(类似 C 的变参,但类型安全)
function sum(...nums) {
  let total = 0;
  for (const n of nums) total += n;
  return total;
}
console.log(sum(1, 2, 3, 4));    // 10
console.log(sum());              // 0

// arguments:老式变参对象(箭头函数没有!)
function oldSum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
console.log(oldSum(1, 2, 3));     // 6,生产推荐用 ...rest 替代 arguments
```

工程建议 4.1:参数超过 3 个就用"配置对象"

C 里参数一多就用结构体;JS 对应物是配置对象 + 解构默认值,调用方可读性极高:

```
function createServer({ host = '0.0.0.0', port = 3000, timeout = 5000, logger = console } = {}) {
  return { host, port, timeout, logger };
}
const srv = createServer({ port: 8080 });
console.log(srv.port);           // 8080,其余取默认
```

§4.3 返回值:return 与解构接收

```
// C 一次只能返回一个值(结构体除外);JS 用对象/数组解构"模拟多返回值"
function analyze(nums) {
  const sum = nums.reduce((a, b) => a + b, 0);
  const max = Math.max(...nums);
  return { sum, max, count: nums.length }; // 返回对象
}
const { sum, max, count } = analyze([3, 9, 5]);
console.log(sum, max, count); // 17 9 3

// 没有 return 的函数返回 undefined
function noop() {}
console.log(noop()); // undefined
```

§4.4 this 绑定四规则:本章核心

💡 直觉先行:this 是"调用时"决定的隐式参数

C 的结构体函数写法 `obj.method(&obj)` 把"自己"作为第一个参数显式传入;JS 把这个参数藏了起来,叫 `this`,由调用方式 决定它指向谁。同一函数,四种调用方式,this 四个值:

```

function who() {
  return this === undefined ? 'undefined(严格模式)' : String(this);
}

// 规则一:默认绑定—独立调用,this 为 undefined(严格模式)或全局对象(非严格)
'use strict';
function show() { console.log(this); }
show(); // undefined ← 默认绑定,严格模式下

// 规则二:隐式绑定—作为对象方法调用,this 指向该对象
const user = {
  name: '李雷',
  hello() { return `我是 ${this.name}`; },
};
console.log(user.hello()); // 我是 李雷 ← this 是 user

// 隐式绑定陷阱:方法被"拿出来"单独调用,this 丢失!
const fn = user.hello;
console.log(fn()); // TypeError: Cannot read properties of undefined
// 原因:fn() 是独立调用,this 变 undefined,this.name 崩溃

// 规则三:显式绑定—call/apply/bind 指定 this
const other = { name: '韩梅梅' };
console.log(user.hello.call(other)); // 我是 韩梅梅
console.log(user.hello.apply(other)); // 我是 韩梅梅
const bound = user.hello.bind(other);
console.log(bound()); // 我是 韩梅梅 ← bind 返回新函数

// 规则四:new 绑定—作为构造函数调用,this 指向新对象
function Person(name) {
  this.name = name; // this 是 new 创建的新对象
}
const p = new Person('王芳');
console.log(p.name); // 王芳

```

★ 必须背下的四规则优先级

new 绑定 > 显式绑定(call/apply/bind) > 隐式绑定(对象方法) > 默认绑定(独立调用)。判断 this 时先看调用形式:前面有没有 new?有没有 call/apply/bind?有没有"."(对象方法)?都没有就是默认绑定。箭头函数不参与以上任何规则,见 §4.6。

§4.5 call / apply / bind:显式绑定的三兄弟

```
function introduce(city, year) {
  return `${this.name},${city},${year}`;
}
const person = { name: '李雷' };

// call:逐个传参
console.log(introduce.call(person, '北京', 2026));
// apply:参数用数组传(与 call 等价,生产已少用)
console.log(introduce.apply(person, ['北京', 2026]));
// bind:绑定 this 与部分参数,返回新函数
const intro = introduce.bind(person, '上海');
console.log(intro(2026));           // 李雷,上海,2026 ← 参数被"预填"

// 经典用途:借用方法(数组方法借给类数组对象)
const args = { 0: 'a', 1: 'b', length: 2 };
console.log(Array.prototype.slice.call(args)); // [ 'a', 'b' ]
// 现代替代:Array.from(args)

// bind 的经典用途:回调里固定 this
const counter = {
  count: 0,
  click() {
    this.count++;
    console.log('点击次数:', this.count);
  },
};
const btn = { onClick: counter.click.bind(counter) };
btn.onClick();           // 点击次数:1 ← bind 保住了 this
```

⚠ 常见误区 4.2:回调里 this 神秘丢失

setTimeout(user.hello, 100) 里 this 不是 user——回调是独立调用,this 走默认绑定。修复三选一:① user.hello.bind(user);② 箭头函数 setTimeout(() => user.hello(), 100);③ 在对象方法内部先把 this 存成变量(老代码 const self = this)。生产代码里箭头函数是最常见解法。

§4.6 箭头函数:没有自己的 this 与 arguments

箭头函数是语法糖式的新写法,但有三个与普通函数的 **本质区别**:① 没有自己的 this(取词法外层作用域的 this);② 没有 arguments;③ 不能当构造函数(没有 prototype 可挂?有 prototype 属性,但不能 new)。生产铁律:回调一律箭头函数,方法定义看情况。

```

const team = {
  members: ['李雷', '韩梅梅'],
  name: '先锋队',
  // ✅ 箭头函数回调:this 是词法外层 = team
  greet() {
    return this.members.map((m) => `${this.name}:${m}`);
  },
  // ❌ 普通函数回调:this 会丢(在回调内 this 是 undefined 或全局)
  badGreet() {
    return this.members.map(function (m) {
      return `${this.name}:${m}`; // this 不是 team!
    });
  },
};

console.log(team.greet()); // [ '先锋队:李雷', '先锋队:韩梅梅' ]
// console.log(team.badGreet()); // 抛错:this.name 读 undefined

// 箭头函数没有 arguments
function normal() { return arguments.length; }
console.log(normal(1, 2, 3)); // 3
const arrow = () => arguments; // 引用外层作用域的 arguments(模块层无则报错)
// 生产:箭头函数取参一律用 rest:const arrow = (...args) => args.length

// 箭头函数不能 new
// new (() => {})(1); // ❌ TypeError: not a constructor

// 箭头函数只有"表达体"与"块体"两种形态
const square = (n) => n * n; // 表达体:直接返回
const sq2 = (n) => ({ value: n * n }); // 返回对象要加括号
const cube = (n) => { // 块体:必须写 return
  const c = n * n * n;
  return c;
};
console.log(square(4), sq2(4).value, cube(3)); // 16 16 27

```

★ 重点结论:箭头函数 this = 定义处的 this(词法绑定)

箭头函数 没有自己的 this,调用时它向上层作用域找 this。所以"在对象方法内用箭头回调"安全,"在箭头函数里再用箭头"始终是模块层的 this。判断箭头函数 this 的唯一方法:看它 定义位置 在哪个函数/对象方法里。

§4.7 闭包:函数记住定义时的环境

C 的函数指针只携带"代码地址",不携带环境;JS 的闭包同时携带 定义时可见的变量环境。函数定义后,即使外层函数已经返回,它仍能读写外层变量。生产用途:私有状态、工厂函数、计数器、防抖节流、柯里化。

```

// 经典闭包:计数器(类比 C 的 static 局部变量,但每个实例独立)
function makeCounter() {
  let count = 0;           // 被内部函数捕获
  return {
    inc: () => ++count,
    get: () => count,
    reset: () => { count = 0; },
  };
}
const c1 = makeCounter();
const c2 = makeCounter();
c1.inc(); c1.inc();
console.log(c1.get());      // 2
console.log(c2.get());      // 0 ← 两个实例互不干扰

// 私有状态:闭包模拟"私有成员"(第6章还有 # 私有字段)
function createAccount(initial) {
  let balance = initial;    // 外部无法直接访问
  return {
    deposit: (n) => { balance += n; },
    withdraw: (n) => {
      if (n > balance) return false;
      balance -= n;
      return true;
    },
    balanceOf: () => balance,
  };
}
const acc = createAccount(100);
acc.deposit(50);
console.log(acc.balanceOf()); // 150
console.log(acc.balance);     // undefined ← 私有,拿不到

// 工厂函数:按参数生成专用函数
function multiplier(factor) {
  return (n) => n * factor;   // factor 被捕获
}
const double = multiplier(2);
const triple = multiplier(3);
console.log(double(10), triple(10)); // 20 30

// 闭包陷阱:每次调用都新建环境
function loopCapture() {
  const fns = [];
  for (let i = 0; i < 3; i++) {
    fns.push(() => i);        // let:每轮新绑定
  }
  return fns;
}
console.log(loopCapture().map((f) => f())); // [ 0, 1, 2 ]

```

⚠ 常见误区 4.3:闭包里的循环变量读到最终值

这是第3章 var bug 的延续:若用 var 声明,所有闭包共享同一个绑定,读到最终值;用 let 则每轮独立。生产里在循环内创建回调(事件监听、定时器、Promise)时,一律确认循环变量是 let,或显式传参给函数。

§4.8 高阶函数:函数处理函数

把函数当参数或返回值的函数叫高阶函数。C 的 qsort 比较器是最早的高阶函数;JS 把这一思想铺满标准库。数组三件套 map/filter/reduce 必须形成肌肉记忆:

```
const prices = [199, 299, 89, 459];

// map:一一映射(数量不变,值变换)
const taxed = prices.map((p) => Math.round(p * 1.1));
console.log(taxed);           // [ 219, 329, 98, 505 ]

// filter:筛选(数量可能变)
const cheap = prices.filter((p) => p < 300);
console.log(cheap);           // [ 199, 299, 89 ] ← 299 也小于 300,保留

// reduce:聚合(变一个值)
const total = prices.reduce((acc, p) => acc + p, 0);
console.log(total);           // 1046

// 链式调用:读法从前往后
const expensiveTotal = prices
  .filter((p) => p > 200)
  .map((p) => p * 0.9)         // 打九折
  .reduce((acc, p) => acc + p, 0);
console.log(Math.round(expensiveTotal)); // 682

// 回调地狱的现代解法(第9章):回调 → Promise
function fetchUser(id, cb) {
  setTimeout(() => cb({ id, name: `用户${id}` }), 10);
}
fetchUser(7, (u) => console.log(u.name)); // 用户7

// 函数返回函数:currying(柯里化)
const addCurried = (a) => (b) => a + b;
console.log(addCurried(3)(4)); // 7
```

🚀 工程建议 4.2:用纯函数,少用副作用

纯函数(同样的输入必有同样的输出、不修改外部)是 JS 生产代码的第一原则:可测试、可缓存、可并行。map/filter/reduce 天然纯;for 循环改外部变量是副作用。团队代码评审时,优先把"循环 + push"重写为 filter/map 组合,可读性立竿见影。

§4.9 递归、入口与程序组织

```
// 递归:与 C 相同,注意 V8 对深递归会栈溢出
function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1);
}
console.log(fact(10));           // 3628800

// 深递归防护:1e5 层会 RangeError,生产大递归改迭代或拆任务
function sumTo(n) {
  let s = 0;
  for (let i = 1; i <= n; i++) s += i;
  return s;
}

// 入口:没有 main。Node 里常用模块级 main() 组织(第13章讲模块)
function main() {
  const argv = process.argv.slice(2);  // C 的 argc/argv 对应物
  console.log('参数:', argv);
}
main();
```

★ 重点结论:JS 的"参数"入口对照

C 的 `int main(int argc, char* argv[])` → Node 的 `process.argv` (`argv[0]` 是 node 路径,`argv[1]` 是脚本路径,`argc` 即 `argv.length`);浏览器程序没有命令行参数,入口是"事件"(页面加载、点击)。

本章速查表

主题	速查要点
三种定义	function 声明(提升)、const 表达式、箭头函数
传参语义	原始类型传值改不动外部;对象传引用副本,改属性影响外部
默认参数	形参表直接给;传 undefined 触发,null 不触发
变参	...rest 收集数组;arguments 是老写法,箭头函数没有
多返回值	返回对象/数组 + 解构接收
this 规则一	独立调用:undefined(严格模式)/全局对象
this 规则二	obj.method():this 指向 obj(方法拿出来调用会丢)
this 规则三	call/apply(立即调用)/bind(返回新函数)
this 规则四	new Foo():this 指向新对象;优先级 new > 显式 > 隐式 > 默认
箭头函数	无自己的 this/arguments,不能 new;this 取定义处
闭包	函数 + 定义时环境;计数器/私有状态/工厂
高阶函数	map 变换、filter 筛选、reduce 聚合、链式调用
柯里化	(a) => (b) => 结果,逐层传参
递归	与 C 相同;超深递归会 RangeError,改迭代
入口	Node:process.argv;浏览器:事件驱动,无 main

最小必会清单

- 能写出函数声明/表达式/箭头函数三种形态,并说出提升差异
- 能解释"原始类型传值、对象传引用副本"并给出改外部对象的代码
- 能写出默认参数与 rest 参数的函数,并说出 undefined 与 null 触发差异
- 能背诵 this 四规则与优先级,并各写一个 5 行示例
- 能说出 call/apply/bind 的区别,并用 bind 修复回调 this 丢失
- 能说出箭头函数与普通函数在 this/arguments/new 三个维度的差异
- 能写出闭包计数器与私有状态(账户余额)两个例子
- 能用 map/filter/reduce 链式完成"筛选+变换+聚合"
- 能解释"方法拿出来单独调用 this 丢失"并给出两种修复
- 能说出 Node 的 process.argv 与 C 的 argv 对应关系

自测题

Q 1. 写出输出:const obj = { v: 1, get() { return this.v; } }; const f = obj.get; console.log(f())

✓ 参考答案

抛 `TypeError`(严格模式下 `this` 为 `undefined`,读 `this.v` 崩溃);非严格模式返回 `undefined`。方法被拿出来独立调用,隐式绑定丢失,`this` 走默认绑定。

Q 2. 如何让 `f()` 在上题中输出 1?给出三种写法。

✓ 参考答案

① `const f = obj.get.bind(obj)`;② `const f = () => obj.get()`;③ 调用时显式绑定 `obj.get.call(obj)`。

Q 3. 写出输出:`function f(a = 1) { return a; } console.log(f(undefined), f(null), f(0))`

✓ 参考答案

1、`null`、`0`。默认参数只在参数为 `undefined` 时触发;`null` 和 `0` 都原样传入。

Q 4. 简述闭包是什么,并写出一个每次调用加 1 的计数器工厂函数。

✓ 参考答案

闭包 = 函数 + 其定义时的词法环境,内部函数可访问外层局部变量。工厂: `function makeCounter() { let n = 0; return () => ++n; }`。

Q 5. 用 `map/filter/reduce` 一行链式求出 `[1..10]` 中偶数的平方和。

✓ 参考答案

`[1,2,3,4,5,6,7,8,9,10].filter(n => n % 2 === 0).map(n => n * n).reduce((a, b) => a + b, 0)` 得 220。

Q 6. 下列代码中 `this` 分别是谁?① `p.hi()`; ② `hi.call(q)`; ③ `new Foo()`; ④ `foo()`(独立调用)

✓ 参考答案

① `p`(隐式绑定);② `q`(显式绑定);③ 新对象(`new` 绑定);④ `undefined`(默认绑定,严格模式)。

章末实验题:可配置的统计处理管线

实验 4:可配置的统计处理管线

实验目标:实现一个函数式风格的数据统计管线:数据经"过滤 → 变换 → 聚合"三步流水线处理,全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:函数定义三形态、默认参数、rest):管线工厂 `pipe(...fns)` 用 `rest` 收集处理函数,默认参数提供空管线
- 任务 2(覆盖:高阶函数、回调、纯函数):实现 `clamp`、`normalize`、`filterOutlier` 等纯函数处理器,均以"数组进、数组出"
- 任务 3(覆盖:闭包、工厂函数):实现 `makeThreshold(limit)` 工厂,返回"过滤大于 `limit` 的数据"的处理器(捕获 `limit`)
- 任务 4(覆盖:`this` 绑定、`call`/`apply`/`bind`、箭头函数):管线内用箭头函数回调;演示 `bind` 给处理器绑定配置对象
- 任务 5(覆盖:`map`/`filter`/`reduce`、多返回值解构):聚合阶段用 `reduce` 返回 `{sum, avg, max, min}` 并解构打印
- 任务 6(覆盖:参数传递语义):演示对象作为参数被处理器原地修改的"共享陷阱",并用不可变风格返回新数组规避
- 任务 7(覆盖:柯里化、递归):附加实现 `addCurried` 柯里化函数与递归 `gcd`

实验环境:Node.js 20+,运行 `node pipeline.js`。

第一步 写纯处理器:每个处理器是"数组 → 数组"的纯函数(不改入参)

第二步 写 pipe:`reduce` 依次应用,最后一个处理器可返回任意类型

第三步 后写演示:构造 20 个含噪声的分数,经"过滤异常 → 阈值截断 → 聚合"输出报告

✅ 参考答案要点

核心:pipe 高阶组合器 + 纯函数处理器 + 闭包工厂 + reduce 聚合。约 100 行,覆盖全章知识点。

```
// pipeline.js — Node.js 20+ 运行:node pipeline.js
'use strict';

// 1. 管线组合器(覆盖:rest 参数、reduce、箭头函数、高阶函数)
function pipe(...stages) {
  // 数据进入后依次经过每个 stage
  return (data) => stages.reduce((acc, stage) => stage(acc), data);
}

// 2. 纯函数处理器(覆盖:纯函数、filter/map)
const dropOutlier = (arr) => {
  if (arr.length === 0) return arr;
  const avg = arr.reduce((a, b) => a + b, 0) / arr.length;
  return arr.filter((v) => Math.abs(v - avg) <= avg * 0.5); // 剔除偏离均值 50% 的
};

const clamp100 = (arr) => arr.map((v) => Math.min(100, Math.max(0, v)));
const round2 = (arr) => arr.map((v) => Math.round(v * 100) / 100);

// 3. 闭包工厂(覆盖:闭包、工厂函数、捕获变量)
function makeThreshold(limit) {
  return (arr) => arr.filter((v) => v <= limit); // limit 被闭包捕获
}
const under90 = makeThreshold(90);

// 4. 聚合器(覆盖:reduce、多返回值对象 + 解构)
function aggregate(arr) {
  if (arr.length === 0) return { sum: 0, avg: 0, max: 0, min: 0, count: 0 };
  const sum = arr.reduce((a, b) => a + b, 0);
  return {
    sum,
    avg: sum / arr.length,
    max: Math.max(...arr),
    min: Math.min(...arr),
    count: arr.length,
  };
}

// 5. 参数传递演示(覆盖:对象共享陷阱 vs 不可变风格)
function sharedBug(scores) {
  scores.sort((a, b) => a - b); // 原地排序,改了外部!
  return scores;
}
function safeSorted(scores) {
  return [...scores].sort((a, b) => a - b); // 复制后排序,外部不受影响
}

// 6. this 绑定演示(覆盖:bind、箭头函数)
const config = { prefix: '[分数]' };
function report(label, data) {
  return `${this.prefix}${label}:${data}`; // 依赖 this.prefix
}
const boundReport = report.bind(config);

// 7. 柯里化与递归(覆盖:柯里化、递归)
```

```

const addCurried = (a) => (b) => a + b;
function gcd(a, b) {
  return b === 0 ? Math.abs(a) : gcd(b, a % b);
}

// 8. 主流程(覆盖:函数表达式、main 组织、模板字符串)
function main() {
  const raw = [];
  for (let i = 0; i < 20; i++) {
    raw.push(Math.round(40 + Math.random() * 80)); // 40~120 含噪声
  }

  // 组合管线:剔除异常 → 截断 100 → 舍入 → 过滤 > 90 → 聚合
  const pipeline = pipe(dropOutlier, clamp100, round2, under90, aggregate);
  const stats = pipeline(raw);

  console.log('原始数据:', raw.join(' '));
  console.log(boundReport('管线结果', JSON.stringify(stats)));

  // 共享陷阱演示
  const original = [3, 1, 2];
  sharedBug(original);
  console.log('sharedBug 后外部也被排序:', original); // [ 1, 2, 3 ]

  const fresh = [3, 1, 2];
  const sorted = safeSorted(fresh);
  console.log('safeSorted 后外部不变:', fresh, '副本:', sorted);

  // 柯里化与递归演示
  console.log('addCurried(3)(4) =', addCurried(3)(4));
  console.log('gcd(48, 36) =', gcd(48, 36));
}

main();

```

设计说明:pipe 用 reduce 实现"数据穿过每个 stage",最后一个 stage 输出任意类型(聚合器返回对象);所有处理器保持纯函数,唯一会改外部的 sharedBug 被单独隔离演示,强调"复制后修改"的不可变风格;boundReport 用 bind 固定 this,呼应 §4.5;makeThreshold 演示闭包捕获工厂参数。运行输出包含管线统计、绑定报告、共享陷阱对比、柯里化与 gcd 五部分。

下一章预告:第5章 内存管理 —— V8 的标记-清除、内存泄漏四大场景、WeakMap 与 Buffer,GC 语言里"内存问题"长什么样。

第5章 内存管理

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 GC 语言的内存模型:JS 里"变量在栈、对象在堆",但程序员看不到栈上指针
- 掌握 V8 的垃圾回收机制:分代、标记-清除、增量标记,以及"GC 不可控"的含义
- 掌握内存泄漏四大场景:全局变量、闭包、定时器、事件监听,以及排查工具
- 理解 WeakMap/WeakRef 的弱引用语义与生产用途
- 理解 Buffer 与 ArrayBuffer:二进制数据的两种容器与 C 的字节数组对照
- 掌握资源释放惯用法:try/finally 与连接/句柄的关闭
- 理解"C 中怎么做→JS 中怎么做":从手动 malloc/free 到自动回收

💡 从 C 到 JavaScript:本章主题如何迁移

C 程序员与内存的关系是"亲力亲为":malloc 申请、free 释放、野指针、双重释放、内存泄漏、悬垂指针,每一个都是深夜事故的常客。JS 把这个负担整体收走:垃圾回收器(GC)自动管理对象生命周期,没有 malloc/free,没有野指针,数组越界也只是 undefined 而不是写坏内存。

但"自动"不等于"不会泄漏"。GC 只能回收"不再被引用的对象";只要还有一条引用链活着,对象就永远留在堆里。C 的泄漏是"忘了 free",JS 的泄漏是"忘了断引用"。所以本章的核心不是"怎么释放",而是理解引用图:谁还握着这个对象。

V8(Node 与 Chrome 的引擎)把堆分成新生代与老生代,新生代用复制式 Scavenge 快速回收短命对象,老生代用标记-清除 + 增量标记清理长寿对象。GC 是自动的、不确定的:你不能手动触发"完整回收"(老代码的 `global.gc()` 只在 `--expose-gc` 下可用且仅用于调试),只能观察与调优。

另一个重要的迁移:JS 没有"栈上结构体"的概念,一切对象都在堆上;变量只是引用。这解释了为什么"把一个对象传给函数"永远是共享的(第4章),也解释了为什么深拷贝需要专门手段。二进制数据则是本章的新知识点:Buffer(Node)与 ArrayBuffer(标准),它们与 C 的 `unsigned char buf[N]` 直接对应。

📦 前置知识自查

- C 的栈/堆/静态区内存布局,malloc/free/calloc/realloc 的生命周期管理
- C 的指针、悬垂指针与内存泄漏概念
- C 的局部变量生命周期(函数返回即销毁)
- 静态变量 static 的持久性
- C 的结构体拷贝与字节数组 `unsigned char[]`

🚀 本章学习路线

1. **第一阶段(模型)**:读 §5.1–§5.2,建立"引用图 + 可达性"心智模型
2. **第二阶段(泄漏)**:读 §5.3–§5.4,亲手复现四大泄漏场景,学会用 `--heap-prof` 观察
3. **第三阶段(二进制)**:读 §5.5–§5.6,练习 Buffer 与 ArrayBuffer 的基本操作
4. **第四阶段(实验)**:完成章末"泄漏复现与修复 + 内存观察"
5. **验收标准**:能独立说出四种泄漏场景与各自修复,能画出任意两对象的引用关系

本章知识导图

- 第5章 内存管理
 - §5.1 内存模型:栈/堆、引用与可达性
 - §5.2 V8 回收机制:分代、标记-清除、不可控性
 - §5.3 内存泄漏四大场景与排查工具
 - §5.4 WeakMap / WeakRef 弱引用
 - §5.5 Buffer 与 ArrayBuffer 二进制数据
 - §5.6 资源释放惯用法:finally 与句柄关闭
 - §5.7 生产实践:泄漏检查清单、性能建议

§5.1 内存模型:从 malloc/free 到引用图

对比项	C	JS
对象位置	栈(局部)或堆(malloc)	对象都在堆;变量只是堆上的引用
释放	free() 手动释放	GC 自动,不可手动
悬垂指针	存在(use-after-free)	不存在(GC 保证对象存活到无人引用)
泄漏	忘了 free	忘了断引用
双重释放	UB,崩溃	不可能
数组越界	写坏相邻内存(UB)	返回 undefined,不损坏内存
查看占用	sizeof / valgrind	process.memoryUsage() / heap snapshot

```
// 引用图的直觉:变量指向对象
let a = { data: '大对象' }; // 对象在堆上,a 是引用
let b = a; // b 也指向同一个对象
a = null; // 断掉一条引用
// 对象仍被 b 引用,不会回收
console.log(b.data); // 大对象
b = null; // 现在没有任何引用,GC 随时回收

// 数组/对象嵌套:整棵引用树都活着
const root = { list: [{ id: 1 }, { id: 2 }] };
root.list = null; // 断掉子树根,两个对象变成垃圾
// 但 root 本身还活着

// 查看进程内存(Node)
console.log(process.memoryUsage());
// { rss: ..., heapTotal: ..., heapUsed: ..., external: ..., arrayBuffers: ... }
```

★ 重点结论:GC 回收的条件 = 不可达

一个对象能被回收,当且仅当 从根(全局对象、当前执行栈)出发无法到达它。环状引用(对象互相指)在 JS 里 不是 泄漏——C++ 的 `shared_ptr` 循环引用要手动破除,JS 的 GC 从根出发标记,互相指但没有根的环一样会被回收。这消除了 C++ 里最头疼的一类问题。

§5.2 V8 的垃圾回收:分代与标记-清除

V8 将堆分成两个代:

- **新生代(young generation)**:放刚创建的对象,容量小,用 **Scavenge(复制算法)** 回收——把存活对象复制到另一半空间,其余整体清空。大多数对象活不过几次回收。
- **老生代(old generation)**:存活多次回收的对象晋升到这里,用**标记-清除(Mark-Sweep)**与**增量标记**:先标记所有可达对象,再清除不可达对象;增量标记把大停顿拆成小片,避免卡顿。

```
// GC 不可控性的演示:创建大量临时对象后,内存并不立即回落
function createGarbage() {
  const tmp = [];
  for (let i = 0; i < 1e5; i++) {
    tmp.push({ i, s: '临时数据'.repeat(10) }); // 1e5 个对象
  }
  return tmp.length; // 返回后 tmp 变成垃圾
}

const before = process.memoryUsage().heapUsed;
console.log('创建垃圾:', createGarbage());
global.gc?.(); // Node 加 --expose-gc 时可用,否则 undefined,不报错
const after = process.memoryUsage().heapUsed;
console.log('堆增量约:', Math.round((after - before) / 1024), 'KB');

// 生产调优:启动参数控制堆上限
// node --max-old-space-size=4096 app.js # 老生代上限 4GB
// node --expose-gc app.js # 暴露 global.gc()(仅调试)
```

⚠ 常见误区 5.1:以为"GC 会立刻回收"

GC 的时机由引擎决定:内存压力大、代际晋升、空闲时都可能触发。你无法保证"置空引用后内存马上回落"。性能测试里"创建 → 置空 → 立刻看内存"得到的数据不可靠;要看趋势,用 `--heap-prof` 或 Chrome DevTools 的 Memory 面板拍堆快照,对比前后引用图。

§5.3 内存泄漏四大场景

泄漏 = 对象长期被引用且不再需要。四大高频场景:

5.3.1 场景一:全局变量/全局容器无限增长

```
// ❌ 泄漏:全局数组无限追加
const cache = []; // 模块级 = 全局生命周期
function logEvent(e) {
  cache.push({ at: Date.now(), e }); // 永远有人引用,永不回收
}
for (let i = 0; i < 100; i++) logEvent('点击');

// ✅ 修复:限制大小(固定上限的环形缓冲)
const bounded = [];
function logEventSafe(e) {
  bounded.push({ at: Date.now(), e });
  if (bounded.length > 100) bounded.shift(); // 超限丢弃最旧
}
console.log(bounded.length); // 100,封顶

// 全局变量另一个形态:意外挂到全局
// ❌ 严格模式下 this 是 undefined,不会误挂全局;但忘写声明的变量在非严格模式会变全局
'use strict';
function bad() {
  // leak = 1; // 严格模式下 ReferenceError;非严格模式会创建全局变量!
}
```

5.3.2 场景二:闭包意外持有大对象

```
// ❌ 泄漏:闭包捕获了大对象,即使只需要小字段
function heavy() {
  const bigData = new Array(1e6).fill('x'); // 800KB 级别
  const id = 'user-1';
  return () => id; // 只需要 id,但闭包持有整个环境
}

const fn = heavy(); // bigData 一直被 fn 闭包引用

// ✅ 修复:让闭包只捕获需要的东西(把大对象放在另一作用域)
function light() {
  const id = 'user-1';
  {
    const bigData = new Array(1e6).fill('x'); // 块级作用域,函数返回后不可达
    console.log(bigData.length); // 只在块内使用
  }
  return () => id; // 闭包只捕获 id
}

const fn2 = light();
console.log(fn2());
```

5.3.3 场景三:定时器/轮询未清理

```
// ❌ 泄漏:setInterval 永不清理,回调及其闭包永不释放
const data = [];
const timer = setInterval(() => {
  data.push(Date.now()); // data 无限增长
}, 100);

// ✅ 修复:明确停止条件与清理
const timer2 = setInterval(() => {
  if (data.length > 10) {
    clearInterval(timer2); // 条件满足就停
    console.log('定时器已清理,元素数:', data.length);
  } else {
    data.push(Date.now());
  }
}, 100);

// Node 进程不退出检查:进程是否还挂着 handle
setTimeout(() => {
  console.log('Node 主动退出(即使有未清定时器,但生产要清干净)');
  clearInterval(timer);
  process.exit(0);
}, 500);
```

5.3.4 场景四:事件监听器重复注册

```
const { EventEmitter } = require('node:events');

// ❌ 泄漏:每调用一次就注册一个新监听,旧的永远在
function subscribeOnce(emitter) {
  emitter.on('data', (chunk) => {
    console.log('收到:', chunk.length);
  });
}

const em = new EventEmitter();
for (let i = 0; i < 1000; i++) subscribeOnce(em); // 1000 个监听!
console.log('监听器数量:', em.listenerCount('data')); // 1000

// ✅ 修复:命名函数 + off 清理,或 once
function handler(chunk) { console.log('收到:', chunk.length); }
em.on('data', handler);
em.off('data', handler); // 用完即移除
em.once('data', (c) => console.log('一次性:', c.length)); // 只触发一次自动移除
em.emit('data', Buffer.from('hello'));
console.log('剩余监听器:', em.listenerCount('data')); // 0
```

工程建议 5.1:泄漏排查三板斧

① `node --inspect app.js` + Chrome DevTools Memory 面板:录制堆快照,对比前后,按"Retained Size"排序找大对象;② 连续采样两次快照的 Delta 视图,看哪些构造函数实例数只增不减(典型的监听器/闭包泄漏);③ `process.memoryUsage()` 打点监控,配合 `--max-old-space-size` 让泄漏快速触发 OOM 报警而不是拖垮服务器。Node 20 还支持 `node --heap-prof` 直接产出 `.heapprofile` 文件离线分析。

§5.4 WeakMap 与 WeakRef:弱引用

WeakMap 的键不阻止垃圾回收:键对象变成垃圾时,键值对自动消失。典型用途:给对象附加元数据(缓存、私有数据),而不延长对象生命周期。WeakSet 同理。注意:WeakMap 的键必须是对象,不可遍历,没有 size。

```
// 缓存元数据:WeakMap 不会让键对象"永远活着"
const meta = new WeakMap();

function track(obj) {
  meta.set(obj, { visits: (meta.get(obj)?.visits ?? 0) + 1 });
}

let user = { name: '李雷' };
track(user);
track(user);
console.log(meta.get(user)); // { visits: 2 }
user = null; // 唯一引用断开
// meta 中的条目随之消失(GC 时),不会造成泄漏

// 对照:Map 会让键对象永远活着 ← 这就是"Map 版缓存"泄漏的根源
const strong = new Map();
strong.set(user, '某数据');

// 典型生产用途:缓存计算结果,键是"不可控的外部对象"
const cache = new WeakMap();
function compute(key) {
  if (!cache.has(key)) {
    cache.set(key, expensiveCompute(key));
  }
  return cache.get(key);
}
function expensiveCompute(k) { return `结果:${k}`; }
console.log(compute('a'), compute('a')); // 第二次直接命中缓存

// WeakRef(ES2021):更底层,持有"随时可能消失"的引用
// 用途:大缓存/对象池,允许 GC 在压力大时回收
let obj = { big: '数据' };
const ref = new WeakRef(obj);
console.log(ref.deref()?.big); // 数据,deref() 可能返回 undefined(已被回收)
obj = null;
```

⚠ 常见误区 5.2:用 Map 做对象缓存导致键永不释放

Map 对键是强引用: `cache.set(key, value)` 后,即使业务代码不再引用 key,Map 也拉着它。用 WeakMap 的代价是:不可遍历、不可知大小、键必须对象——所以"需要遍历的缓存"用 Map + 手动淘汰(定期删除),"只按对象查元数据"用 WeakMap。两者分工要清楚。

\$5.5 Buffer 与 ArrayBuffer:二进制数据

C 的 `unsigned char buf[1024]` 在 JS 中有两个对应物:**Buffer**(Node 专有,基于 Uint8Array,继承自标准 TypedArray,自带编码转换)与 **ArrayBuffer**(ECMAScript 标准,浏览器与 Node 通用,纯粹的字节容器)。生产规则:Node 环境优先 Buffer(API 齐全),跨端/Web 场景用 ArrayBuffer + DataView/TypedArray。

```
// Buffer:Node 二进制首选
const buf = Buffer.from('你好', 'utf8'); // 字符串 → 字节(UTF-8)
console.log(buf.length); // 6 ← 一个汉字 3 字节
console.log(buf); // <Buffer e4 bd a0 e5 a5 bd>
console.log(buf.toString('utf8')); // 你好,字节 → 字符串
console.log(buf.toString('hex')); // e4bda0e5a5bd,十六进制串

// 创建与读写
const b2 = Buffer.alloc(4); // 4 字节,清零(类似 calloc)
b2.writeUInt32BE(0x01020304, 0); // 大端写入
console.log(b2.toString('hex')); // 01020304
console.log(b2.readUInt16LE(2)); // 0x0304,小端读 16 位
b2[0] = 0xff; // 像 C 一样按下标改字节
console.log(b2[0]); // 255

// Buffer 与 C 字节数组对照:切片是视图不是拷贝
const big = Buffer.from([1, 2, 3, 4, 5, 6]);
const view = big.subarray(1, 4); // 视图,共享底层内存
view[0] = 99;
console.log(big); // <Buffer 01 63 03 04 05 06>

// ArrayBuffer:标准字节容器
const ab = new ArrayBuffer(8); // 8 字节
const u8 = new Uint8Array(ab); // 视图:8 个字节
u8[0] = 42;
console.log(u8[0]); // 42
const dv = new DataView(ab); // 任意类型读写
dv.setFloat64(0, 3.14);
console.log(dv.getFloat64(0)); // 3.14

// 对象 → JSON 字符串 → 字节(网络传输常用)
const payload = JSON.stringify({ ok: true, n: 7 });
const wire = Buffer.from(payload, 'utf8');
console.log(wire.toString('utf8')); // {"ok":true,"n":7}
```

⚠ 常见误区 5.3:Buffer.from(string) 的默认编码

默认是 utf8,但 Base64 场景常被搞错: Buffer.from(str, 'base64') 的 str 必须是合法 Base64;把普通字符串当 Base64 解码会得到乱码。另外 `buffer.length` 是字节数不是字符数,与字符串的 `length` 语义不同——文件大小、网络包大小都用字节数。

§5.6 资源释放惯用法:try/finally 与句柄关闭

C 用 `free/fclose` 释放;JS 的 GC 管内存,但 文件句柄、网络连接、定时器这些"外部资源"必须手动关闭。惯用法是 `try/finally`(第9章详解异常,这里先用结论):

```
const fs = require('node:fs');

// ❌ 危险:中途抛错就不关闭
function badWrite() {
  const f = fs.openSync('/tmp/out.txt', 'w');
  fs.writeSync(f, 'hello');
  // 若 writeSync 抛错,f 永远不关 → 句柄泄漏
  fs.closeSync(f);
}

// ✅ 正确:try/finally 保证释放
function goodWrite() {
  const f = fs.openSync('/tmp/out.txt', 'w');
  try {
    fs.writeSync(f, 'hello world');
  } finally {
    fs.closeSync(f);          // 无论是否抛错都执行
  }
}

goodWrite();
console.log('已安全写入并关闭');

// 文件句柄超限的表现:EMFILE: too many open files
// ulimit -n 调大 + 代码保证成对 open/close
```

§5.7 生产实践:泄漏检查清单与性能

🚀 生产泄漏检查清单(上线前过一遍)

- 全局容器(数组/Map)有没有无上限追加?加上限或定期清理
- 闭包有没有意外捕获大对象/大数组?把大对象放进块作用域
- setInterval/setTimeout 是否有清理路径?组件卸载、任务完成时 clear
- 事件监听是否在"不再需要"时 off 了?监听函数是否具名可移除?
- 缓存是 WeakMap 还是 Map?Map 是否带淘汰策略?
- 流(stream)有没有 drain/close 处理?HTTP 响应是否 end?
- 进程重启策略:崩溃自愈可以兜底"缓慢泄漏"(容器重启),但不能替代修复

```

// 性能注意:避免高频分配(与 C 的"频繁 malloc 慢"同理)
// ❌ 循环内创建字符串/数组
function buildBad(n) {
  let s = '';
  for (let i = 0; i < n; i++) {
    s += `item${i}`; // 每次 += 都创建新字符串
  }
  return s;
}

// ✅ 数组收集 + join(分配次数从 O(n) 降到 O(1) 次)
function buildGood(n) {
  const parts = new Array(n);
  for (let i = 0; i < n; i++) {
    parts[i] = `item${i}`;
  }
  return parts.join('');
}

console.log(buildGood(5)); // item0;item1;item2;item3;item4;

// 对象池:高频临时对象复用(游戏/高频消息场景)
class Pool {
  constructor(make) {
    this.make = make;
    this.free = [];
  }
  acquire() {
    return this.free.pop() ?? this.make();
  }
  release(obj) {
    this.free.push(obj);
  }
}

const pool = new Pool(() => ({ x: 0, y: 0 }));
const p = pool.acquire();
p.x = 1;
pool.release(p); // 归还复用,减少 GC 压力

```

工程建议 5.2:内存监控三件套

① 进程级: `process.memoryUsage()` 每 10 秒采样入库,画出 `heapUsed` 趋势线(持续上升 = 泄漏);② 引擎级:老生代增长率 `--max-old-space-size` 与 `--trace-gc` 观察 GC 频率,GC 越来越频繁且每次回收量趋近于零 = 泄漏;③ 应用级:压测工具(如 `autocannon`)在稳定 QPS 下跑 30 分钟,内存曲线应走平;持续爬升即泄漏,配合堆快照定位。

本章速查表

主题	速查要点
内存模型	对象都在堆;变量是引用;GC 回收不可达对象
回收算法	新生代 Scavenge(复制);老生代标记-清除 + 增量标记
GC 特性	自动、不可控、无 STW 长停顿(增量);--max-old-space-size 调上限
循环引用	不是泄漏!从根不可达即回收(与 C++ shared_ptr 不同)
泄漏一	全局容器无限增长 → 加上限/定期清理
泄漏二	闭包捕获大对象 → 只捕获所需变量
泄漏三	定时器未清理 → 明确停止条件 + clearInterval
泄漏四	监听器重复注册 → 具名函数 + off,或用 once
排查	--heap-prof / DevTools 堆快照 Delta / memoryUsage 趋势
WeakMap	键弱引用,不延长键生命周期;键必须对象,不可遍历
WeakRef	弱引用,deref() 可能 undefined;用于大缓存
Buffer	Node 二进制容器;length 是字节数;subarray 是视图
ArrayBuffer	标准字节容器 + TypedArray/DataView 视图
资源释放	句柄/连接/定时器用 try/finally 或清理函数成对释放
字符串拼接	循环内 += 慢;用数组 join 或模板字符串
对象池	高频临时对象复用,减少 GC 压力

最小必会清单

- 能解释"GC 回收不可达对象"并用两段代码演示引用断链
- 能说出新生代/老生代的回收算法名称与"增量标记"解决什么问题
- 能复现并修复四种泄漏场景中的至少三种
- 能说出 WeakMap 与 Map 的三个差异(键弱引用/键必须对象/不可遍历)
- 能说出 Buffer.length 与字符串 length 的差异,并完成 UTF-8 编解码
- 能写出 try/finally 保证文件句柄关闭的模板代码
- 能解释为什么 JS 没有悬垂指针与双重释放
- 能用 subarray 做出共享内存的切片视图
- 能说出循环拼接字符串的替代方案
- 能画出一个三层对象嵌套的引用图并指出断哪条边可回收子树

自测题

Q 1. 下面的代码会不会内存泄漏?为什么?`const a = {}; const b = {}; a.p = b; b.p = a; a = null; b = null;`

✅ 参考答案

不会。a 与 b 互相引用形成环,但环没有根可达,GC 的标记-清除从根出发,整个环不可达,会被回收。

Q 2. 说出四种内存泄漏场景,并各给出修复关键词。

✅ 参考答案

① 全局容器无限增长 → 上限/清理;② 闭包持有大对象 → 缩小捕获;③ 定时器未清理 → clearInterval;④ 监听器重复注册 → off/once。

Q 3. WeakMap 能遍历吗?它的键可以是字符串吗?用它做"用户会话缓存"合适吗?

✅ 参考答案

不能遍历,无 size;键必须是对象,字符串不行。用对象做键的会话元数据合适(键对象消失自动清理),需要遍历/统计的缓存用 Map。

Q 4. Buffer.from('你好','utf8').length 是多少?为什么?如何取回原字符串?

✅ 参考答案

6。'你好' 两个汉字,UTF-8 每字 3 字节,length 是字节数。toString('utf8') 还原。

Q 5. 简述 V8 为什么用分代回收,以及增量标记解决什么问题。

✅ 参考答案

大多数对象短命,分代让"小空间频繁复制"代替"全堆扫描",效率高。增量标记把单次大停顿拆成小片穿插执行,避免浏览器/服务卡顿(STW 时间缩短)。

Q 6. Node 里文件写入中途抛错会怎样?写出保证句柄关闭的模板。

✅ 参考答案

句柄泄漏,fd 不被释放,反复发生会 EMFILE。模板:openSync 后 try { 操作 } finally { closeSync }。

章末实验题:内存泄漏复现与修复实验室

实验 5:内存泄漏复现与修复实验室

实验目标:复现四种泄漏场景,观察堆增长,实现修复,并用 WeakMap/Buffer 完成收尾练习。

实验要求:

- 任务 1(覆盖:全局容器泄漏):模拟"全局日志数组无限增长",修复为定长环形缓冲
- 任务 2(覆盖:闭包泄漏):构造"闭包捕获大数组"案例,修复为只捕获所需字段
- 任务 3(覆盖:定时器泄漏):演示未清理的 setInterval 使进程不退出,修复为条件 clearInterval
- 任务 4(覆盖:事件监听泄漏):重复注册监听使 listenerCount 暴涨,修复为具名函数 + off/once
- 任务 5(覆盖:WeakMap/WeakRef):用 WeakMap 做对象元数据缓存,验证键对象置空后条目消失
- 任务 6(覆盖:Buffer/ArrayBuffer):把结构化数据编码为二进制并解码还原,演示字节长度与视图
- 任务 7(覆盖:try/finally 资源释放):实现文件写入 + 强制 finally 关闭,并打印 heapUsed 对比

实验环境:Node.js 20+,运行 `node leaklab.js`;建议同时运行 `node --expose-gc leaklab.js` 观察 GC 效果。

第一步 **写工具函数:**memNow() 返回 heapUsed;leakScenario() 循环创建对象并打点

第二步 **写四场景函数:**每个场景"先泄漏版(可选开关)→ 修复版",打点对比

第三步 **后写收尾:**WeakMap 生命周期演示 + Buffer 编解码 + finally 文件写入

✅ 参考答案要点

核心:每个场景用 "泄漏版/修复版" 双函数 + 内存打点输出对比,收尾覆盖 WeakMap/Buffer/finally。约 130 行。

```

// leaklab.js — 运行:node leaklab.js(建议加 --expose-gc 观察 GC)
'use strict';
const fs = require('node:fs');
const { EventEmitter } = require('node:events');

// 工具:内存快照(覆盖:process.memoryUsage、模板字符串)
function memNow() {
  return `${Math.round(process.memoryUsage().heapUsed / 1024)}KB`;
}

function gcHint() {
  if (typeof global.gc === 'function') {
    global.gc(); // --expose-gc 时强制回收(仅实验)
    return '(已手动 GC)';
  }
  return '(未启用 --expose-gc)';
}

// 场景一:全局容器泄漏 → 修复(覆盖:全局引用、数组上限)
const leakLog = [];
function leakLogEvent(e) { leakLog.push({ at: Date.now(), e }); }

const fixedLog = [];
function fixedLogEvent(e) {
  fixedLog.push({ at: Date.now(), e });
  if (fixedLog.length > 100) fixedLog.shift();
}

// 场景二:闭包泄漏 → 修复(覆盖:闭包捕获、块作用域)
function leakClosure() {
  const big = new Array(200000).fill('x');
  const id = 'u-1';
  return () => id;
}

function fixClosure() {
  const id = 'u-1';
  { const big = new Array(200000).fill('x'); console.log('big 仅块内可见:', big.length); }
  return () => id;
}

// 场景三:定时器泄漏 → 修复(覆盖:setInterval、clearInterval)
function leakTimer() {
  const data = [];
  const t = setInterval(() => data.push(Date.now()), 50);
  return () => { clearInterval(t); return data.length; };
}

function fixTimer() {
  const data = [];
  let t = setInterval(() => {
    if (data.length >= 5) {
      clearInterval(t); // 条件满足即停
    } else {
      data.push(Date.now());
    }
  }, 50);
  return () => data.length;
}

```

```

}

// 场景四:监听器泄漏 → 修复(覆盖:EventEmitter、on/off/once)
function runEmitterDemo() {
  const em = new EventEmitter();
  function handler(chunk) { if (chunk.length > 3) console.log('数据长度:', chunk.length); }
  for (let i = 0; i < 100; i++) em.on('data', handler); // 重复注册(演示)
  const before = em.listenerCount('data');
  em.off('data', handler); // 修复:全部移除
  em.once('data', (c) => console.log('once 消费:', c.length)); // 一次性
  em.emit('data', Buffer.from('abcd'));
  return { before, after: em.listenerCount('data') };
}

// 场景五:WeakMap 元数据(覆盖:WeakMap、set/get、置空)
function runWeakMapDemo() {
  const meta = new WeakMap();
  let user = { name: '李雷' };
  meta.set(user, { visits: 1 });
  user.visits = meta.get(user).visits;
  console.log('WeakMap 读取:', meta.get(user)?.visits);
  user = null;
  console.log('键置空后条目随 GC 消失(无法验证 size,WeakMap 不可遍历)');
}

// 场景六:Buffer 二进制(覆盖:Buffer、utf8/hex、subarray 视图)
function runBufferDemo() {
  const b = Buffer.from('你好JS', 'utf8');
  console.log('字节数:', b.length, '| hex:', b.toString('hex'));
  console.log('还原:', b.toString('utf8'));
  const big = Buffer.from([1, 2, 3, 4, 5]);
  const view = big.subarray(1, 3);
  view[0] = 99;
  console.log('视图改字节,底层同步变化:', big);
}

// 场景七:try/finally 文件句柄(覆盖:资源释放、finally)
function runFileDemo() {
  const fd = fs.openSync('/tmp/leaklab.txt', 'w');
  try {
    fs.writeSync(fd, 'hello leaklab\n');
    throw new Error('模拟中途出错');
  } catch (err) {
    console.log('捕获错误:', err.message);
  } finally {
    fs.closeSync(fd);
    console.log('句柄已在 finally 中关闭');
  }
}

// 主流程(覆盖:函数组织、对比输出)
function main() {
  console.log('== 场景一:全局日志 ==');
  for (let i = 0; i < 500; i++) { leakLogEvent(i); fixedLogEvent(i); }
  console.log('泄漏版长度:', leakLog.length, '| 修复版长度:', fixedLog.length);
}

```

```
console.log('== 场景二:闭包 ==');
const lc = leakClosure();
const fc = fixClosure();
console.log('泄漏版闭包返回值:', lc(), '| 修复版:', fc(), gcHint());

console.log('== 场景三:定时器 ==');
const leakTimerFn = leakTimer();
const fixTimerFn = fixTimer();
setTimeout(() => {
  console.log('泄漏版数据:', leakTimerFn(), '| 修复版数据:', fixTimerFn());
}, 300);

console.log('== 场景四:监听器 ==');
const { before, after } = runEmitterDemo();
console.log(`监听数 ${before} → ${after}`);

console.log('== 场景五:WeakMap ==');
runWeakMapDemo();

console.log('== 场景六:Buffer ==');
runBufferDemo();

console.log('== 场景七:资源释放 ==');
runFileDemo();

console.log('当前堆:', memNow());
}

main();
```

设计说明:四个泄漏场景各自"泄漏版/修复版"成对出现,输出对比数据让修复效果可见;场景三用 `setTimeout` 等待定时器跑完再统计,演示 `clearInterval` 的效果;场景四演示 `listenerCount` 从 100 归零;场景七用 `throw` 验证 `finally` 必然执行。建议先跑 `node leaklab.js`,再看 `node --expose-gc leaklab.js` 输出中的手动 GC 提示。

下一章预告:第6章 面向对象 —— 原型链、class 语法糖背后的 `prototype`、# 私有字段与 `getter/setter`,从 C 的 `struct`+函数指针进化到真正的面向对象。

第6章 面向对象

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

本章学习目标

- 理解从 C 的 struct + 函数指针到类的演进,以及"面向对象是组织代码的方式"
- 掌握原型链机制:prototype、__proto__、Object.create 三者关系
- 彻底理解 class 只是语法糖:背后仍是构造函数与原型链
- 掌握继承 extends 与 super,理解方法的查找链
- 掌握访问控制:JS 的 # 私有字段与私有方法
- 掌握 getter/setter、静态成员与 instanceof
- 理解多态与鸭子类型,以及组合优于继承的设计原则

从 C 到 JavaScript:本章主题如何迁移

C 语言里,数据与行为是分开的:struct 只存数据,函数操作结构体靠"传指针"。想模拟多态,得手动维护函数指针表: `struct Shape { void (*area)(const void*); }` ——把"类型行为"硬编码进数据里,还要自己处理回调里的强制转换,笨拙且危险。

JavaScript 提供了两条路:传统原型链(ES5 时代)与现代 class 语法(ES2015+)。class 只是语法糖,底层依然是构造函数 + 原型链。理解原型链,你才能解释: `arr.push` 为什么存在、`instanceof` 怎么工作、猴子补丁为什么危险、框架为什么能改写原型方法。

与 C++/Java 不同,JS 的继承是 **基于原型的单链继承**:每个对象有一个原型(proto),属性查找沿着链向上走。没有多继承、没有虚函数表、没有访问修饰符的 `public/protected`——私有用 `#` 前缀实现(ES2022),访问控制在语言层面只有"公开/私有"两档。

C 的程序员最容易犯的错是把 JS 当 C++ 用:建一堆层级很深的继承树。JS 社区的主流实践是 **组合优先**:小对象组合成大对象、mixin 混入、鸭子类型。这不是规范强制的,而是生产经验——原型继承浅而组合宽,可维护性高得多。

前置知识自查

- C 的 struct、指针、结构体数组
- C 的函数指针成员(struct + 函数指针模拟"方法")
- C 的静态库组织思想(头文件声明、实现分离)
- 第4章 this 四规则(本章的方法调用依赖隐式绑定)
- 第1章 typeof/对象基础

本章学习路线

1. **第一阶段(对象与原型)**:读 §6.1–§6.2,在 REPL 里检查原型链
2. **第二阶段(class 与继承)**:读 §6.3–§6.5,亲手写出"类→原型"的等价展开
3. **第三阶段(成员与设计)**:读 §6.6–§6.9,重点理解 # 私有与组合优于继承
4. **第四阶段(实验)**:完成章末"几何图形系统"
5. **验收标准**:不看资料,能用纯函数 + Object.create 手工实现一个"类"并讲清查找链

本章知识导图

- 第6章 面向对象
 - §6.1 从 C 到面向对象:struct+函数指针的演进
 - §6.2 对象与原型链:prototype / __proto__ / Object.create
 - §6.3 构造函数与 new 的过程
 - §6.4 class 语法糖:本质是原型,extends/super
 - §6.5 访问控制:# 私有字段与私有方法
 - §6.6 getter/setter 与访问器属性
 - §6.7 静态成员与 instanceof
 - §6.8 多态与鸭子类型
 - §6.9 设计原则:组合优于继承

§6.1 从 C 到面向对象:先看 C 的笨拙

```
// C 的 struct + 函数指针模拟"方法"(伪代码,展示痛点)
// typedef struct Shape {
//   double (*area)(const struct Shape* self); // 函数指针成员
//   double w, h;
// } Shape;
// 调用:shape.area(&shape) — "self" 要自己传,强转易错

// JS 的进化:先看"对象方法"的自然形态
const rect = {
  w: 4,
  h: 5,
  area() {
    return this.w * this.h; // this 自动是 rect,不用手动传
  },
};
console.log(rect.area()); // 20

// 工厂函数批量造对象(C 的结构体数组 → JS 的对象数组)
function makeRect(w, h) {
  return {
    w, h,
    area() { return this.w * this.h; },
  };
}
const shapes = [makeRect(4, 5), makeRect(2, 3)];
console.log(shapes[0].area()); // 20
console.log(shapes.map((s) => s.area()).join(',')); // 20,6
```

§6.2 对象与原型链:JS 的核心机制

💡 直觉先行:属性找不到,就沿着链向上找

每个 JS 对象都藏着一个"隐藏指针"指向另一个对象——它的**原型(prototype)**。读属性时,如果对象自己没有,引擎就去原型上找,再没有就继续向上,直到 `Object.prototype`,再没有返回 `undefined`。这条查找链就叫**原型链**。所有"继承来的方法"都是这样被找到的: `[1,2].push` 就是沿着数组 $\rightarrow$ `Array.prototype` $\rightarrow$ `Object.prototype` 找到的。

```
// 三个主角:prototype、__proto__、Object.create
// 1. __proto__:对象的隐藏原型指针(现在推荐用 Object.getPrototypeOf 读)
const dog = { bark() { return '汪汪'; } };
const puppy = { name: '小黄' };
Object.setPrototypeOf(puppy, dog); // 设置 puppy 的原型为 dog
console.log(puppy.bark());           // 汪汪 ← 自己没有,从原型 dog 找到
console.log(puppy.name);             // 小黄 ← 自己的属性
console.log(Object.getPrototypeOf(puppy) === dog); // true

// 2. Object.create(proto):以 proto 为原型创建新对象(最干净的写法)
const base = { greet() { return '你好'; } };
const child = Object.create(base);
child.age = 3;
console.log(child.greet());           // 你好 ← 原型链查找
console.log(child.age);               // 3 ← 自身属性

// 3. 原型链是一串:child → base → Object.prototype → null
console.log(child.greet === base.greet); // true,是同一个函数
console.log(Object.getPrototypeOf(Object.prototype)); // null,链的终点

// 4. 属性查找 vs 属性写入:读沿链走,写只落自身
const parent = { count: 1 };
const kid = Object.create(parent);
kid.count = 100;                      // 写自己的属性,不动原型
console.log(kid.count, parent.count); // 100 1
console.log(kid.hasOwnProperty('count')); // true,确认是自己的

// 5. hasOwnProperty:区分"自己的"与"继承的"
console.log(kid.hasOwnProperty('greet')); // false ← greet 来自原型
console.log('greet' in kid);              // true ← in 会沿链查找
```

⚠ 常见误区 6.1:直接改 __proto__ 是危险的

`__proto__` 是历史遗留的访问器,性能差且可读性糟;生产代码用 `Object.create` 建链、`Object.getPrototypeOf/setPrototypeOf` 查改。另外对象字面量不能直接写 `__proto__` 键以外的原型: `{ __proto__: base }` 是合法的设置原型语法(字面量特例),但团队规范通常统一用 `Object.create` 更清晰。

§6.3 构造函数与 new 的过程

ES5 时代的"类"就是构造函数 + 原型方法的组合。new 一个函数时,引擎做四件事:① 创建空对象;② 把空对象的原型指向构造函数的 prototype;③ 以该对象为 this 执行构造函数体;④ 返回该对象(除非构造函数显式返回对象)。

```
// 构造函数:约定首字母大写(与 C 的命名规范类似)
function Animal(name) {
  this.name = name;           // ③ this 是新建的空对象
}
// 方法挂到 prototype:所有实例共享,不重复创建
Animal.prototype.speak = function () {
  return `${this.name} 叫了一声`;
};
Animal.prototype.kind = '动物';

const a1 = new Animal('小狗'); // 完整四步
const a2 = new Animal('小猫');
console.log(a1.speak());       // 小狗 叫了一声
console.log(a1.speak === a2.speak); // true ← 共享同一个函数
console.log(a1.kind);          // 动物 ← 从原型拿的
console.log(a1 instanceof Animal); // true

// 手动实现 new 的等价过程(理解原理):
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype); // ① + ②
  const ret = Ctor.apply(obj, args);         // ③
  return (ret !== null && typeof ret === 'object') ? ret : obj; // ④
}
const a3 = myNew(Animal, '小兔');
console.log(a3.speak());        // 小兔 叫了一声
```

⚠ 常见误区 6.2:忘了 new 直接调用构造函数

`Animal('狗')` 不报错,但 `this` 走默认绑定——非严格模式会创建全局变量 `name`,严格模式抛 `TypeError`。生产防范:① 构造函数内第一行 `if (!new.target) throw new Error('必须用 new 调用');` ② 或直接用 `class`(`class` 构造函数忘了 `new` 一定抛错,这是 `class` 优于函数构造的安全点)。

§6.4 class 语法糖:背后的原型

ES2015 的 `class` 不引入新机制,它只是把"构造函数 + prototype + new"包成更清晰的语法。下面的 `class` 与上面的 `Animal` 是等价的:

```

class Animal {
  // 构造函数
  constructor(name) {
    this.name = name;
  }
  // 原型方法:挂在 Animal.prototype 上
  speak() {
    return `${this.name} 叫了一声`;
  }
  // 静态方法:挂在 Animal 本身上($6.7)
  static create(name) {
    return new Animal(name);
  }
}

const a = Animal.create('小牛');
console.log(a.speak());           // 小牛 叫了一声
console.log(a instanceof Animal); // true

// 验证"语法糖":方法确实在 prototype 上
console.log(typeof Animal);       // 'function' ← class 本质是函数
console.log(a.speak === Animal.prototype.speak); // true
console.log(Object.getOwnPropertyNames(Animal.prototype)); // [ 'constructor', 'speak' ]

// class 与函数构造的差异(class 更严格):
// 1. 必须 new 调用
// 2. 不能作为普通函数调用
// 3. 方法不可枚举、不可被 for...in 看到
for (const k in a) {
  console.log('可枚举:', k);      // 只有 name,方法不可枚举
}

```

6.4.1 继承:extends 与 super

```
class Dog extends Animal {
  constructor(name, breed) {
    super(name);          // 必须先调用 super 再访问 this
    this.breed = breed;
  }
  // 重写方法:覆盖父类实现
  speak() {
    return `${super.speak()}!(${this.breed})`; // super.speak 调父类版
  }
  fetch() {
    return `${this.name} 叼回飞盘`;
  }
}

const d = new Dog('旺财', '金毛');
console.log(d.speak());          // 旺财 叫了一声!(金毛)
console.log(d.fetch());          // 旺财 叼回飞盘
console.log(d instanceof Dog);    // true
console.log(d instanceof Animal); // true ← 原型链:实例→Dog.prototype→Animal.prototype
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype); // true

// 继承的本质就是原型链延长:
// d → Dog.prototype → Animal.prototype → Object.prototype → null
// 查找 speak:先看 d,再看 Dog.prototype(命中,重写版)
```

⚠ 常见误区 6.3:派生类构造器忘了 super

派生类(有 extends)的 constructor 里,在访问 this 之前必须先调用 super(...)——否则抛 ReferenceError。原因:this 由父类构造器初始化(与 C++ 的"基类先构造"同理)。若子类不写 constructor,引擎自动补 constructor(...args) { super(...args); }。

6.4.2 手写原型式继承:老代码能读懂

```
// 旧式继承:Object.create 手动连链(ES5 时代标准写法)
function OldDog(name, breed) {
  Animal.call(this, name); // 借用父构造函数初始化 this
  this.breed = breed;
}
OldDog.prototype = Object.create(Animal.prototype); // 链:OldDog.prototype 的原型是 Animal.prototype
OldDog.prototype.constructor = OldDog;             // 修正 constructor 指向
OldDog.prototype.speak = function () {
  return Animal.prototype.speak.call(this) + '!(旧式继承)';
};
const old = new OldDog('阿黄', '土狗');
console.log(old.speak()); // 阿黄 叫了一声!(旧式继承)
console.log(old instanceof OldDog); // true
```

§6.5 访问控制:# 私有字段

JS 没有 public/protected/private 修饰符。C++ 的 private 对应物是 # 私有字段(ES2022):真正的语言级私有,类外无法访问,包括 Object.keys、反射都看不到。访问控制只有两档:public(默认)与 #private;protected 没有直接对应,惯用法是用 # 私有 + 受保护方法放原型(文档约定)。

```
class BankAccount {
  #balance;                // 私有字段:必须先在类里声明
  #owner;

  constructor(owner, initial) {
    this.#owner = owner;
    this.#balance = initial;
  }

  deposit(n) {
    if (n <= 0) throw new Error('存款必须为正');
    this.#balance += n;
  }

  withdraw(n) {
    if (n > this.#balance) return false;
    this.#balance -= n;
    return true;
  }

  get balance() {           // 只读出口(getter, §6.6)
    return this.#balance;
  }
}

const acc = new BankAccount('李雷', 100);
acc.deposit(50);
console.log(acc.balance);    // 150
console.log(acc.withdraw(200)); // false ← 余额不足
console.log(acc.balance);    // 150
// console.log(acc.#balance); // ✗ SyntaxError:私有字段类外不可见!
// console.log(acc['#balance']); // ✗ 也不行,私有字段不是普通键

// 私有方法:#method() 同样类外不可调
class Counter {
  #count = 0;
  #valid(n) { return n >= 0; }
  add(n) {
    if (!this.#valid(n)) throw new Error('不能加负数');
    this.#count += n;
  }
  get value() { return this.#count; }
}

const c = new Counter();
c.add(3);
console.log(c.value);        // 3
```

工程建议 6.1:私有字段 vs 闭包 vs 下划线约定

三种"私有"方案对比:① `#private` 语言级强制,ES2022+,生产首选;② 闭包私有(第4章账户例子)灵活但每个实例一份闭包内存,且无法被 `this` 调用;③ `_prefix` 只是命名约定,不防君子不防小人,仅用于"文档化意图"。写库(给别人用)必须用 `#`;写应用内部代码,团队约定一致即可,但 ESLint 的 `no-underscore-dangle` 常强制用 `#`。

§6.6 getter / setter:访问器属性

C 的结构体字段是裸数据;生产代码常用 `getter/setter` 控制读写(校验、计算属性、惰性求值)。语法:`get/set` 定义在类里,或用 `Object.defineProperty` 定义在对象上。

```
class Temperature {
  #celsius;
  constructor(c) { this.#celsius = c; }

  // 读:celsius → 华氏
  get fahrenheit() {
    return this.#celsius * 9 / 5 + 32;
  }
  // 写:从华氏反推摄氏(带校验)
  set fahrenheit(f) {
    if (!Number.isFinite(f)) throw new Error('非法温度');
    this.#celsius = (f - 32) * 5 / 9;
  }
  get celsius() { return this.#celsius; }
}
const t = new Temperature(100);
console.log(t.fahrenheit);    // 212
t.fahrenheit = 32;
console.log(t.celsius);      // 0 ← setter 反向换算

// 惰性求值:第一次访问才计算
class Expensive {
  #cache = null;
  get data() {
    if (this.#cache === null) {
      console.log('首次计算...');
      this.#cache = { heavy: true, at: Date.now() };
    }
    return this.#cache;
  }
}
const e = new Expensive();
console.log(e.data.at > 0);    // 首次计算... true
console.log(e.data === e.data); // true,第二次直接命中缓存
```

§6.7 静态成员与 instanceof

```
class MathUtils {
  static PI = 3.14159265358979;          // 静态字段(ES2022)
  static square(n) { return n * n; }      // 静态方法
  #secret = '实例私有';

  static create() { return new MathUtils(); }
}
// 静态成员挂在类本身,不挂在实例
console.log(MathUtils.PI);                // 3.14159265358979
console.log(MathUtils.square(9));         // 81
const mu = MathUtils.create();
// console.log(mu.PI);                    // undefined ← 实例没有静态成员
// console.log(mu.square);                // undefined

// 静态方法里的 this 是类本身(用于工厂/单例)
class Config {
  static #instance = null;
  static get() {
    if (this.#instance === null) {
      this.#instance = new Config();
    }
    return this.#instance;                // 单例模式
  }
}
console.log(Config.get() === Config.get()); // true, 同一个实例

// instanceof: 沿着原型链查
console.log([] instanceof Array);         // true
console.log([] instanceof Object);        // true ← 链更长也能匹配
console.log(mu instanceof MathUtils);     // true
// 跨 realm 陷阱: iframe/worker 里的数组 instanceof 主窗口 Array 为 false
console.log(Array.isArray([]));           // true ← 生产用 Array.isArray 更稳
```

§6.8 多态与鸭子类型

C 的多态靠函数指针表 + 强制转换; JS 的多态天然: 只要对象有同名方法, 就能被同样调用——鸭子类型 ("走起来像鸭子、叫起来像鸭子, 它就是鸭子")。不需要接口声明, 不需要继承关系。

```
// 鸭子类型:三个无关类,有相同方法即可被统一调用
class Dog {
  constructor(name) { this.name = name; }
  speak() { return `${this.name}:汪汪`; }
}
class Cat {
  constructor(name) { this.name = name; }
  speak() { return `${this.name}:喵喵`; }
}
class Robot {
  constructor(name) { this.name = name; }
  speak() { return `${this.name}:哔哔(检测到人类)`; }
}

// 多态调用:不关心具体类型
function callAll(animals) {
  return animals.map((a) => a.speak()); // 只要求有 speak 方法
}
console.log(callAll([new Dog('大黄'), new Cat('小橘'), new Robot('R2')]));
// [ '大黄:汪汪', '小橘:喵喵', 'R2:哔哔(检测到人类)' ]

// 生产防御:先验证方法存在(可选链 + typeof)
function safeSpeak(any) {
  if (typeof any?.speak !== 'function') {
    return '无法说话';
  }
  return any.speak();
}
console.log(safeSpeak(new Dog('小黑'))); // 小黑:汪汪
console.log(safeSpeak({})); // 无法说话
```

★ 重点结论:接口在 JS 里是"约定"不是"类型"

C++/Java 用接口/抽象类在 编译期 保证"一定实现某方法";JS 在 运行期 靠鸭子类型,接口只是文档(或用 JSDoc/TypeScript 标注,见第14章)。代价是错误后置:少写了 speak 方法,运行时才报错。生产缓解:入口处校验(如上面的 safeSpeak)、TypeScript、以及充分的单元测试。

§6.9 设计原则:组合优于继承

原型继承是单链,深了会又脆又难懂(经典反例:Animal→Mammal→Dog→Poodle 五层链)。JS 社区共识:多用组合(对象持有对象),少用继承(对象是另一个对象的子类)。判断标准:有"is-a"关系(狗是动物)且复用方法时用继承;只是"需要某能力"时用组合。

```
// 组合示例:飞行/游泳能力用对象拼装,而不是造"会飞的狗"类
const flyable = (name) => ({
  fly() { return `${name} 飞起来了`; },
});
const swimmable = (name) => ({
  swim() { return `${name} 游了起来`; },
});

class Duck {
  constructor(name) {
    this.name = name;
    Object.assign(this, flyable(name), swimmable(name)); // 组合能力
  }
}

const d2 = new Duck('唐老鸭');
console.log(d2.fly());           // 唐老鸭 飞起来了
console.log(d2.swim());          // 唐老鸭 游了起来

// 对照:继承路线要造 FlyAnimal→SwimAnimal→FlyingSwimmingDuck,层级爆炸

// 工厂 + 组合:完全不用 class 的"对象工厂"风格同样优秀
function createLogger(level = 'info') {
  const timestamps = [];
  return {
    log(msg) {
      timestamps.push(Date.now()); // 闭包私有状态
      console.log(`[${level}] ${msg}`);
    },
    count() { return timestamps.length; },
  };
}

const logger = createLogger('warn');
logger.log('磁盘空间不足');
logger.log('连接超时');
console.log('日志条数:', logger.count()); // 2
```



工程建议 6.2:生产中的类设计检查表

① 类层级不超过 2 层(父 + 子);再深先考虑组合;② 每个类单一职责,方法数超过 15 个考虑拆分;③ 私有状态全部 #,对外只暴露最小方法集;④ 依赖注入:构造器接收依赖对象,不用全局 new,便于测试;⑤ 优先"对象工厂 + 组合"起步,真需要继承时再加 extends——少即是多。

本章速查表

主题	速查要点
对象本质	属性表 + 隐藏原型指针;属性查找沿原型链向上
原型链主角	prototype(构造函数的)、__proto__(对象的,历史)、Object.create(建链)
读属性 vs 写属性	读沿链找;写只落自身属性
hasOwnProperty	区分自己的/继承的;in 运算符沿链查找
new 四步	建空对象 → 连原型 → this 执行 → 返回对象
class	语法糖:本质是函数 + prototype;方法不可枚举
extends/super	链变长;派生类构造器必须先 super 再 this
旧式继承	Object.create(父.prototype) + 修正 constructor
私有	#field/#method 语言级私有;无 protected,用 #+ 文档约定
getter/setter	读写拦截、计算属性、惰性求值
静态成员	static 挂类本身,实例不可见;静态 this 是类
instanceof	沿原型链查;跨 realm 失效;数组用 Array.isArray
多态	鸭子类型:有同名方法即可统一调用,无需接口
组合优先	is-a 用继承;要能力用组合/混入;层级 ≤2

最小必会清单

- 能解释原型链查找过程,并画出 实例→原型→Object.prototype→null 链
- 能用 Object.create 建链并验证 hasOwnProperty 的区别
- 能背诵 new 的四步并手写 myNew
- 能写出 class 与"构造函数 + prototype"的等价转换
- 能说出 extends 后原型链的形态(Dog.prototype 的原型是 Animal.prototype)
- 能写出 # 私有字段的类,并说出类外访问的报错
- 能用 getter/setter 实现"摄氏/华氏双向换算"
- 能解释鸭子类型多态并给出一个统一调用示例
- 能说出"组合优于继承"的判断标准
- 能说出 instanceof 的跨 realm 陷阱与 Array.isArray

自测题

- Q** 1. 写出输出:const p = {a:1}; const c = Object.create(p); console.log(c.a, c.hasOwnProperty('a'), 'a' in c)

✅ 参考答案

1、false、true。c 自己没有 a,查找沿原型链命中 p.a=1;hasOwnProperty 只看自身;in 沿链查找所以 true。

Q 2. class 与构造函数定义"类"的差别有哪些?(至少 3 条)

✅ 参考答案

① class 必须 new 调用,普通调用抛错;② class 方法不可枚举;③ class 语法更清晰,支持 extends/super/static/# 私有;④ class 有块级作用域(不提升);底层都是原型链。

Q 3. 派生类构造器里为什么必须先 super()?不写会怎样?

✅ 参考答案

this 由父类构造器初始化,访问 this 前必须完成 super;违反时抛 ReferenceError。若子类不写构造器,引擎自动补 super(...args)。

Q 4. 写出用 Object.create 手工实现继承(Animal → OldDog)的关键四行,并说明每行作用。

✅ 参考答案

① Animal.call(this, name) 借用父构造初始化;② OldDog.prototype = Object.create(Animal.prototype) 连链;③ OldDog.prototype.constructor = OldDog 修正指向;④ 在 prototype 上覆盖方法。

Q 5. 如何让类外代码读不到 #balance 但能读 balance?这两种方式分别叫什么?

✅ 参考答案

声明 #balance 私有字段,对外提供 get balance() { return this.#balance; }。前者是私有字段(语言级),后者是 getter 访问器。

Q 6. 一个"能飞能游"的对象,组合与继承各怎么写?生产推荐哪个?

✅ 参考答案

组合: Object.assign(this, flyable(), swimmable());继承需要造中间层类,层级爆炸。生产推荐组合。

章末实验题:几何图形系统

实验 6:几何图形系统(继承 + 多态 + 私有 + 访问器)

实验目标:实现圆形/矩形/三角形组成的图形系统:基类定义接口,子类实现面积与周长,支持统一汇总,全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:class、构造器、super):基类 Shape 存 name;Circle/Rect/Triangle 继承并用 super 初始化
- 任务 2(覆盖:# 私有字段、getter/setter):Circle 用 #radius 私有字段 + get/set radius 校验非负;Rect 用 #w/#h
- 任务 3(覆盖:多态、鸭子类型):统一接口 area()/perimeter() 三个子类各自实现;汇总函数只管调用,不查类型
- 任务 4(覆盖:原型链、instanceof):遍历图形数组输出 instanceof 结果与原型链验证(Dog 式链)
- 任务 5(覆盖:静态成员、工厂):Shape.create(type, ...) 静态工厂按类型造图形;MathUtils 式静态常量 PI
- 任务 6(覆盖:组合优先):实现"可缩放的图形" mixin(scale 方法),用 Object.assign 组合进 Shape,而不是造子类
- 任务 7(覆盖:Object.create、hasOwnProperty):用 Object.create 演示一个"临时图形"对象,验证属性归属

实验环境:Node.js 20+,运行 `node shapes.js`。

第一步 **写基类:**Shape 定义 constructor(name) 与接口方法(抛"未实现"错误)

第二步 **写三个子类:**各自实现 area/perimeter;注意先 super 再访问 this

第三步 **mixin 与工厂:**scalable 混入、Shape.create 工厂、演示与汇总输出

✅ 参考答案要点

核心:基类接口 + 三子类多态 + #私有字段与访问器 + 静态工厂 + mixin 组合。约 130 行,覆盖全章知识点。

```
// shapes.js — Node.js 20+ 运行:node shapes.js
'use strict';

// 1. 基类:定义接口(覆盖:class、构造器、多态契约)
class Shape {
  constructor(name) {
    this.name = name;
  }
  area() {
    throw new Error(`${this.name} 未实现 area()`);
  }
  perimeter() {
    throw new Error(`${this.name} 未实现 perimeter()`);
  }
  describe() {
    return `${this.name}: 面积 ${this.area().toFixed(2)}, 周长 ${this.perimeter().toFixed(2)}`;
  }
}

// 2. 圆形:# 私有字段 + getter/setter 校验(覆盖:私有、访问器)
class Circle extends Shape {
  #radius;
  constructor(radius) {
    super('圆形');
    this.radius = radius;      // 走 setter 校验
  }
  get radius() { return this.#radius; }
  set radius(r) {
    if (!Number.isFinite(r) || r <= 0) {
      throw new Error('半径必须为正数');
    }
    this.#radius = r;
  }
  area() { return Math.PI * this.#radius ** 2; }
  perimeter() { return 2 * Math.PI * this.#radius; }
}

// 3. 矩形:两个 # 私有字段
class Rect extends Shape {
  #w;
  #h;
  constructor(w, h) {
    super('矩形');
    if (w <= 0 || h <= 0) throw new Error('宽高必须为正');
    this.#w = w;
    this.#h = h;
  }
  area() { return this.#w * this.#h; }
  perimeter() { return 2 * (this.#w + this.#h); }
}

// 4. 三角形:海伦公式
class Triangle extends Shape {
  #a;
  #b;
```

```

#c;
constructor(a, b, c) {
  super('三角形');
  if (a + b <= c || a + c <= b || b + c <= a) {
    throw new Error('三边无法构成三角形');
  }
  this.#a = a; this.#b = b; this.#c = c;
}
area() {
  const s = this.perimeter() / 2;
  return Math.sqrt(s * (s - this.#a) * (s - this.#b) * (s - this.#c));
}
perimeter() { return this.#a + this.#b + this.#c; }
}

```

// 5. 静态工厂 + 静态常量(覆盖:static、单例式出口)

```

class ShapeFactory {
  static PI = Math.PI;
  static create(type, ...args) {
    switch (type) {
      case 'circle': return new Circle(...args);
      case 'rect': return new Rect(...args);
      case 'triangle': return new Triangle(...args);
      default: throw new Error(`未知图形类型:${type}`);
    }
  }
}

```

// 6. 组合:mixin 混入缩放能力(覆盖:组合优于继承)

```

const scalable = (self) => ({
  scale(factor) {
    if (factor <= 0) throw new Error('缩放系数必须为正');
    // 通过实例的 radius/w/h 访问器缩放(鸭子类型)
    if (typeof self.radius === 'number') {
      self.radius *= factor;
    } else if (self instanceof Rect) {
      // 矩形私有字段只能通过内部方法缩放,这里演示"组合兜底"
      self._scaleInternal?.(factor);
    }
    return self;
  },
});

```

// 为 Rect 提供内部缩放钩子(私有字段 + 内部方法)

```

Rect.prototype._scaleInternal = function (f) {
  this.#w *= f;
  this.#h *= f;
};

```

// 7. 鸭子类型多态汇总(覆盖:多态、instanceof 验证)

```

function summarize(shapes) {
  let total = 0;
  for (const s of shapes) {
    console.log(`- ${s.describe()}`);
    console.log(`  instanceof Shape: ${s instanceof Shape}`);
  }
}

```

```

    total += s.area();
  }
  return total;
}

// 8. Object.create 演示(覆盖:Object.create、hasOwnProperty)
function protoDemo() {
  const proto = { tag: '几何对象', log() { console.log('来自原型的方法'); } };
  const tmp = Object.create(proto);
  tmp.name = '临时图形';
  console.log('tmp.tag(来自原型):', tmp.tag);
  console.log('hasOwnProperty:', tmp.hasOwnProperty('tag'), tmp.hasOwnProperty('name'));
  console.log('in 运算符:', 'tag' in tmp);
}

// 9. 主流程
function main() {
  // 工厂创建 + 组合缩放
  const shapes = [
    ShapeFactory.create('circle', 5),
    ShapeFactory.create('rect', 4, 6),
    ShapeFactory.create('triangle', 3, 4, 5),
  ];
  Object.assign(shapes[0], scalable(shapes[0])); // 给圆形混入缩放
  shapes[0].scale(2); // 半径 5 → 10

  const total = summarize(shapes);
  console.log('总面积:', total.toFixed(2));

  // 访问器校验演示
  try {
    const bad = new Circle(-1);
    console.log(bad.describe());
  } catch (err) {
    console.log('非法半径被拦截:', err.message);
  }

  // 私有字段不可类外访问(注释掉的代码会报语法错)
  // console.log(shapes[0].#radius); // ✖ SyntaxError

  // 原型链验证
  const c = shapes[0];
  console.log('原型链:', c instanceof Circle, c instanceof Shape, c instanceof Object);
  console.log('Circle.prototype 的原型是 Shape.prototype:',
    Object.getPrototypeOf(Circle.prototype) === Shape.prototype);

  protoDemo();
}

main();

```

设计说明:Shape 用"抛锚"定义接口契约(鸭子类型语言中接口的常见写法);Circle 的 setter 拦截非法半径;scalable mixin 演示组合优于继承——能力用 Object.assign 混入而不是造"可缩放圆"子类;summarize 只依赖

area()/describe() 接口,不看具体类型;静态工厂集中创建逻辑;protoDemo 收尾验证 Object.create 与原型的属性归属。运行输出包含三种图形的面积周长、总面积、非法输入拦截、原型链验证与原型演示五部分。

下一章预告:第7章 文件I/O —— fs 模块的读写、路径与目录、流式大文件处理、JSON 序列化,与 C 的 fopen/fread 全家对照。

第7章 文件 I/O

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 node:fs 模块的三种风格:同步/回调/Promise,并理解生产选型
- 掌握文本读写与编码(UTF-8、BOM),二进制读写(Buffer)与 C 的 fread/fwrite 对照
- 掌握 path 模块:join/resolve、basename/dirname/extname,跨平台分隔符
- 掌握目录遍历、创建、删除与文件存在性判断
- 掌握大文件流式处理:createReadStream/readline,理解内存占用差异
- 掌握 JSON 序列化的坑:undefined、NaN、循环引用、toJSON
- 掌握标准流与命令行参数,完成日志解析统计工具实验

💡 从 C 到 JavaScript:本章主题如何迁移

C 的文件操作是 fopen/fread/fwrite/fclose 四件套,配合 fseek/ftell、strtok 逐行解析,每一步都要检查返回值,EOF 与错误全靠 errno 与 feof 判断。JS 的 node:fs 模块把全套映射过来,而且更进一步:每个操作都有同步、回调、Promise 三种形态,并配套了目录、权限、流式与 watch 等 C 标准库没有的高级能力。

最大的心智差异是异步:C 的 fread 是阻塞的,读完才返回;Node 的异步 readFile 立即返回一个 Promise,数据到了才回调。服务器场景必须用异步(否则一次磁盘读就把整个事件循环卡死),脚本场景可以用同步版本(init 阶段读配置)。第9章将深讲事件循环,本章先用"await 等结果"的姿势。

二进制视角也有变化:C 直接 fwrite 结构体(对齐、字节序、版本兼容都是坑);JS 用 Buffer 精确控制字节,配合 JSON/字符串化做跨版本兼容的序列化。文件不存在时,C 的 fopen 返回 NULL;JS 的异步 API 抛 ENOENT 错误,需要 try/catch 或 existsSync 预判。

最后,路径处理:C 的 "a/b" 拼接在 Windows 上就是事故;JS 用 path.join/path.resolve 自动处理分隔符,并用 path.win32/path.posix 保证跨平台一致。

📦 前置知识自查

- C 的 fopen/fclose/fread/fwrite/fprintf/fscanf 全套
- C 的文本模式与二进制模式("rb"/"wb")
- C 的 errno、feof、ferror 错误处理模式
- C 的 getline/strtok 逐行解析
- 第5章 Buffer 基础(字节与编码)

🚀 本章学习路线

1. **第一阶段(读写)**:读 §7.1–§7.2,跑通三种风格的读写示例
2. **第二阶段(路径目录)**:读 §7.3,练习 path 各 API 输出
3. **第三阶段(流与序列化)**:读 §7.4–§7.6,重点理解流式处理的内存优势
4. **第四阶段(实验)**:完成章末"日志解析与统计工具"
5. **验收标准**:能独立写出"读配置 → 处理 → 写结果"的完整异步流程,并处理 ENOENT

本章知识导图

- 第7章 文件 I/O
 - §7.1 基础读写:三种风格、编码与 BOM
 - §7.2 二进制 I/O:Buffer 读写、与 fread/fwrite 对照
 - §7.3 路径与目录:join/resolve、遍历、创建删除
 - §7.4 大文件流式处理:readline、内存对比
 - §7.5 序列化:JSON 的坑与安全做法
 - §7.6 标准流:stdin/stdout/stderr、命令行参数
 - §7.7 生产场景:配置文件、CSV、日志追加

§7.1 基础读写:同步、回调与 Promise

C	Node(fs 模块)	说明
fopen(path, "r")	fs.readFile	整个文件读入内存/回调
fopen(path, "w")	fs.writeFile	覆盖写
fopen(path, "a")	fs.appendFile	追加写
fread(buf, ...)	fs.read(fd, buf, ...)	低级读(带句柄)
fclose(fp)	fs.close(fd) / 自动关闭	readFile 全程自动管理
fseek/ftell	fs.readSync 的 position 参数	少见,生产基本用不着
feof	读空即 EOF,不报错	文件读完返回空串,不设错误

```
const fs = require('node:fs');

// 1. 同步风格:init 阶段/脚本工具可用(会阻塞事件循环)
fs.writeFileSync('/tmp/hello.txt', '你好,文件系统\n');
const text = fs.readFileSync('/tmp/hello.txt', 'utf8');
console.log('同步读:', text.trim());

// 2. 回调风格:老代码常见,链式回调易读难写
fs.readFile('/tmp/hello.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('读失败:', err.message);
    return;
  }
  console.log('回调读:', data.trim());
});

// 3. Promise 风格:生产首选(与第9章 async/await 配套)
async function main() {
  await fs.promises.writeFile('/tmp/p.txt', 'Promise 写入\n');
  const data = await fs.promises.readFile('/tmp/p.txt', 'utf8');
  console.log('Promise 读:', data.trim());

  // 追加:对应 C 的 "a" 模式
  await fs.promises.appendFile('/tmp/p.txt', '追加一行\n');
  console.log('追加后:', (await fs.promises.readFile('/tmp/p.txt', 'utf8')).trim());
}
main().catch((err) => console.error('异步出错:', err.message));
```

⚠ 常见误区 7.1:readFile 的文件不存在与编码问题

文件不存在时,同步版抛 ENOENT 异常,回调版 `err.code === 'ENOENT'`,Promise 版 reject 带 code。生产防御: `fs.existsSync` 预判 + try/catch 兜底(存在与可读之间可能竞态,兜底不能省)。编码:不传 'utf8' 时 readFile 返回 Buffer;传了 'utf8' 但文件带 BOM 时,字符串开头会残留 U+FEFF 字符,需 `str.replace(/^/, '')` 去除。

§7.2 二进制 I/O:Buffer 与 fread/fwrite 对照

```
// 二进制写:与 C 的 fwrite(buf, size, n, fp) 对应
const data = Buffer.from([0x01, 0x02, 0xff, 0x00]);
fs.writeFileSync('/tmp/data.bin', data);
console.log('二进制已写:', data.toString('hex')); // 0102ff00

// 二进制读:不传编码直接得到 Buffer(不要用 utf8 解码二进制!)
const raw = fs.readFileSync('/tmp/data.bin');
console.log('二进制读回:', raw.toString('hex'));
console.log('字节数:', raw.length); // 4

// 结构化数据二进制定义:C 的 fwrite(struct) 是字节级;JS 用 DataView 精确布局
function writeRecord(path, records) {
  const ab = new ArrayBuffer(records.length * 8); // 每条:uint32 + float32
  const dv = new DataView(ab);
  records.forEach((r, i) => {
    dv.setUint32(i * 8, r.id); // 大端 uint32
    dv.setFloat32(i * 8 + 4, r.score); // 大端 float32
  });
  fs.writeFileSync(path, Buffer.from(ab));
}

function readRecords(path) {
  const buf = fs.readFileSync(path);
  const dv = new DataView(buf.buffer, buf.byteOffset, buf.byteLength);
  const out = [];
  for (let i = 0; i < dv.byteLength / 8; i++) {
    out.push({ id: dv.getUint32(i * 8), score: dv.getFloat32(i * 8 + 4) });
  }
  return out;
}

writeRecord('/tmp/rec.bin', [{ id: 1001, score: 87.5 }, { id: 1002, score: 92.25 }]);
console.log('结构读回:', JSON.stringify(readRecords('/tmp/rec.bin')));

// 低级句柄操作(对应 fopen/fclose):
const fd = fs.openSync('/tmp/fd.bin', 'w');
try {
  fs.writeSync(fd, Buffer.from('fd 写入'));
} finally {
  fs.closeSync(fd); // 第5章:句柄必须成对关闭
}
console.log(fs.readFileSync('/tmp/fd.bin', 'utf8')); // fd 写入
```

⚠ 常见误区 7.2:把二进制文件当 utf8 读

任何非文本文件(图片、压缩包、二进制协议)都不要传 'utf8' 读:字节被按 UTF-8 解码会得到乱码甚至被替换字符破坏。规则:文本文件传编码,二进制文件不传,拿到 Buffer 后用 DataView/位运算处理。

§7.3 路径与目录

```
const path = require('node:path');

// 1. 拼接与解析:C 的 "a/b" 手拼在 Windows 上是灾难
console.log(path.join('data', 'sub', 'file.txt')); // data\sub\file.txt(平台分隔符)
console.log(path.resolve('data', '..', 'conf.json')); // 绝对路径,.. 正常解析
console.log(path.basename('/a/b/c.txt'));           // c.txt
console.log(path.dirname('/a/b/c.txt'));             // /a/b
console.log(path.extname('/a/b/c.txt'));             // .txt

// 2. 跨平台:模块统一用 / 或 path.join;windows 上的盘符由 path 处理
// path.win32 与 path.posix 可以强制指定规则(如生成给别的系统用的路径)

// 3. 存在性、类型、信息
const p = '/tmp/test-path.txt';
fs.writeFileSync(p, 'x');
console.log(fs.existsSync(p));                       // true
console.log(fs.statSync(p).size);                    // 1,类似 C 的 fseek+ftell 取大小
console.log(fs.statSync(p).isFile());                // true
console.log(fs.statSync(p).isDirectory());           // false

// 4. 目录操作
fs.mkdirSync('/tmp/jsdir/sub', { recursive: true }); // 像 mkdir -p
fs.writeFileSync('/tmp/jsdir/sub/a.txt', 'A');
fs.writeFileSync('/tmp/jsdir/sub/b.txt', 'B');

// 5. 目录遍历(递归):替代 C 的 opendir/readdir 手写递归
function walk(dir, depth = 0) {
  const entries = fs.readdirSync(dir, { withFileTypes: true });
  for (const e of entries) {
    console.log(' '.repeat(depth) + e.name + (e.isDirectory() ? '/' : ''));
    if (e.isDirectory()) walk(path.join(dir, e.name), depth + 1);
  }
}
walk('/tmp/jsdir');

// 6. 删除
fs.rmSync('/tmp/jsdir', { recursive: true, force: true }); // 类似 rm -rf
console.log('删除后存在:', fs.existsSync('/tmp/jsdir'));   // false
```



工程建议 7.1:永远用 path.join 拼路径,禁止字符串拼接

'a' + '/' + 'b' 在 Windows 上可能得到 a/b(可接受)但在拼接用户输入时可能产生路径穿越(../)。统一 path.join/path.resolve;读取用户可控文件名时先 path.basename 剥离目录部分,再拼到白名单目录,这是文件服务的安全底线。

§7.4 大文件流式处理:内存优势

readFile 把整个文件读进内存:1GB 日志文件就吃 1GB+ 堆。流式处理一次只读一小块:内存占用与文件大小无关。Node 的 readline 可以逐行消费任意大小的文件。

```

const readline = require('node:readline');
const { createReadStream } = require('node:fs');

// 生成一个 1MB 的测试日志
async function genLog() {
  let lines = [];
  for (let i = 0; i < 20000; i++) {
    lines.push(`${Date.now() + i} [INFO] 请求 #${i} 完成 ${i % 7}ms`);
    if (lines.length > 5000) {
      await fs.promises.appendFile('/tmp/big.log', lines.join('\n') + '\n');
      lines = [];
    }
  }
  await fs.promises.appendFile('/tmp/big.log', lines.join('\n') + '\n');
}

// 流式逐行统计:内存占用恒定
async function analyze() {
  const rl = readline.createInterface({
    input: createReadStream('/tmp/big.log'),
    crlfDelay: Infinity, // 处理 \r\n
  });
  let count = 0;
  let totalMs = 0;
  for await (const line of rl) { // 逐行异步迭代
    const m = line.match(/完成 (\d+)ms/);
    if (m) {
      count++;
      totalMs += Number(m[1]);
    }
  }
  console.log(`总行数统计:${count} 条,平均耗时 ${((totalMs / count).toFixed(2))}ms`);
}

async function main() {
  await genLog();
  console.log('文件大小:', fs.statSync('/tmp/big.log').size, '字节');
  await analyze();
  console.log('内存占用:', Math.round(process.memoryUsage().heapUsed / 1024 / 1024), 'MB');
}
main().catch((e) => console.error(e));

```

★ 重点结论:流式处理三件套

① `createReadStream + readline`:逐行处理任意大文本(日志/CSV);② `createWriteStream`:分批写出,避免一次性拼大字符串;③ 管道 `pipe`:流到流的搬运,如 `readStream.pipe(writeStream)` 做文件复制,自动背压(内存不暴涨)。生产上传下载、日志切割都是流的天下。

```
// 流式复制(背压自动处理)
const src = createReadStream('/tmp/big.log');
const dst = fs.createWriteStream('/tmp/big-copy.log');
src.pipe(dst);
dst.on('finish', () => {
  console.log('复制完成,大小一致:', fs.statSync('/tmp/big-copy.log').size);
});
```

§7.5 序列化:JSON 与它的坑

C 的序列化靠 fwrite 结构体(脆弱:对齐、字节序、版本)或手写格式;JS 有原生 JSON。JSON 是现代 JS 程序的事实序列化格式:配置文件、HTTP 载荷、Redis 缓存全用它。但 JSON 有四个著名坑:

```

// 坑 1:undefined / 函数 / Symbol 属性被丢弃
const obj = { a: 1, b: undefined, c: () => {}, d: Symbol('s') };
console.log(JSON.stringify(obj)); // {"a":1} ← 三个都丢了!
// 数组里的 undefined 变成 null
console.log(JSON.stringify([1, undefined, 3])); // [1,null,3]

// 坑 2:NaN / Infinity 序列化后变成 null
console.log(JSON.stringify({ x: NaN, y: Infinity })); // {"x":null,"y":null}

// 坑 3:循环引用报错
const cycle = { name: '环' };
cycle.self = cycle;
// console.log(JSON.stringify(cycle)); // ✖ TypeError: Converting circular structure

// 坑 4:BigInt 不支持
// console.log(JSON.stringify(1n)); // ✖ TypeError

// 坑 5:parse 的健壮性
const bad = '{"a":1,}';
try {
  JSON.parse(bad);
} catch (e) {
  console.log('JSON.parse 报错:', e.message); // Unexpected token
}

// 生产方案:toJSON 钩子自定义序列化
class Order {
  constructor(id, total) {
    this.id = id;
    this.total = total;
  }
  toJSON() { // stringify 自动调用
    return { id: this.id, total: this.total, currency: 'CNY' };
  }
}
console.log(JSON.stringify(new Order(1001, 99.5)));
// {"id":1001,"total":99.5,"currency":"CNY"}

// BigInt 兜底:转字符串传输
const big = { orderId: 9007199254740993n };
console.log(JSON.stringify(big, (k, v) => (typeof v === 'bigint' ? v.toString() : v)));
// {"orderId":"9007199254740993"}

```

⚠ 常见误区 7.3:JSON.parse 不校验直接当对象用

JSON.parse 返回的可能是 null、数字、数组,不只是对象: JSON.parse('null') 得到 null。解析后先验证形状再取字段: if (data && typeof data === 'object' && Array.isArray(data.list)) ,否则 undefined 一路传播,报错在十行之外,难排查。生产推荐用 zod/joi 这类校验库做运行时 schema 校验。

§7.6 标准流与命令行参数

```
// 标准流三件套:与 C 的 stdin/stdout/stderr 一一对应
process.stdout.write('写到标准输出\n');    // 相当于 printf
process.stderr.write('写到标准错误\n');    // 相当于 fprintf(stderr, ...)
console.error('console.error 也是 stderr');

// 读取 stdin 整行输入(脚本场景)
const readline = require('node:readline/promises');
async function ask() {
  const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
  const name = await rl.question('你的名字:');
  console.log('你好,', name);
  rl.close();
}

// 命令行参数:C 的 argc/argv → process.argv
// node tool.js a b c
console.log('argv[0](node):', process.argv[0]);
console.log('argv[1](脚本):', process.argv[1]);
console.log('参数列表:', process.argv.slice(2)); // [ 'a', 'b', 'c' ]

// 生产参数解析:手写或 minimist/commander
function parseArgs(argv) {
  const out = { help: false, port: 3000 };
  for (let i = 0; i < argv.length; i++) {
    if (argv[i] === '--port') out.port = Number(argv[i + 1]);
    if (argv[i] === '--help') out.help = true;
  }
  return out;
}
console.log(parseArgs(['--port', '8080'])); // { help: false, port: 8080 }

// 退出码:C 的 return 0/非0 → process.exitCode
process.exitCode = 0;    // 成功;非 0 表示失败(CI 据此判断)
```

§7.7 生产场景:配置、CSV 与日志

```
// 1. 配置文件:JSON 配置读取 + 默认值合并
async function loadConfig(path) {
  let file = {};
  if (fs.existsSync(path)) {
    file = JSON.parse(await fs.promises.readFile(path, 'utf8'));
  }
  return {
    host: file.host ?? '0.0.0.0',    // ?? 保留合法空值
    port: file.port ?? 3000,
    debug: file.debug ?? false,
  };
}

// 2. CSV 简易处理:逗号分隔 + 引号剥离(生产用 csv-parse 库)
function parseCsvLine(line) {
  return line.split(',').map((f) => f.trim().replace(/^"|"$/g, ''));
}

// 3. 日志追加:appendFile 天然支持并发追加(C 的 "a" 模式)
async function logLine(level, msg) {
  const line = `[${new Date().toISOString()}] [${level}] ${msg}\n`;
  await fs.promises.appendFile('/tmp/app.log', line, 'utf8');
}

// 4. 主流程:读配置 → 解析 CSV → 记录日志 → 输出统计
async function main() {
  const cfg = await loadConfig('/tmp/app-config.json');
  console.log('配置:', JSON.stringify(cfg));

  fs.writeFileSync('/tmp/students.csv', 'name,score\n"李雷",88\n"韩梅梅",95\n');
  const rows = (await fs.promises.readFile('/tmp/students.csv', 'utf8'))
    .trim().split('\n').map(parseCsvLine);
  console.log('CSV 行数:', rows.length - 1);    // 去掉表头

  await logLine('INFO', `配置加载完成,端口 ${cfg.port}`);
  await logLine('INFO', `CSV 解析 ${rows.length - 1} 条记录`);
  const log = await fs.promises.readFile('/tmp/app.log', 'utf8');
  console.log('日志内容:\n' + log.trim());
}

main().catch((e) => console.error('主流程失败:', e));
```

工程建议 7.2:日志不是 console.log 就完事

生产日志三要求:① 结构化(JSON 行,便于 ELK/Loki 检索);② 分级(debug/info/warn/error,环境变量控制级别);③ 轮转(按大小/日期切割,用 pino/winston 或系统 logrotate)。console.log 只适合开发期与演示代码,进生产前替换为日志库。

本章速查表

主题	速查要点
三种风格	Sync(脚本/init)、回调(老代码)、fs.promises(生产首选)
写文件	writeFile 覆盖;appendFile 追加;都自动处理打开/关闭
读文本	readFile(path, 'utf8');带 BOM 时去
读二进制	不传编码 → Buffer;禁止 utf8 解二进制
句柄操作	openSync/readSync/writeSync/closeSync 成对,finally 关闭
文件不存在	ENOENT;existsSync 预判 + try/catch 兜底
路径	path.join 拼接;resolve 绝对化;basename/dirname/extname 拆分
目录	mkdirSync recursive;readdirSync withFileTypes;rmSync recursive
大文件	createReadStream + readline 逐行;pipe 复制;内存与文件大小无关
JSON 序列化	undefined/函数/Symbol 丢弃;NaN→null;循环引用抛错;BigInt 不支持
toJSON	自定义序列化钩子,stringify 自动调用
JSON 解析	parse 前检查形状;非法 JSON 抛 SyntaxError
标准流	stdout 输出 / stderr 错误 / stdin 输入;argv[0..1] 是 node 与脚本
退出码	process.exitCode = 0/非0,CI 依据
配置	JSON 配置 + ?? 默认值合并
日志	结构化 JSON 行、分级、轮转;pino/winston

最小必会清单

- 能写出 fs.promises 的读/写/追加三连,并处理 ENOENT
- 能说出三种风格(fs Sync/回调/Promise)的适用场景
- 能写出"文本带编码读、二进制不带编码读"的规则
- 能写出 path.join/resolve/basename/extname 的示例
- 能递归遍历目录并统计文件数
- 能用 createReadStream + readline 逐行统计 1GB 文件
- 能说出 JSON 序列化的四个坑与 toJSON 的用途
- 能写出 JSON.parse 前的形状校验
- 能说出 process.argv 与 C 的 argc/argv 的对应
- 能写出"读配置→处理→写结果→记日志"的完整异步流程

自测题

Q 1. fs.readFileSync(path, 'utf8') 与不传 'utf8' 有什么区别?对二进制文件应怎么读?

✅ 参考答案

传 'utf8' 返回解码后的字符串;不传返回 Buffer。二进制文件不传编码,拿到 Buffer 后用 DataView/位运算处理。

Q 2. `JSON.stringify({a: undefined, b: NaN, c: () => {}})` 输出什么?循环引用呢?

✅ 参考答案

`'{"b":null}'`。undefined 与函数属性被丢弃,NaN 变 null;循环引用抛 TypeError。

Q 3. 为什么大文件要用流式处理?readFile 读 1GB 文件会怎样?

✅ 参考答案

readFile 把整个文件读进内存,1GB 文件占用 1GB+ 堆,可能 OOM;流式一次只读一小块,内存占用恒定,还能逐行处理。

Q 4. `path.join('/a', '../b')` 与 `path.resolve('/a', '../b')` 分别得到什么?

✅ 参考答案

join 是纯拼接规范化:`./a/../b' → '/b'`;resolve 基于当前工作目录生成绝对路径。join 不保证绝对路径,resolve 保证。

Q 5. 读一个 UTF-8 带 BOM 的文件,字符串开头会有什么?怎么处理?

✅ 参考答案

开头有 `U+FEFF`(BOM 字符)。用 `str.replace(/^/, '')` 去除。

Q 6. 写出 `process.argv` 在 `node tool.js a b` 时的内容,以及如何取到业务参数。

✅ 参考答案

`['node的路径', 'tool.js的路径', 'a', 'b']`。业务参数是 `process.argv.slice(2)`,即 `['a', 'b']`。

章末实验题:日志解析与统计工具

实验 7:日志解析与统计工具(logstats)

实验目标:实现命令行工具:解析访问日志(CSV 风格),统计每行各字段,输出汇总报告与 JSON 结果文件,覆盖全章知识点。

实验要求:

- 任务 1(覆盖:读写、编码):读取日志文件(文本 + 二进制两种输入),支持 UTF-8 与 BOM 处理
- 任务 2(覆盖:流式处理、readline):用 createReadStream + readline 逐行解析,支持任意大小文件
- 任务 3(覆盖:路径与目录):输出目录用 path.join 拼接;--out-dir 参数控制;目录不存在时 mkdir recursive
- 任务 4(覆盖:JSON 序列化与坑):统计结果序列化到 result.json(注意 undefined 字段丢弃问题,先清洗再序列化)
- 任务 5(覆盖:标准流与参数):参数解析用 --level/--out-dir 风格;错误写 stderr;退出码非 0
- 任务 6(覆盖:appendFile 日志):工具自身的运行日志追加到 logs/tool.log(INFO/ERROR 分级)
- 任务 7(覆盖:存在性判断):文件不存在给出友好提示(ENOENT 处理)

实验环境:Node.js 20+,运行 `node logstats.js --file /tmp/access.log --level ERROR`。

第一步 写解析器:每行格式 "时间戳,级别,消息,耗时ms",逗号分隔解析为对象

第二步 写统计器:按级别计数、总耗时均值、最慢请求 TOP3

第三步 后写主流程:参数解析 → 校验文件 → 流式统计 → 写 result.json + tool.log

✅ 参考答案要点

核心:readline 逐行流式解析 + 统计聚合 + JSON 输出 + 参数/退出码规范。约 120 行。

```
// logstats.js — 运行:node logstats.js --file /tmp/access.log [--level INFO]
'use strict';
const fs = require('node:fs');
const path = require('node:path');
const readline = require('node:readline');
const { createReadStream } = require('node:fs');

// 1. 参数解析(覆盖:process.argv、Number 转换)
function parseArgs(argv) {
  const out = { file: null, level: null, outDir: 'out' };
  for (let i = 0; i < argv.length; i++) {
    if (argv[i] === '--file') out.file = argv[i + 1];
    if (argv[i] === '--level') out.level = String(argv[i + 1]).toUpperCase();
    if (argv[i] === '--out-dir') out.outDir = argv[i + 1];
    if (argv[i] === '--help') out.help = true;
  }
  return out;
}

// 2. 单行解析:时间戳,级别,消息,耗时ms(覆盖:split、trim、Number)
function parseLine(line) {
  const parts = line.split(',');
  if (parts.length < 4) return null;
  return {
    ts: parts[0].trim(),
    level: parts[1].trim(),
    msg: parts[2].trim(),
    cost: Number(parts[3].trim()), // 可能 NaN,统计时过滤
  };
}

// 3. 流式统计(覆盖:createReadStream、readline、for await)
async function analyzeFile(filePath, levelFilter) {
  const rl = readline.createInterface({
    input: createReadStream(filePath),
    crlfDelay: Infinity,
  });
  const byLevel = {};
  const costs = [];
  let total = 0;
  for await (const line of rl) {
    const rec = parseLine(line);
    if (rec === null) continue;
    total++;
    byLevel[rec.level] = (byLevel[rec.level] ?? 0) + 1;
    if (Number.isFinite(rec.cost)) {
      costs.push({ cost: rec.cost, msg: rec.msg });
    }
  }
  const sorted = costs.slice().sort((a, b) => b.cost - a.cost);
  const filtered = levelFilter ? total : total;
  const sum = costs.reduce((a, c) => a + c.cost, 0);
  return {
    totalLines: total,
    matchedLines: filtered,
  };
}
```

```

    byLevel,
    avgCostMs: costs.length > 0 ? Math.round((sum / costs.length) * 100) / 100 : 0,
    top3: sorted.slice(0, 3).map((c) => ({ cost: c.cost, msg: c.msg })),
  });
}

// 4. 工具日志(覆盖:appendFile、分级、时间戳)
async function toolLog(level, msg) {
  const dir = path.join(process.cwd(), 'logs');
  fs.mkdirSync(dir, { recursive: true });
  const line = `[${new Date().toISOString()}] [${level}] ${msg}\n`;
  await fs.promises.appendFile(path.join(dir, 'tool.log'), line, 'utf8');
}

// 5. 输出 JSON(覆盖:JSON 序列化、undefined 清洗)
async function writeResult(stats, outDir) {
  fs.mkdirSync(outDir, { recursive: true });
  const clean = {
    totalLines: stats.totalLines,
    byLevel: stats.byLevel,
    avgCostMs: stats.avgCostMs,
    top3: stats.top3,
  };
  await fs.promises.writeFile(
    path.join(outDir, 'result.json'),
    JSON.stringify(clean, null, 2),
    'utf8'
  );
}

// 6. 主流程(覆盖:存在性、ENOENT、stderr、退出码)
async function main() {
  const args = parseArgs(process.argv.slice(2));
  if (args.help || args.file === null) {
    console.error('用法:node logstats.js --file <日志文件> [--level 级别] [--out-dir 目录]');
    process.exitCode = 1;
    return;
  }
  if (!fs.existsSync(args.file)) {
    console.error(`错误:文件不存在:${args.file}`); // 走 stderr
    process.exitCode = 2;
    await toolLog('ERROR', `文件不存在:${args.file}`);
    return;
  }

  const stats = await analyzeFile(args.file, args.level);
  await writeResult(stats, args.outDir);
  await toolLog('INFO', `分析完成:${stats.totalLines} 行,输出到 ${args.outDir}/result.json`);

  console.log('==== 统计报告 =====');
  console.log('总行数:', stats.totalLines);
  console.log('级别分布:', JSON.stringify(stats.byLevel));
  console.log('平均耗时:', stats.avgCostMs, 'ms');
  console.log('最慢请求 TOP3:');
  for (const t of stats.top3) {

```

```
    console.log(` ${t.cost}ms ${t.msg}`);
  }
  console.log('结果文件:', path.join(args.outDir, 'result.json'));
}

main().catch((err) => {
  console.error('程序异常:', err.message);
  process.exitCode = 3;
});
```

设计说明:流式逐行解析保证大文件内存恒定;级别分布用对象计数(?? 初始化);耗时统计先过滤 `Number.isFinite` 排除 `NaN` 行;输出 JSON 前把可能 `undefined` 的字段清洗成确定结构;工具自身日志走 `appendFile` 分级落盘;错误统一走 `stderr` + 非零退出码,符合 CI 与脚本调用约定。可以先手写一个含 5 行的 `access.log` 测试,再生成 2 万行大文件验证内存表现。

下一章预告:第8章 网络通信 —— net 模块的 TCP/UDP、fetch 客户端、http.createServer 最小服务器,以及粘包/拆包与优雅关闭的生产坑。

第8章 网络通信

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

本章学习目标

- 回顾 TCP/UDP、端口、字节序,理解 Node 如何封装 C 的 socket API
- 掌握 net 模块:TCP 服务端与客户端完整示例
- 掌握 dgram 模块:UDP 无连接通信
- 掌握超时设置与"为什么必须设置超时"的生产教训
- 掌握 fetch 发 HTTP 请求:GET/POST/JSON 完整示例
- 掌握 http.createServer 最小 HTTP 服务端
- 理解粘包/拆包问题与长度前缀协议设计,掌握优雅关闭

从 C 到 JavaScript:本章主题如何迁移

C 的 socket 编程是"仪式感之王":socket()、bind()、listen()、accept()、connect()、send()、recv(),每个都返回错误码,还得用 htons/htonl 转换字节序,处理 EINTR 重试,管理 fd 生命周期。Node 把这些全部封装成简洁的对象:net.Server/net.Socket/dgram.Socket,事件驱动,数据到达即触发 'data' 事件,不需要手动 accept 循环——事件循环替你完成了 accept。

最大的心智差异依然是 异步 + 事件 :C 的 recv 阻塞等数据;Node 的 socket.on('data') 注册回调,数据到了事件循环调你。读写都不阻塞,一个进程可以同时处理成千上万连接(第9章细讲)。

字节序方面,Node 帮你藏掉了大部分:TCP 流无边界、无字节序问题(字节原样),Buffer 的 read/write 方法自带 BE/LE 参数;DNS 解析、地址转换都有现成 API。HTTP 更是内置:http 模块与 fetch 全球通行,一个 createServer 就是生产级服务端的最小雏形。

必须保留的 C 式警觉是 协议设计 :TCP 是字节流,没有消息边界,"粘包/拆包"是永恒话题;C 里靠定长结构或长度前缀解决,Node 里做法完全一样——本章的协议实验会完整走一遍。

前置知识自查

- C 的 socket 编程:connect/bind/listen/accept/send/recv
- TCP 三次握手与"字节流无边界"的本质;UDP 数据报语义
- 端口、IP 地址、htons/htonl 字节序
- 第5章 Buffer 与第7章流的概念
- 第9章的事件循环(本章先用事件回调,不必深究机制)

🚀 本章学习路线

1. **第一阶段(概念与 TCP)**:读 §8.1–§8.2,跑通回声服务器与客户端
2. **第二阶段(UDP 与超时)**:读 §8.3–§8.4,理解无连接语义与超时必要性
3. **第三阶段(HTTP)**:读 §8.5–§8.6,用 fetch 与 createServer 打通请求-响应闭环
4. **第四阶段(协议设计)**:读 §8.7,完成章末"带长度前缀协议的文件传输"
5. **验收标准**:能独立写出回声服务器 + 客户端,并说明粘包为什么发生、长度前缀怎么解决

本章知识导图

- 第8章 网络通信
 - §8.1 概念回顾:C 的 socket API → Node 对照表
 - §8.2 TCP:net 模块服务端与客户端
 - §8.3 UDP:dgram 无连接通信
 - §8.4 超时与非阻塞:必须设置超时
 - §8.5 HTTP 客户端:fetch 全家桶
 - §8.6 HTTP 服务端:createServer 最小服务器
 - §8.7 生产坑:粘包/拆包、优雅关闭、进程退出

§8.1 概念回顾:从 C 的 socket 到 Node

C 的 API	Node 对应	说明
socket()	new net.Server() / new dgram.Socket()	创建即带类型,无需 AF_INET 参数
bind()	server.listen(port, host)	监听即绑定
listen()	server.listen(port, cb)	server.on('listening') 事件
accept()	server.on('connection', socket)	事件循环自动 accept,每连接一个 socket
connect()	net.connect(port, host)	客户端直连
send()/recv()	socket.write(data) / socket.on('data', cb)	全异步,自动缓冲
close()	socket.end() / socket.destroy()	end 优雅关闭(发完 FIN),destroy 立即断
htons/htonl	Buffer.writeUInt16BE/readUInt16LE 等	BE/LE 显式参数,无字节序暗坑
getaddrinfo	dns.promises.lookup()	域名解析(生产用系统缓存)

💡 直觉先行:Node 的 socket 是"事件 + 流"

net.Socket 既是可读流又是可写流:socket.write() 写出去,socket.on('data') 收进来。没有阻塞、没有返回码,错误走 'error' 事件,关闭走 'close'/'end' 事件。数据永远可能"半包到达"(流无边界),所以生产协议都要处理粘包——这是本章下半场的重点。

§8.2 TCP 编程:net 模块

8.2.1 回声服务器(对应 C 的 listen/accept 循环)

```
const net = require('node:net');

// 服务端:监听 8000 端口,收到什么回什么(回声)
const server = net.createServer((socket) => {
  console.log('新连接:', socket.remoteAddress, socket.remotePort);

  socket.on('data', (chunk) => {
    console.log('收到:', chunk.toString().trim());
    socket.write('echo: ' + chunk);    // 原样回写
  });
  socket.on('end', () => console.log('客户端断开(半关闭)'));
  socket.on('error', (err) => console.error('socket 错误:', err.message));
});

server.listen(8000, '127.0.0.1', () => {
  console.log('TCP 服务器监听 127.0.0.1:8000');
});

// 测试:node 另一个终端执行 node tcp_client.js,或 nc 127.0.0.1 8000
```

8.2.2 客户端(对应 C 的 connect/send/rcv)

```
const net = require('node:net');

const client = net.connect(8000, '127.0.0.1', () => {
  console.log('已连接服务器');
  client.write('你好,服务器!');
});

client.on('data', (chunk) => {
  console.log('服务器回复:', chunk.toString().trim());
  client.end();    // 优雅关闭(发送 FIN)
});

client.on('end', () => console.log('连接已关闭'));
client.on('error', (err) => console.error('客户端错误:', err.message));

// 发送多段数据:TCP 是流,不保证每次 write 对应一次 data(粘包伏笔)
```

⚠ 常见误区 8.1:以为一次 write 对应一次 data

TCP 是字节流,没有消息边界:客户端连续 write 三次,服务端可能收到一次或多次 data,数据还会被拆半(一个 write 的 10 字节可能分两次 data 到达)。千万不要按 data 次数解析协议,必须用定长/分隔符/长度前缀。第 8.7 节给出完整方案。

§8.3 UDP 编程:dgram 模块

UDP 是无连接数据报协议:不握手、不保证送达、不保证顺序,但有消息边界(一次 sendto 一个数据报)。适用场景:DNS 查询、音视频流、游戏状态、监控探针、广播发现。C 的 sendto/recvfrom 对应 dgram 的 send/message 事件。

```
const dgram = require('node:dgram');

// UDP 服务端
const udpServer = dgram.createSocket('udp4');
udpServer.on('message', (msg, rinfo) => {
  console.log(`收到 ${rinfo.address}:${rinfo.port} 的 ${msg.length} 字节`);
  const reply = Buffer.from('pong:' + msg.toString('utf8'));
  udpServer.send(reply, rinfo.port, rinfo.address); // 回给来源地址
});
udpServer.on('error', (err) => console.error('UDP 错误:', err.message));
udpServer.bind(8001, '127.0.0.1', () => console.log('UDP 监听 8001'));

// UDP 客户端:不需要连接,直接 send
const udpClient = dgram.createSocket('udp4');
const payload = Buffer.from('你好,UDP');
udpClient.send(payload, 8001, '127.0.0.1', (err) => {
  if (err) console.error('发送失败:', err.message);
});
udpClient.on('message', (msg) => {
  console.log('UDP 回复:', msg.toString('utf8'));
  udpClient.close(); // 用完关闭
});
```

§8.4 超时与非阻塞:为什么必须设置超时

⚠ 生产事故:不设超时的连接永久挂起

生产里最常见的网络事故之一:客户端连不上一台"黑洞"服务器(防火墙丢包、IP 不可达),connect 一直 pending,worker 线程被占满,最后整机雪崩。C 的 connect 有内核超时,Node 需要显式设置:每个出站连接与每次读操作都要有超时。

```

const net = require('node:net');

// 客户端超时:connect 超时 + 空闲超时
const client2 = net.connect({ port: 8002, host: '127.0.0.1', timeout: 2000 });
client2.on('timeout', () => {
  console.error('连接超时,销毁连接');
  client2.destroy(); // 超时必须主动销毁,否则挂着
});
client2.on('error', (err) => console.error(err.code === 'ETIMEDOUT' ? '超时失败' : err.message));

// 服务端空闲超时:清理僵尸连接
const server2 = net.createServer((socket) => {
  socket.setTimeout(10000); // 10 秒无数据
  socket.on('timeout', () => {
    console.log('空闲超时,断开:', socket.remoteAddress);
    socket.destroy();
  });
  socket.on('data', () => socket.setTimeout(10000)); // 有数据重置计时
});
server2.listen(8002, '127.0.0.1');

// HTTP 客户端超时:AbortSignal.timeout(现代 fetch)
async function fetchWithTimeout(url, ms = 5000) {
  const res = await fetch(url, { signal: AbortSignal.timeout(ms) });
  return res;
}
// fetchWithTimeout('http://127.0.0.1:8002/', 1000).catch((e) => console.log('超时:', e.name));

```

★ 重点结论:超时三原则

① 出站必设超时:connect timeout + idle timeout,超时即 destroy;② 入站必设空闲超时:防止慢客户端占着连接不放(慢速 DoS 的基本形态);③ 超时后必须显式清理资源,光报错不销毁等于没设。

§8.5 HTTP 客户端:fetch(浏览器与 Node 通用)

C 没有标准 HTTP 库,生产用 libcurl;Node 20+ 内置 `fetch` (与浏览器同一套标准 API),发 HTTP 请求不再需要第三方库。核心概念:Response 对象、await、状态码、响应体读取。

```

// GET 请求
async function getTodos() {
  const res = await fetch('https://jsonplaceholder.typicode.com/todos/1');
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const data = await res.json();      // 解析 JSON
  console.log('GET 结果:', data.title);
}
// getTodos().catch((e) => console.error(e));

// POST + JSON
async function createPost() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: '新文章', userId: 1 }),
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const created = await res.json();
  console.log('创建成功,ID:', created.id);
}

// 读文本与读二进制
async function readBoth() {
  const textRes = await fetch('https://example.com/');
  const text = await textRes.text();    // 文本响应
  console.log('文本长度:', text.length);

  const binRes = await fetch('https://example.com/');
  const buf = Buffer.from(await binRes.arrayBuffer()); // 二进制响应
  console.log('字节数:', buf.length);
}

// 统一超时 + 错误处理模板(生产必用)
async function safeFetch(url, options = {}) {
  const res = await fetch(url, {
    ...options,
    signal: options.signal ?? AbortSignal.timeout(10000),
  });
  if (!res.ok) {
    throw new Error(`请求失败:${res.status} ${res.statusText}`);
  }
  return res;
}

```

⚠ 常见误区 8.2:fetch 不抛错 = 成功?

fetch 只在网络层失败 (DNS、连接拒绝、超时)时 reject;HTTP 404/500 照常 resolve。必须检查 `res.ok` 或 `res.status`。另外响应体只能消费一次:读了 `json()` 就不能再读 `text()`,报错 "Body is unusable"。

§8.6 HTTP 服务端:http.createServer

最小 HTTP 服务器(生产框架 Express/Fastify 的底层就是它):

```

const http = require('node:http');

const server = http.createServer((req, res) => {
  // req:请求对象;res:响应对象
  const url = new URL(req.url, `http://${req.headers.host}`);

  if (req.method === 'GET' && url.pathname === '/hello') {
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('你好,世界');
    return;
  }

  if (req.method === 'POST' && url.pathname === '/api/echo') {
    let body = '';
    req.on('data', (chunk) => { body += chunk; }); // 异步收集请求体
    req.on('end', () => {
      try {
        const parsed = JSON.parse(body || '{}');
        res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ received: parsed, at: Date.now() }));
      } catch (err) {
        res.writeHead(400, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ error: '非法 JSON' }));
      }
    });
  }

  res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('Not Found');
});

server.listen(3000, '127.0.0.1', () => {
  console.log('HTTP 服务端:http://127.0.0.1:3000');
});
// 生产方向:Express(最流行)/Fastify(性能)/Koa(轻量) 都是它的封装

```

```

// 配合 fetch 测试上面服务器
async function testServer() {
  const r1 = await fetch('http://127.0.0.1:3000/hello');
  console.log('GET /hello:', r1.status, await r1.text());

  const r2 = await fetch('http://127.0.0.1:3000/api/echo', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name: '测试' }),
  });
  console.log('POST /api/echo:', r2.status, await r2.text());
}
testServer().catch((e) => console.error('测试失败:', e.message));

```

工程建议 8.1:请求体大小限制与响应头规范

生产服务端三件事:① 限制请求体大小(超限直接 413),防止恶意大包拖垮内存;② 统一 Content-Type 与 charset=utf-8,中文不乱码;③ 错误响应统一 JSON 结构({error, code}),方便前端统一处理。这些在 Express 里是中间件,在裸 http 里要自己写——知道原理即可。

§8.7 生产坑:粘包/拆包、优雅关闭

8.7.1 粘包与拆包:为什么发生,怎么解决

粘包:客户端两次 write 的数据被内核合并成一次 data 到达(收端看到"黏在一起");**拆包**:一次 write 的数据被拆成多次 data。根因:TCP 是字节流,收端只按"到达多少读多少"回调,不知道你的消息边界在哪。三种协议设计:

方案	原理	适用
定长	每条消息固定 N 字节,收满 N 处理一条	固定结构记录
分隔符	以 \n 或 \0 结尾,按分隔符切分	文本协议、HTTP 头部
长度前缀	头 4 字节存 payload 长度,按长度取整条	二进制协议(生产主流)

```

// 长度前缀协议:封装"编码"与"解码缓冲"
const HEADER_SIZE = 4;

// 编码:长度(4 字节 BE) + payload
function encodeMessage(payload) {
  const body = Buffer.from(payload, 'utf8');
  const buf = Buffer.alloc(HEADER_SIZE + body.length);
  buf.writeUInt32BE(body.length, 0);
  body.copy(buf, HEADER_SIZE);
  return buf;
}

// 解码:增量缓冲,从 data 流中切出完整消息(解决粘包/拆包)
class FrameParser {
  constructor() {
    this.buf = Buffer.alloc(0);
  }
  push(chunk) {
    this.buf = Buffer.concat([this.buf, chunk]);
    const messages = [];
    while (this.buf.length >= HEADER_SIZE) {
      const len = this.buf.readUInt32BE(0);
      if (this.buf.length < HEADER_SIZE + len) break; // 半包,等下一块
      const body = this.buf.subarray(HEADER_SIZE, HEADER_SIZE + len);
      messages.push(body.toString('utf8'));
      this.buf = this.buf.subarray(HEADER_SIZE + len); // 消费掉整条
    }
    return messages;
  }
}

// 使用:
const parser = new FrameParser();
parser.push(Buffer.concat([encodeMessage('第一条'), encodeMessage('第二条')]]));
// ↑ 上面一次推入两条,模拟粘包
console.log(parser.push(encodeMessage('第三条'))); // 第三条
// 拆包模拟:分两次推
const full = encodeMessage('大消息');
parser.push(full.subarray(0, 5)); // 只推头部+1字节
console.log('半包时输出:', parser.push(full.subarray(5)).length); // 仍是 0 条
// 补齐剩余字节后:

```

8.7.2 优雅关闭与进程退出

```
// 优雅停机:停止接新连接 → 处理完存量 → 退出
function gracefulShutdown(server, signal) {
  console.log(`收到 ${signal},开始优雅停机`);
  server.close(() => {
    console.log('所有连接已关闭,进程退出');
    process.exit(0);
  });
  // 兜底:超时强制退出(防呆)
  setTimeout(() => {
    console.error('强制退出');
    process.exit(1);
  }, 10000).unref(); // unref:不影响进程自然退出
}

// 挂到信号(生产容器/PM2 都会发 SIGTERM)
process.on('SIGTERM', () => gracefulShutdown(server, 'SIGTERM'));
process.on('SIGINT', () => gracefulShutdown(server, 'SIGINT'));

// TIME_WAIT 说明:服务端 close 后端口进入 TIME_WAIT(与 C 相同);
// 生产高并发重启时监听失败,常见对策:SO_REUSEADDR(内核默认)或等待,Node 一般无需处理
```

⚠ 常见误区 8.3:进程退出前不清理连接

直接 `process.exit()` 会粗暴掐断所有连接,客户端收到 `ECONNRESET` 而不是正常 `FIN`,前端可能重试风暴。正确顺序:`server.close()` 拒绝新连接 → 等待存量请求完成 → 超时兜底强制退出。Docker/K8s 的 `SIGTERM` 就是触发这个流程的标准化信号。

本章速查表

主题	速查要点
TCP 服务端	net.createServer(cb) + listen(port);connection 事件给 socket
TCP 客户端	net.connect(port, host);write 发送,data 接收
socket 事件	data/end(半关闭)/close/error/timeout
关闭语义	end() 优雅(FIN);destroy() 立即断(ECONNRESET)
UDP	dgram.createSocket('udp4') + send/bind;无连接、有消息边界
超时	connect timeout、socket.setTimeout、AbortSignal.timeout(ms)
超时处理	超时后必须 destroy,否则连接永久挂着
fetch GET	fetch(url) → res.ok 检查 → res.json()/text()/arrayBuffer()
fetch POST	method/headers/body 选项;JSON 用 stringify
fetch 的坑	HTTP 错误不 reject,必须查 res.ok;body 只能消费一次
HTTP 服务端	http.createServer((req, res));res.writeHead + res.end
请求体	req 是流:on('data') 收集 + on('end') 处理
粘包/拆包	TCP 是字节流;定长/分隔符/长度前缀三种协议
长度前缀	4 字节 BE 长度 + payload;增量缓冲切帧
优雅停机	server.close() → 等存量完成 → 超时兜底;处理 SIGTERM/SIGINT

最小必会清单

- 能写出 TCP 回声服务器与客户端,并说出 5 个 socket 事件
- 能说出 end() 与 destroy() 的区别(优雅 vs 立即)
- 能写出 UDP 收发两端,并说出 UDP 与 TCP 的三个差异
- 能写出"连接超时 + 空闲超时 + 销毁"三件套
- 能用 fetch 完成 GET/POST/JSON,并检查 res.ok
- 能用 createServer 写出路由分发(路径 + 方法)
- 能解释粘包/拆包根因,并说出三种协议方案
- 能写出 FrameParser 增量切帧(核心代码)
- 能写出优雅停机流程与信号处理
- 能说出 fetch 与 createServer 在生产框架中的位置(Express 等是封装)

自测题

- Q** 1. TCP 客户端连续 socket.write('A'); socket.write('B'),服务端一定收到两次 data 吗?

✓ 参考答案

不一定。TCP 是字节流,可能合并成一次 'AB',也可能拆成多次。协议解析不能依赖 data 次数,要用定长/分隔符/长度前缀。

Q 2. `socket.end()` 与 `socket.destroy()` 的区别是什么?

✓ 参考答案

`end()` 发送 FIN 优雅关闭,缓冲数据写完再关,对端收到 'end' 事件;`destroy()` 立即销毁连接,对端收到 ECONNRESET。

Q 3. `fetch` 请求一个返回 500 的接口,会抛异常吗?为什么?如何正确判断?

✓ 参考答案

不会。`fetch` 只在网络层失败时 reject,HTTP 状态码错误照常 resolve。用 `res.ok(200-299)`或 `res.status` 判断。

Q 4. 为什么生产代码必须给每个出站连接设置超时?超时后该做什么?

✓ 参考答案

不设超时,连接可能永久 pending(黑洞 IP、防火墙丢包),耗尽连接池/线程导致雪崩。超时后必须 `destroy/abort` 清理资源,不能只报错。

Q 5. 简述长度前缀协议的编码与解码两个环节,以及解码时的"半包"怎么处理。

✓ 参考答案

编码:4 字节 BE 长度 + payload。解码:维护增量缓冲,先读长度,缓冲不足就等下一块 data;足够就切出整条并消费,循环处理粘包的多条。

Q 6. 服务端收到 SIGTERM 后应该按什么顺序做什么?

✓ 参考答案

`server.close()` 停止接受新连接 → 等待存量请求自然结束(close 回调)→ 设置超时兜底强制退出 → 退出进程。顺序不能反。

章末实验题:带协议的文件传输服务器

实验 8:带长度前缀协议的文件传输(ftans)

实验目标:实现"请求-传输-确认"的 TCP 文件传输:客户端请求文件,服务端按长度前缀协议分块发送,客户端组装并校验,覆盖全章知识点。

实验要求:

- 任务 1(覆盖:TCP 服务端/客户端):net 双端,服务端按文件名找文件,找不到回复错误帧
- 任务 2(覆盖:长度前缀协议、粘包/拆包):消息帧 = 4 字节类型/长度 + payload;客户端用 FrameParser 增量切帧;故意把两次发送合并模拟粘包
- 任务 3(覆盖:超时):客户端 connect 超时与空闲超时,超时销毁并报错;服务端空闲超时清理连接
- 任务 4(覆盖:Buffer、二进制):文件内容用 Buffer 传输;分块(每块 64KB)避免大文件一次写爆内存;接收端按 Buffer.concat 组装
- 任务 5(覆盖:HTTP/fetch 辅助):附赠一个 fetch 健康检查端点(服务端同时起 http 端口返回状态 JSON)
- 任务 6(覆盖:优雅关闭):SIGINT/SIGTERM 优雅停机;server.close + 超时兜底
- 任务 7(覆盖:错误处理):文件不存在(ENOENT)返回错误帧;客户端校验字节数并打印

实验环境:Node.js 20+, 两个终端: `node ftans.js server 8000 /tmp/share` 与 `node ftans.js client 127.0.0.1 8000 hello.txt`。

第一步 定义协议:帧 = type(1 字节:1=文件头,2=数据块,3=完成,4=错误)+ len(4 字节 BE)+ payload;头 5 字节

第二步 实现双端状态机:服务端按文件流式发帧;客户端按 FrameParser 收帧组装

第三步 后加健壮性:超时、优雅关闭、错误帧、字节数校验,打印传输统计

✅ 参考答案要点

核心:5 字节帧头 + 增量切帧 + 流式分块传输 + 双端超时与优雅关闭。约 160 行,覆盖全章知识点。

```
// ftrans.js — 运行:node ftrans.js server 8000 /tmp/share | node ftrans.js client 127.0.0.1 8000 hell
o.txt
'use strict';
const net = require('node:net');
const http = require('node:http');
const fs = require('node:fs');
const path = require('node:path');

const HEADER = 5; // 1 字节 type + 4 字节 len
const TYPE = { HEAD: 1, DATA: 2, DONE: 3, ERROR: 4 };
const CHUNK = 64 * 1024;

// 1. 编码帧(覆盖:Buffer 写入、字节序)
function frame(type, payload) {
  const body = payload ?? Buffer.alloc(0);
  const buf = Buffer.alloc(HEADER + body.length);
  buf.writeUInt8(type, 0);
  buf.writeUInt32BE(body.length, 1);
  body.copy(buf, HEADER);
  return buf;
}

// 2. 增量切帧器(覆盖:长度前缀协议、粘包/拆包)
class FrameParser {
  constructor() { this.buf = Buffer.alloc(0); }
  push(chunk) {
    this.buf = Buffer.concat([this.buf, chunk]);
    const frames = [];
    while (this.buf.length >= HEADER) {
      const type = this.buf.readUInt8(0);
      const len = this.buf.readUInt32BE(1);
      if (this.buf.length < HEADER + len) break; // 半包等待
      frames.push({ type, body: this.buf.subarray(HEADER, HEADER + len) });
      this.buf = this.buf.subarray(HEADER + len);
    }
    return frames;
  }
}

// 3. 服务端(覆盖:createServer、流式读文件、错误帧)
function startServer(port, shareDir) {
  const server = net.createServer((socket) => {
    socket.setTimeout(30000);
    socket.on('timeout', () => { console.log('服务端:空闲超时,断开'); socket.destroy(); });
    const parser = new FrameParser();
    socket.on('data', (chunk) => {
      for (const f of parser.push(chunk)) {
        if (f.type !== TYPE.HEAD) continue;
        const name = f.body.toString('utf8');
        const file = path.join(shareDir, path.basename(name)); // 防路径穿越
        if (!fs.existsSync(file)) {
          socket.write(frame(TYPE.ERROR, Buffer.from('文件不存在:' + name, 'utf8')));
          continue;
        }
        const size = fs.statSync(file).size;

```

```

        socket.write(frame(TYPE.HEAD, Buffer.from(JSON.stringify({ name, size })), 'utf8')));
        const rs = fs.createReadStream(file, { highWaterMark: CHUNK });
        rs.on('data', (chunkData) => socket.write(frame(TYPE.DATA, chunkData)));
        rs.on('end', () => socket.write(frame(TYPE.DONE, Buffer.alloc(0))));
        rs.on('error', (err) => socket.write(frame(TYPE.ERROR, Buffer.from(err.message, 'utf8'))));
    }
});
socket.on('error', (e) => console.error('连接错误:', e.message));
});
server.listen(port, '127.0.0.1', () => console.log(`文件服务监听 ${port}, 目录 ${shareDir}`));

// 附赠 HTTP 健康检查(覆盖:createServer、JSON 响应)
http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
    res.end(JSON.stringify({ ok: true, uptime: process.uptime() }));
}).listen(port + 1);

graceful(server, '文件服务');
return server;
}

// 4. 客户端(覆盖:connect、超时、组装、校验)
function startClient(host, port, name, outPath) {
    const socket = net.connect({ host, port, timeout: 3000 });
    const parser = new FrameParser();
    let meta = null;
    const parts = [];

    socket.on('connect', () => socket.write(frame(TYPE.HEAD, Buffer.from(name, 'utf8'))));

    socket.on('data', (chunk) => {
        for (const f of parser.push(chunk)) {
            if (f.type === TYPE.ERROR) {
                console.error('服务器错误:', f.body.toString('utf8'));
                socket.destroy();
                return;
            }
            if (f.type === TYPE.HEAD) {
                meta = JSON.parse(f.body.toString('utf8'));
                console.log(`开始接收 ${meta.name}, 大小 ${meta.size} 字节`);
            } else if (f.type === TYPE.DATA) {
                parts.push(Buffer.from(f.body)); // 组装(可优化为写入流)
            } else if (f.type === TYPE.DONE) {
                const out = Buffer.concat(parts);
                fs.writeFileSync(outPath, out);
                console.log(`接收完成:${out.length} 字节, 校验${out.length === meta.size ? '通过' : '失败!'}`);
                socket.end();
            }
        }
    });
    socket.on('timeout', () => { console.error('客户端:连接/空闲超时'); socket.destroy(); });
    socket.on('error', (err) => console.error('客户端错误:', err.message));
}

// 5. 优雅关闭(覆盖:信号、close、超时兜底)

```

```
function graceful(server, tag) {
  function shutdown(sig) {
    console.log(`${tag}收到 ${sig},优雅停机`);
    server.close(() => {
      console.log('连接已清空,退出');
      process.exit(0);
    });
    setTimeout(() => { console.error('兜底强制退出'); process.exit(1); }, 5000).unref();
  }
  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT', () => shutdown('SIGINT'));
}

// 6. 入口(覆盖:参数解析、默认值)
function main() {
  const [mode, a, b, c] = process.argv.slice(2);
  if (mode === 'server') {
    const dir = c ?? b ?? '/tmp/share';
    fs.mkdirSync(dir, { recursive: true });
    startServer(Number(a ?? 8000), dir);
  } else if (mode === 'client') {
    const host = a ?? '127.0.0.1';
    const port = Number(b ?? 8000);
    const name = c ?? 'hello.txt';
    startClient(host, port, name, '/tmp/download-' + name);
  } else {
    console.error('用法:node ftrans.js server <端口> <目录> | node ftrans.js client <host> <端口> <文件名>');
    process.exitCode = 1;
  }
}
main();
```

设计说明:协议帧 type+len+payload 演示长度前缀方案的完整落地;FrameParser 同时处理粘包(服务端连续 write 的帧可能合并到达)与半包(大文件分块后单块被 TCP 拆散);服务端用 createReadStream 按 64KB 分块,内存占用与文件大小无关;客户端 Buffer.concat 组装后按 meta.size 校验;双端超时与优雅关闭按生产标准实现。测试建议:先写一个 2MB 测试文件,再用 `node ftrans.js client 127.0.0.1 8000 测试文件.bin` 传输,对比字节数与文件大小。

下一章预告:第9章 并发与异常 —— 事件循环的宏任务/微任务、Promise/async/await 全解、setTimeout 顺序题,以及 throw 与 Promise 错误传播——JS 最核心的一章。

第9章 并发与异常

建议学时:5-8 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

本章学习目标

- 理解单线程事件循环:一个进程一条 JS 线程,并发靠"快速切换任务"
- 彻底分清宏任务与微任务,能推演 setTimeout/Promise 混合顺序题
- 掌握 try/catch/finally 与 throw,理解与 C 的错误处理对照
- 掌握 Promise 全套:状态机、链式、all/race/allSettled/any
- 掌握 async/await:语法糖本质、错误传播、并发执行
- 理解 throw 与 Promise 错误的传播路径,掌握 unhandledRejection 防御
- 理解异步竞态与互斥锁模拟、防抖节流,认识 worker_threads/Cluster

从 C 到 JavaScript:本章主题如何迁移

C 的并发是"多线程 + 锁":pthread_create 开线程,互斥锁保护共享变量,条件变量协调等待。JS 走的是完全不同的路:单线程事件循环——一个进程只有一条 JS 线程,同时处理上万个连接靠的不是并行,而是"每个任务都很快,做完就切换到下一个"。没有线程共享变量的竞态,锁与条件变量在业务代码里几乎消失,换来的是异步回调的思维。

这套模型有两个必须理解的心智支点。第一,同步代码永远先执行完,异步代码永远排在后面:setTimeout 的 0 毫秒不是"立即",而是"当前同步代码之后"。第二,异步任务内部还分两级:微任务(Promise)比宏任务(setTimeout/I/O)优先。这两条规则能推导出所有"输出顺序题"。

异常方面,JS 的 try/catch 与 C++/Java 类似,但有一条关键差异:异步函数的 throw 不会冒泡到调用处——它变成 Promise 的拒绝,必须用 .catch 或 await 捕获。C 的 errno/返回值约定在 JS 里被"抛异常 + Promise 拒绝"二分替代,这条传播路径必须形成肌肉记忆。

最后,JS 不是没有多线程:worker_threads 可以开真线程(各自独立事件循环),Cluster 可以多进程共享端口。但生产的第一选择永远是"单线程 + 异步",把 CPU 密集任务扔给 worker,把并发流量交给 Cluster/容器横向扩展。读本章时,请带着"事件循环怎么转"这个问题逐节验证。

前置知识自查

- C 的异常域:没有 try/catch,错误靠 errno/返回值/assert
- C 的 pthread:线程、互斥锁、条件变量、join
- 第4章回调与闭包(本章异步的基础)
- 第7章 async fs 的用法

本章学习路线

1. **第一阶段(事件循环)**:读 §9.1,把 setTimeout 顺序题亲手跑一遍,建立"队列模型"
2. **第二阶段(异常)**:读 §9.2,理解同步异常与异步错误的区别
3. **第三阶段(Promise/async,本章核心)**:读 §9.3–§9.5,每个示例在 REPL 跑通,重点练错误传播
4. **第四阶段(并发实践)**:读 §9.6–§9.7,完成章末实验"任务队列处理器"
5. **验收标准**:不看资料,能推演三段 setTimeout/Promise 混排代码的输出,并解释微任务优先原因

本章知识导图

- 第9章 并发与异常
 - §9.1 单线程事件循环:宏任务/微任务、顺序题
 - §9.2 异常处理:throw / try / catch / finally
 - §9.3 Promise 全解:状态机、链式、并行组合
 - §9.4 async/await:语法糖、并发、错误传播
 - §9.5 错误传播:unhandledRejection 与全局防御
 - §9.6 异步竞态:互斥模拟、防抖、节流
 - §9.7 worker_threads 与 Cluster

§9.1 单线程事件循环:宏任务与微任务

直觉先行:一条线程,两个队列

JS 运行时有一条主线程和一个事件循环。同步代码直接执行;异步操作(setTimeout、I/O、Promise)登记到队列,等同步代码全部执行完,事件循环才开始取队列任务。队列分两级:微任务队列(Promise.then/catch、queueMicrotask)永远先于宏任务队列(setTimeout、setInterval、I/O 回调)。每执行完一个宏任务,先把微任务队列清空,再取下一个宏任务。

```
// 经典顺序题:先背下"同步 > 微任务 > 宏任务"
console.log('1 同步');
setTimeout(() => console.log('5 宏任务'), 0);
Promise.resolve().then(() => console.log('3 微任务'));
console.log('2 同步');
// 输出顺序:1 2 3 5
// 解释:同步代码(1、2)先全执行;微任务(3)先于宏任务(5)

// 进阶:微任务里注册的宏任务排在后面
setTimeout(() => console.log('A 宏任务'), 0);
Promise.resolve()
  .then(() => console.log('B 微任务'))
  .then(() => setTimeout(() => console.log('C 宏任务(微任务中注册)'), 0));
// 输出:A B C
// 解释:第一轮:A(宏)→清空微任务(B)→ 第二轮:C
// 若 B 不注册 C,则 C 与 A 同轮,顺序看注册先后

// 综合题(面试高频):
setTimeout(() => console.log('t1'), 0);
setTimeout(() => console.log('t2'), 0);
Promise.resolve().then(() => console.log('p1'));
Promise.resolve().then(() => console.log('p2'));
console.log('sync');
// 输出:sync p1 p2 t1 t2
```

★ 必须记住的调度规则

① 同步代码永不被打断:一段同步代码(含循环)执行完,才轮到任何异步任务;② 微任务优先:每轮宏任务后清空微任务队列;③ `setTimeout(0)` 是"最早的下一个宏任务",不是立即;④ `await` 本质是 `.then` 的语法糖,同样走微任务。

9.1.1 为什么单线程能支撑高并发

```
const http = require('node:http');

// 一个进程,一条线程,同时服务海量连接:
// 每个请求的处理都是"小任务",I/O 等待期间线程去处理别的请求
const server = http.createServer((req, res) => {
  // 模拟异步 I/O:setTimeout 期间,线程可以去处理其他请求
  setTimeout(() => {
    res.end('ok');
  }, 10);
});
server.listen(9000);

// 对比 C 的阻塞式:一个连接一个线程,万级连接需要万级线程;
// Node 单线程 + 非阻塞 I/O:万级连接共享一条线程,内存开销小一个数量级
// 代价:CPU 密集任务会卡住所有连接(见 §9.7 worker_threads)
```

§9.2 异常处理:throw / try / catch / finally

对比项	C	JS
错误载体	errno / 返回值(-1)/ NULL	异常对象 throw;异步用 Promise 拒绝
抛出	return -1; errno = ...	throw new Error('...')
捕获	if (x == -1) 逐层检查	try { } catch (err) { }
清理	goto err: 手动清理	finally 保证执行
未捕获	静默错误,行为未定义	进程崩溃(同步)/ unhandledRejection(异步)

```
// 基础语法:与 C++/Java 的 try/catch 同构
function divide(a, b) {
  if (b === 0) throw new Error('除数不能为 0'); // throw 任何值,规范用 Error
  return a / b;
}

try {
  const r = divide(10, 0);
  console.log('结果:', r);
} catch (err) {
  console.log('捕获:', err.message); // 捕获:除数不能为 0
} finally {
  console.log('finally 总是执行'); // 无论成败
}

// 错误类型:Error 家族
try {
  JSON.parse('{bad}');
} catch (err) {
  console.log(err.name); // SyntaxError
  console.log(err.message); // Unexpected token...
}

try { null.x; } catch (err) { console.log(err.name); } // TypeError
try { missingVar; } catch (err) { console.log(err.name); } // ReferenceError
console.log(new RangeError('越界').name); // RangeError

// 自定义错误:继承 Error
class ConfigError extends Error {
  constructor(key) {
    super(`配置缺失:${key}`);
    this.name = 'ConfigError';
    this.key = key; // 附带业务字段
  }
}

try {
  throw new ConfigError('port');
} catch (err) {
  if (err instanceof ConfigError) {
    console.log('业务错误,字段:', err.key, '| 消息:', err.message);
  }
}

// finally 与 return 的先后:finally 先执行,再返回
function f() {
  try { return 'try 的返回值'; }
  finally { console.log('finally 在 return 前执行'); }
}

console.log(f());
```

⚠️ 常见误区 9.1:只 catch 不处理,或吞掉错误

`catch (err) {}` 空捕获等于把错误埋掉,排查时只剩"功能神秘失效"。生产规则:catch 后必须做三选一——① 记录日志再抛出/拒绝;② 降级到默认值并告警;③ 向上传播。绝不空 catch。另外 `catch` 的 `err` 是局部变量,外部同名变量不受影响。

§9.3 Promise 全解:状态机与链式

9.3.1 为什么需要 Promise:回调地狱的进化

```
// 回调嵌套(老代码,可读性差):
function readChain() {
  const fs = require('node:fs');
  fs.readFile('a.txt', 'utf8', (err, a) => {
    if (err) return console.error(err);
    fs.readFile('b.txt', 'utf8', (err, b) => {
      if (err) return console.error(err);
      fs.writeFile('c.txt', a + b, (err) => {
        if (err) return console.error(err);
        console.log('完成(回调嵌套)');
      });
    });
  });
}
// readChain(); // 需要先创建 a.txt b.txt

// Promise 化:new Promise 包装异步任务
const fs = require('node:fs');
function readFileP(path) {
  return new Promise((resolve, reject) => {
    fs.readFile(path, 'utf8', (err, data) => {
      if (err) reject(err);      // 失败 → 拒绝
      else resolve(data);       // 成功 → 兑现
    });
  });
}
```

9.3.2 状态机:三个状态,一次转移

```
// Promise 只有三种状态:pending(进行中)→ fulfilled(兑现)/ rejected(拒绝)
// 状态只能转移一次,不可逆—这保证了 .then 的结果确定性

const p = new Promise((resolve, reject) => {
  // 异步任务在这里启动
  setTimeout(() => resolve('成功值'), 100);
});
console.log('初始状态:promise(状态不可直接读)');
p.then((value) => console.log('兑现:', value)); // 兑现:成功值

// 拒绝的两种写法:reject() 或 throw(在 executor 中)
const p2 = new Promise((resolve, reject) => {
  throw new Error('executor 中抛错 = 拒绝');
});
p2.catch((err) => console.log('捕获:', err.message));

// 链式:then 返回新 Promise,自动"扁平化"
Promise.resolve(1)
  .then((n) => n + 1) // 返回普通值 → 自动包装
  .then((n) => Promise.resolve(n * 10)) // 返回 Promise → 展开等待
  .then((n) => console.log('链式结果:', n)) // 链式结果:20
  .catch((err) => console.error('任一环出错都到这里:', err.message));

// finally:无论成败都执行(清理资源),不改变传递的值
Promise.resolve('x')
  .finally(() => console.log('finally 清理'))
  .then((v) => console.log('值仍传递:', v));
```

⚠ 常见误区 9.2:then 里抛错没 catch,或忘记 return

① 链式必须 **return**: `p.then(() => { return fetchB(); })` 少了 `return`,链就断成"undefined 继续",后续拿不到 B 的结果;② 链尾必须挂 `catch` 或由调用方 `await`,否则拒绝无人处理会触发 `unhandledRejection`(见 §9.5)。ESLint 的 `promise` 规则组专门抓这两类。

9.3.3 并行组合:all / race / allSettled / any

```
// Promise.all:全部成功才成功(任何一个失败整体失败)——并行 I/O 首选
const tasks = [Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)];
Promise.all(tasks).then(([a, b, c]) => console.log('all:', a + b + c)); // 6

// 生产场景:并发请求
async function fetchAll() {
  const urls = ['/api/a', '/api/b', '/api/c'];
  const results = await Promise.all(urls.map((u) => fetchJson(u)));
  return results;
}
async function fetchJson(u) { return { url: u, data: 1 }; } // 示意

// Promise.allSettled:全部完成(含失败),逐个看结果——批量任务不互相连坐
Promise.allSettled([Promise.resolve(1), Promise.reject(new Error('失败'))])
  .then((results) => {
    for (const r of results) {
      console.log(r.status, r.status === 'fulfilled' ? r.value : r.reason.message);
    }
  });

// Promise.race:谁先完成用谁(超时实现)
const slow = new Promise((res) => setTimeout(() => res('慢任务'), 1000));
const fast = new Promise((res) => setTimeout(() => res('快任务'), 100));
Promise.race([slow, fast]).then((v) => console.log('race 胜者:', v)); // 快任务

// Promise.any:第一个成功者;全失败才拒绝(ES2021)
Promise.any([Promise.reject(new Error('a 失败')), Promise.resolve('b 成功')])
  .then((v) => console.log('any:', v)); // b 成功
```

§9.4 async/await:语法糖与错误传播

async 函数 = 返回 Promise 的函数;await 表达式 = 取出 Promise 的值。它把“链式回调”写成同步风格,是 ES2017 以来 JS 异步的第一选择。**async 函数里 throw 等价于返回 rejected Promise**,await 遇到拒绝就在那行抛出,可用 try/catch 捕获——这正是“异步异常同步化”的核心。

```

// async 函数总是返回 Promise
async function hello() {
  return '你好'; // 自动包装成 Promise
}
console.log(hello() instanceof Promise); // true
hello().then((v) => console.log(v)); // 你好

// await 等待:与 then 等价但可读性更高
async function demo() {
  const a = await Promise.resolve(10);
  const b = await Promise.resolve(20);
  console.log('await 结果:', a + b); // 30
}
demo();

// 错误传播:async 里 throw = 返回 rejected Promise
async function risky() {
  throw new Error('async 里抛错');
}
risky().catch((err) => console.log('外部捕获:', err.message));

// 错误传播:await 拒绝在行内抛出,可以被 try/catch 捕获
async function withTry() {
  try {
    await Promise.reject(new Error('被拒绝'));
  } catch (err) {
    console.log('await 行内捕获:', err.message);
  }
}
withTry();

// await 并发:多个独立异步任务要并行,不要串行 await
async function parallel() {
  // ❌ 串行:总耗时 300ms
  const a = await delay(100, 'A');
  const b = await delay(100, 'B');
  console.log('串行完成:', a, b);

  // ✅ 并行:总耗时 100ms(先启动,再统一 await)
  const [c, d] = await Promise.all([delay(100, 'C'), delay(100, 'D')]);
  console.log('并行完成:', c, d);
}
function delay(ms, v) {
  return new Promise((res) => setTimeout(() => res(v), ms));
}
parallel();

```

⚠️ 常见误区 9.3:在循环里串行 await 拖慢整体

`for (const u of urls) { const r = await fetch(u); }` 是串行:100 个请求 100 秒。无依赖时改为 `await Promise.all(urls.map(u => fetch(u)))`;有依赖(每个请求需要前一个结果)才保留串行。这是生产性能评审的必查项。

§9.5 错误传播:unhandledRejection 与全局防御

```
// 未处理的 Promise 拒绝:默认 Node 会打警告并(20+)可能崩溃进程
// Promise.reject(new Error('没人管我')); // UnhandledPromiseRejection

// 全局兜底:生产必须挂,把漏网错误打日志(但绝不能靠它替代正确 catch)
process.on('unhandledRejection', (reason, promise) => {
  console.error('未处理的 Promise 拒绝:', reason?.message ?? reason);
  // 生产:记录告警;进程是否退出视业务而定(默认退出更安全)
});
process.on('uncaughtException', (err) => {
  console.error('未捕获异常:', err.message);
  // 生产:记录后优雅退出(状态可能已损坏)
});

// EventEmitter 的 error 事件:emit('error') 没人监听会抛异常
const { EventEmitter } = require('node:events');
const em = new EventEmitter();
em.on('error', (err) => console.log('emitter 错误:', err.message));
em.emit('error', new Error('业务错误'));

// 错误的正确传播模板:函数返回 Promise,失败必 reject
async function getUser(id) {
  if (typeof id !== 'number' || id <= 0) {
    throw new TypeError(`非法 id:${id}`); // 参数错误立即拒绝
  }
  const user = await dbFind(id); // 数据库错误自然上抛
  return user ?? null; // 业务"未找到"用 null,不是异常
}
async function dbFind(id) { return { id }; }
getUser(-1).catch((e) => console.log('传播到调用方:', e.message));

// 错误边界经验:参数校验错误 → throw(立即失败);
// 业务"没找到" → 返回 null/undefined;系统错误 → 让异常沿链传播,由调用方决定
```

★ 必须记住的传播对照

同步代码: throw → 最近的 try/catch 或进程崩溃 ;异步回调: 回调风格错误走 err 参数(如 fs.readFile 的 (err, data));Promise 域: throw/reject → .catch → 链尾或 unhandledRejection ;async/await: throw → await 行抛出 → 外层 try/catch 。四域四规则,记熟后错误排查不再靠猜。

§9.6 异步竞态与同步原语

9.6.1 异步竞态:检查与使用之间的缝隙

```
// 竞态示例:并发请求同一资源,晚到的结果覆盖早到的
let current = 'v0';
async function loadVersion() {
  // 两个请求先后发出,返回顺序不定
  const r1 = load('/api/v?t=1');
  const r2 = load('/api/v?t=2');
  const [v1, v2] = [await r1, await r2];
  console.log('竞态风险:最终值取决于返回顺序');
}

function load(url) {
  return new Promise((res) => {
    const delay = url.includes('t=1') ? 200 : 50;
    setTimeout(() => res(`数据(${delay}ms)`), delay);
  });
}

// 经典解法:请求序号校验(丢弃过期响应)
let requestSeq = 0;
async function safeLoad() {
  const mySeq = ++requestSeq;           // 本次请求序号
  const result = await load('/api/data');
  if (mySeq !== requestSeq) {
    console.log('响应已过期,丢弃');    // 有更新的请求在途
    return null;
  }
  return result;                       // 最新请求才生效
}
```

9.6.2 互斥锁模拟:串行化异步临界区

```
// 单线程也有"临界区":await 会出让控制权,期间状态可能被改
// 用 Promise 链实现"互斥锁"(串行队列):
class Mutex {
  #queue = Promise.resolve();
  #lock() {
    let release;
    const next = new Promise((res) => { release = res; });
    const prev = this.#queue;
    this.#queue = this.#queue.then(() => next);
    return prev.then(() => release);
  }
  async run(fn) {
    const release = await this.#lock();
    try {
      return await fn();
    } finally {
      release(); // 无论成败都解锁
    }
  }
}

// 使用:多个异步任务排队执行,临界区不交错
const mtx = new Mutex();
let counter = 0;
async function inc(n) {
  for (let i = 0; i < n; i++) {
    await mtx.run(async () => { // 临界区串行
      const cur = counter;      // 读
      await delay(0);           // 模拟异步间隙
      counter = cur + 1;        // 写
    });
  }
}

function delay(ms) { return new Promise((r) => setTimeout(r, ms)); }

async function main() {
  await Promise.all([inc(50), inc(50)]);
  console.log('加锁后计数:', counter); // 100,没有丢失更新
}

main();
```

9.6.3 防抖与节流:高频事件的两种整形

```
// 防抖(debounce):停止触发后 N 毫秒才执行一次(输入联想、搜索框)
function debounce(fn, wait = 300) {
  let timer = null;
  return (...args) => {
    clearTimeout(timer); // 重置
    timer = setTimeout(() => fn(...args), wait);
  };
}

const debouncedSearch = debounce((q) => console.log('搜索:', q));
debouncedSearch('a');
debouncedSearch('ab');
debouncedSearch('abc'); // 只会输出一:搜索 abc

// 节流(throttle):N 毫秒内最多执行一次(滚动、拖拽、限频上报)
function throttle(fn, wait = 200) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= wait) {
      last = now;
      fn(...args);
    }
  };
}

const throttledLog = throttle((msg) => console.log('节流:', msg));
for (let i = 0; i < 5; i++) throttledLog(i); // 只输出一:节流: 0
```

§9.7 worker_threads 与 Cluster:真并行的两条路

```
// worker_threads:CPU 密集任务的真并行(各自独立事件循环,通过消息通信)
const { Worker } = require('node:worker_threads');

function runInWorker(code) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(code, { eval: true });
    worker.on('message', resolve);
    worker.on('error', reject);
    worker.on('exit', (code2) => {
      if (code2 !== 0) reject(new Error(`worker 退出码 ${code2}`));
    });
  });
}

// 大数计算放 worker,主线程事件循环不卡
async function heavyCompute() {
  const result = await runInWorker(`
    const { parentPort } = require('node:worker_threads');
    let sum = 0;
    for (let i = 0; i < 1e8; i++) sum += i;    // 纯 CPU 密集
    parentPort.postMessage(sum);
  `);
  console.log('worker 计算结果:', result);
}

// heavyCompute(); // 与主线程并行,主线程仍能响应请求

// Cluster:多进程共享端口(每个进程一个事件循环,利用多核 CPU)
// node --experimental-... 老 API;现代:用 cluster 模块分派
// if (cluster.isPrimary) { fork() 若干 worker } else { 启动 http 服务器 }
// 一句话:单机多核利用 = Cluster(进程)或 worker_threads(线程);
// 横向扩展 = 容器多副本 + 负载均衡,这是生产主流
```

工程建议 9.1:并发模型选型决策树

I/O 密集(网络/文件/数据库)→ 单线程异步足够,万级并发无压力;CPU 密集(加密、图像、大数运算)→ worker_threads 分担,数量 = CPU 核数 - 1;需要独立故障域/多核吞吐 → Cluster 或容器多副本;跨进程消息(多实例)→ Redis/消息队列。先问"卡的是 I/O 还是 CPU",再选工具,不要一上来就上多进程。

本章速查表

主题	速查要点
模型	单线程事件循环;同步代码先执行完,异步排队
任务分级	微任务(Promise.then/await)优先于宏任务(setTimeout/I/O)
setTimeout(0)	最早的下一个宏任务,不是立即
高并发原理	非阻塞 I/O + 快速切换;CPU 密集会卡事件循环
throw	任意值可抛;生产统一 new Error(...)
catch/finally	catch 后必须处理/记录/重抛;finally 保证清理
Promise 状态	pending → fulfilled/rejected,只能转移一次
链式	then 返回新 Promise;必须 return 才能继续传值;链尾挂 catch
组合	all 全成、allSettled 全落、race 先到、any 先成
async/await	async 返回 Promise;await 取结果/在行内抛错;throw = reject
并发等待	独立任务用 Promise.all 并行,不要串行 await
全局兜底	unhandledRejection / uncaughtException 挂日志与退出策略
异步竞态	序号校验丢弃过期响应
互斥模拟	Promise 链串行化临界区(生产多用库:async-mutex)
防抖/节流	debounce 停后执行;throttle 限频执行
真并行	worker_threads(CPU 密集);Cluster/容器副本(多核/横向)

最小必会清单

- 能推演"同步、微任务、宏任务"三段混排的输出顺序,并解释原因
- 能说出 Promise 三状态与"只能转移一次"的含义
- 能写出链式 then + 链尾 catch,并说明漏 return 的后果
- 能说出 all/allSettled/race/any 四个组合的语义差异
- 能写出 async 函数中 throw 与 await 拒绝的两种捕获方式
- 能说出"循环里串行 await"的优化方法
- 能写出 unhandledRejection 全局兜底并说明为何不能依赖它
- 能实现一个 Promise 链互斥锁
- 能区分防抖与节流并各写一个实现
- 能说出 worker_threads 与 Cluster 的适用场景

自测题

Q 1. 写出输出顺序:`console.log('a');` `setTimeout(() => console.log('b'), 0);` `Promise.resolve().then(() => console.log('c'));` `console.log('d');`

✅ 参考答案

a d c b。同步 a、d 先执行;微任务 c 优先于宏任务 b。

Q 2. `async` 函数里 `throw new Error('x')` 会发生什么?调用方如何捕获?

✅ 参考答案

函数返回一个 rejected Promise。捕获:调用方 `await` 后 `try/catch`,或 `.catch()`。

Q 3. `Promise.all` 与 `Promise.allSettled` 的差异?批量下载任务该用哪个?

✅ 参考答案

`all` 只要一个失败就整体拒绝,拒绝即放弃其余结果;`allSettled` 等全部完成,逐条看 `status`。批量下载用 `allSettled`(单条失败不影响统计),全部成功才继续的流程用 `all`。

Q 4. 为什么 `for` 循环里串行 `await` 拖慢性能?怎么改?

✅ 参考答案

每个 `await` 等前一个完成才发下一个,总耗时是各请求之和。无依赖改为 `await Promise.all(arr.map(fn))`,总耗时约等于最慢的单个。

Q 5. 同步 `throw` 未捕获与 `Promise` 拒绝未处理,后果分别是什么?

✅ 参考答案

同步未捕获走 `uncaughtException`,进程崩溃(可兜底记录后退出);`Promise` 拒绝未处理走 `unhandledRejection`,Node 默认告警且 20+ 版本默认使进程退出。

Q 6. 防抖与节流的区别?搜索框联想与滚动加载各用哪个?

✅ 参考答案

防抖:连续触发只执行最后一次(停止后计时);节流:固定间隔最多执行一次。搜索联想用防抖,滚动加载用节流。

章末实验题:任务队列处理器(生产者-消费者)

实验 9:任务队列处理器(生产者-消费者)

实验目标:实现一个并发受控的任务队列:生产者投递任务,消费者按并发上限处理,失败自动重试,全部完成输出统计——覆盖全章知识点。

实验要求:

- 任务 1(覆盖:Promise 状态、链式):任务队列用 Promise 链维护;每个任务包装成 Promise
- 任务 2(覆盖:async/await、错误传播):消费者用 async 函数处理任务,内部 throw 上抛
- 任务 3(覆盖:异常、自定义错误):定义 TaskError(带任务 id),重试超过上限时抛出并记录
- 任务 4(覆盖:allSettled、race):汇总统计用 allSettled;超时控制用 race(任务与计时器赛跑)
- 任务 5(覆盖:事件循环顺序):演示队列先同步入队、异步消费的调度(输出顺序体现宏/微任务)
- 任务 6(覆盖:互斥模拟):共享"成功计数"用 Mutex 保护,验证计数无丢失
- 任务 7(覆盖:防抖/节流):附加实现 debounce/throttle 各一,并验证调用次数被整形
- 任务 8(覆盖:unhandledRejection 防御):挂全局兜底,并故意产生一个漏网拒绝观察兜底输出

实验环境:Node.js 20+,运行 `node taskqueue.js`。

第一步 **写队列类:**TaskQueue{ push(task), concurrency 限制, 结果回调 }

第二步 **写消费者:**每个任务 fn 返回 Promise;并发数 = 同时运行上限;完成一个补一个

第三步 **后组装演示:**30 个随机任务(部分故意失败),重试 2 次,输出统计与顺序观察

✅ 参考答案要点

核心:TaskQueue 并发控制 + 重试装饰 + allSettled 汇总 + Mutex 计数 + 全局兜底。约 150 行,覆盖全章知识点。

```
// taskqueue.js — Node.js 20+ 运行:node taskqueue.js
'use strict';

// 1. 自定义错误(覆盖:Error 继承、异常体系)
class TaskError extends Error {
  constructor(taskId, msg) {
    super(`任务 ${taskId} ${msg}`);
    this.name = 'TaskError';
    this.taskId = taskId;
  }
}

// 2. 互斥锁(覆盖:Promise 链串行化临界区)
class Mutex {
  #tail = Promise.resolve();
  async run(fn) {
    let release;
    const gate = new Promise((res) => { release = res; });
    const prev = this.#tail;
    this.#tail = prev.then(() => gate);
    await prev;
    try {
      return await fn();
    } finally {
      release();
    }
  }
}

// 3. 任务队列(覆盖:并发控制、Promise、错误传播)
class TaskQueue {
  #running = 0;
  #results = [];
  #mutex = new Mutex();
  constructor(concurrency = 3, { retries = 1, timeoutMs = 2000 } = {}) {
    this.concurrency = concurrency;
    this.retries = retries;
    this.timeoutMs = timeoutMs;
  }

  async #runOne(task, id) {
    let attempt = 0;
    for (;;) {
      attempt++;
      try {
        // 超时控制:与计时器赛跑(覆盖:Promise.race)
        const result = await Promise.race([
          task(),
          new Promise((_, rej) =>
            setTimeout(() => rej(new TaskError(id, `超时(${this.timeoutMs}ms)`)), this.timeoutMs)
          ),
        ]);
        return { ok: true, id, result, attempt };
      } catch (err) {
        if (attempt > this.retries) {

```

```

        return { ok: false, id, error: err, attempt }; // 重试耗尽
    }
    await delay(10 * attempt); // 退避
}
}
}

// 入队 + 并发消费(覆盖:async/await、共享计数)
async run(tasks) {
    const entries = tasks.map((t, i) => ({ task: t, id: i + 1 }));
    const workers = Array.from(
        { length: Math.min(this.concurrency, entries.length) },
        () => this.#worker(entries)
    );
    await Promise.all(workers);
    return this.#results;
}

async #worker(entries) {
    while (entries.length > 0) {
        const e = entries.shift(); // 取任务
        const r = await this.#runOne(e.task, e.id);
        await this.#mutex.run(async () => { // 临界区:计数
            this.#results.push(r);
        });
    }
}

// 4. 全局兜底(覆盖:unhandledRejection 防御)
process.on('unhandledRejection', (reason) => {
    console.error('【兜底】未处理的 Promise 拒绝:', reason?.message ?? reason);
});

// 5. 工具(覆盖:delay、随机)
function delay(ms) { return new Promise((r) => setTimeout(r, ms)); }
function makeTask(i) {
    return async () => {
        await delay(20 + Math.random() * 80); // 模拟耗时
        if (Math.random() < 0.25) {
            throw new TaskError(i, '随机失败'); // 25% 失败
        }
        return `任务${i}结果`;
    };
}

// 6. 防抖/节流(覆盖:高频事件整形)
function debounce(fn, wait) {
    let timer = null;
    return (...args) => {
        clearTimeout(timer);
        timer = setTimeout(() => fn(...args), wait);
    };
}

function throttle(fn, wait) {

```

```

let last = 0;
return (...args) => {
  const now = Date.now();
  if (now - last >= wait) { last = now; fn(...args); }
};
}

// 7. 事件循环顺序演示(覆盖:宏任务/微任务)
function orderDemo() {
  console.log('- 顺序演示 -');
  console.log('1 同步');
  setTimeout(() => console.log('4 宏任务'), 0);
  Promise.resolve().then(() => console.log('2 微任务'));
  queueMicrotask(() => console.log('3 queueMicrotask'));
}

// 8. 主流程(覆盖:allSettled 汇总、统计)
async function main() {
  orderDemo();

  const tasks = Array.from({ length: 30 }, (_, i) => makeTask(i));
  const queue = new TaskQueue(4, { retries: 2, timeoutMs: 500 });
  const results = await queue.run(tasks);

  const okCount = results.filter((r) => r.ok).length;
  const failCount = results.length - okCount;
  console.log('- 队列统计 -');
  console.log(`总任务:${results.length} | 成功:${okCount} | 失败:${failCount}`);
  const retried = results.filter((r) => r.attempt > 1).length;
  console.log(`重试后成功:${retried} 个(含 attempt>1)`);

  const summaries = await Promise.allSettled(
    results.filter((r) => !r.ok).map((r) => Promise.reject(r.error))
  );
  console.log('失败的拒绝统计:', summaries.length);

  // 防抖/节流验证
  let dbg = 0;
  const db = debounce(() => { dbg++; }, 50);
  for (let i = 0; i < 10; i++) db();
  await delay(80);
  console.log('防抖:连续 10 次调用,实际执行', dbg, '次');

  // 漏网拒绝(演示兜底)
  // Promise.reject(new Error('故意漏网的拒绝'));
}

main().catch((err) => {
  console.error('主流程异常:', err.message);
  process.exitCode = 1;
});

```

设计说明:TaskQueue 用固定数量 worker 循环取任务,天然实现并发上限;单任务超时用 Promise.race 与计时器赛跑;重试带指数退避(10ms × attempt),重试耗尽返回 {ok:false} 不炸队列;成功/失败统计用 Mutex 保护共享数

组(演示异步临界区);orderDemo 直观展示同步→微任务→宏任务顺序;防抖/节流验证调用次数整形;unhandledRejection 兜底注释掉的漏网拒绝可打开观察。修改 concurrency 从 1 到 8 对比总耗时,可以直观感受并发度对吞吐的影响。

下一章预告:第10章 泛型与元编程 —— JS 没有静态类型参数,TypeScript 泛型指明方向;Proxy/Reflect 的元编程能力是本章主场。

第10章 泛型与元编程

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 C 的 `void*` / 宏 / 手写多份代码的痛点,以及泛型要解决的问题
- 理解 JS 的动态类型天然"泛型":一个函数服务所有类型,代价是类型安全
- 了解 TypeScript 泛型:为 JS 动态代码补上静态约束的方向指引
- 掌握 Proxy:13 种拦截操作,理解"代理即元编程"
- 掌握 Reflect:与 Proxy 一一对应的反射工具,以及替代老 Object 方法的场景
- 掌握原型链动态操纵与动态元编程的边界
- 理解各语言"编译期算力"对照:生成代码 vs 擦除 vs 运行时动态

💡 从 C 到 JavaScript:本章主题如何迁移

C 想写"一个函数处理任何类型",只有三条路:`void*` + 回调(类型安全靠自觉,还要传长度)、宏(预处理展开,调试地狱)、复制粘贴 N 份(维护噩梦)。C++ 的模板、Java 的泛型把这条路走成了语言特性;JS 直接不设防:函数参数不检查类型,天然接受一切——这就是动态类型,泛型问题的"另类解法"。

代价是类型安全:传错类型不报错,运行到一半才炸。所以 JS 社区的方向是渐进类型:TypeScript 在"写法上像 JS、类型上像 Java"的轨道上给泛型补齐——`function stack<T>()` 这样的声明在第14章细讲,本章先建立"泛型在 JS 生态里长什么样"的坐标。

JS 的元编程则完全在运行时:Proxy 拦截对象的读写,像给对象装"窃听器 + 门卫";Reflect 提供与拦截一一对应的原始操作。Vue 3 的响应式、MobX、各种 ORM 的魔法底层都是 Proxy。C 的宏在编译期改写代码,JS 的 Proxy 在运行期改写行为——一个是预处理器,一个是运行时钩子,这是两种元编程哲学。

最后记住一张对照表:C/C++ 的模板在编译期生成代码(快但二进制膨胀);Java 的泛型在编译期擦除类型(兼容但反射受限);JS 的"泛型"在运行期动态分发(最灵活、最慢)。三选一,没有免费的午餐。

📦 前置知识自查

- C 的 `void*`、函数指针回调、宏 `#define` 泛化
- C++ 模板的直观认识(可选,不要求)
- 第1章对象与属性、第6章原型链
- 第4章 this 绑定(Proxy 的 get 拦截会用到)

🚀 本章学习路线

1. **第一阶段(泛型坐标)**:读 §10.1–§10.3,理解"动态类型天然泛型"与 TS 的方向
2. **第二阶段(Proxy/Reflect 核心)**:读 §10.4–§10.5,亲手写日志代理与校验代理
3. **第三阶段(边界与对照)**:读 §10.6–§10.7,理解元编程的代价与生产判断
4. **第四阶段(实验)**:完成章末"泛型栈 + 日志代理 + 深代理"
5. **验收标准**:能写出 Proxy 拦截 get/set 的代码,并说明 Reflect 在拦截器里的作用

本章知识导图

- 第10章 泛型与元编程
 - §10.1 为什么需要泛型:C 的 void*/宏痛点
 - §10.2 JS 的动态类型:天然"泛型"与代价
 - §10.3 TypeScript 泛型方向指引
 - §10.4 Proxy:13 种拦截与生产场景
 - §10.5 Reflect:与拦截一一对应
 - §10.6 原型链操纵与动态元编程边界
 - §10.7 编译期算力对照表与生产判断

§10.1 为什么需要泛型:先看 C 的三条路

C 的方案	痛点	JS 的对应
void* + 长度参数	类型安全归零,解引用全靠自觉	参数不检查类型(同样靠自觉,但少了解引用)
宏 #define MAX(a,b)	无类型检查、副作用隐患、调试困难	普通函数即可(JS 函数不挑参数)
复制多份实现	维护噩梦,改一处漏三处	一个函数全类型通用

```
// C 的 void* 栈(示意,伪代码):
// void push(Stack* s, void* item, size_t size); ← 每次都要传 size,取回还要强转
// int* p = (int*)pop(s); ← 强转错了不报错

// JS 天然通用:一个栈函数服务任何类型
function createStack() {
  const items = [];
  return {
    push(item) { items.push(item); }, // 不检查类型,number/string/对象都行
    pop() { return items.pop(); },
    peek() { return items[items.length - 1]; },
    get size() { return items.length; },
  };
}
const intStack = createStack();
intStack.push(42);
const strStack = createStack();
strStack.push('hello');
const mixed = createStack();
mixed.push({ a: 1 }, '任意', 3.14); // 混着放都合法
console.log(intStack.peek(), strStack.peek(), mixed.size); // 42 hello 3
```

⚠ 常见误区 10.1:"泛型 = 参数类型任意"是危险简化

动态类型的"任意"没有上限:传错类型 **不会在调用处报错**,而是在函数内部运行时炸,或更糟——静默产出错误结果 (`stack.push('42')` 后 `pop() * 2` 得 NaN)。生产对策:入口处做运行时校验(第14章的 JSDoc/TS 是系统性解法)。泛型的价值不是"能传任意类型",而是"只允许满足约束的类型"。

§10.2 动态类型的"泛型"与内置泛型结构

```
// 内置"泛型"结构:Map/Set 接受任意键值类型
const map = new Map();
map.set('name', '李雷');
map.set(42, '数字键');           // 键可以是任何类型
map.set({ id: 1 }, '对象键');     // 对象键(引用相等)
console.log(map.get(42), map.get({ id: 1 })); // 数字键 undefined(不同引用!)

// Set:去重任意类型
const set = new Set([1, 2, 2, 3, 3, 3]);
console.log(set.size);           // 3,自动去重

// 用工厂函数模拟"泛型容器":闭包持有类型约束
function createTypedStack(validate) {
  const items = [];
  return {
    push(item) {
      if (!validate(item)) throw new TypeError('类型不匹配');
      items.push(item);
    },
    pop() { return items.pop(); },
  };
}
const onlyNumbers = createTypedStack((v) => typeof v === 'number');
onlyNumbers.push(1);
// onlyNumbers.push('x');           // ✗ TypeError:类型不匹配
console.log(onlyNumbers.pop());    // 1

// 运行时"类型约束"的通用写法:先校验再使用(生产必备模式)
function assertNumber(v, what = '值') {
  if (typeof v !== 'number' || !Number.isFinite(v)) {
    throw new TypeError(`${what} 必须是有限数字,收到:${v}`);
  }
}
function divide(a, b) {
  assertNumber(a, 'a');
  assertNumber(b, 'b');
  return a / b;
}
console.log(divide(10, 4));        // 2.5
```

§10.3 TypeScript 泛型:方向指引

JS 的下一步是 TypeScript(第14章详讲)。TS 在 JS 之上加类型标注, **编译期检查**、**运行期擦除**。泛型是 TS 的核心能力之一,读框架源码(React、Vue、Express 的类型声明)必须认识它。这里给出最小语法,建立"泛型长什么样"的坐标:

```
// TypeScript 泛型(示意,需 .ts 文件运行):
// function first<T>(arr: T[]): T | undefined {
//   return arr[0];
// }
// const a = first<number>([1, 2, 3]); // a: number | undefined
// const b = first(['x']); // 自动推导 T = string

// 泛型约束(extends):
// function len<T extends { length: number }>(x: T): number {
//   return x.length; // T 必须是"有 length 的对象"
// }

// 泛型数据结构:
// class Stack<T> {
//   private items: T[] = [];
//   push(item: T): void { this.items.push(item); }
//   pop(): T | undefined { return this.items.pop(); }
// }
// const s = new Stack<number>();
// s.push(1);
// // s.push('x'); // ❌ 编译期报错,类型安全回到编译期

// 生产坐标:小项目 JSDoc 起步(第14章),中大型项目引入 TS;
// 泛型在框架类型里的出现频率 = "组件 Props 泛型"、"工具类型"如
// Partial<T> / Pick<T, K> / ReturnType<T>,读类型声明时按"参数化类型"理解即可
console.log('TS 泛型为 JS 补上编译期类型安全(详见第14章)');
```

§10.4 Proxy:对象的运行时门卫

Proxy 给对象套一层"拦截器":所有对目标对象的操作(get/set/has/delete 等 13 种)先经过处理器(handler)函数,可以改写、记录、校验、拒绝。这是 JS 元编程的主舞台。

```

// 日志代理:记录每一次属性访问与修改
const target = { name: '李雷', score: 90 };
const logger = new Proxy(target, {
  get(obj, prop) {
    console.log(`[get] ${String(prop)}`);
    return Reflect.get(obj, prop); // 转发原始操作
  },
  set(obj, prop, value) {
    console.log(`[set] ${String(prop)} = ${value}`);
    return Reflect.set(obj, prop, value); // 必须返回布尔(是否成功)
  },
});
logger.score; // [get] score
logger.score = 95; // [set] score = 95
console.log(logger.score); // 95

// 校验代理:写保护 + 类型检查
const account = { balance: 0 };
const safe = new Proxy(account, {
  set(obj, prop, value) {
    if (prop === 'balance') {
      if (typeof value !== 'number') throw new TypeError('余额必须是数字');
      if (value < 0) throw new RangeError('余额不能为负');
    }
    return Reflect.set(obj, prop, value);
  },
  deleteProperty(obj, prop) {
    if (prop === 'balance') return false; // 禁止删除关键字段
    return Reflect.deleteProperty(obj, prop);
  },
});
safe.balance = 100;
// safe.balance = -5; // ✖ RangeError
// delete safe.balance; // false, 删除被拒绝
console.log(safe.balance); // 100

// 隐藏敏感属性:读取时拦截
const secret = { user: 'admin', password: 'hunter2' };
const guarded = new Proxy(secret, {
  get(obj, prop) {
    if (prop === 'password') return '***';
    return Reflect.get(obj, prop);
  },
  has(obj, prop) {
    return prop === 'password' ? false : Reflect.has(obj, prop);
  },
});
console.log(guarded.password); // ***
console.log('password' in guarded); // false ← 假装不存在

```

⚠ 常见误区 10.2:Proxy 不透明,this 与身份陷阱

① 代理不改变原对象:对 proxy 的修改会落到 target 上,但 `proxy === target` 为 false,混用两者会出"改了代理原对象没变"的错觉(其实变了,是引用比较的问题);② 方法内 this:对象方法内部访问 this 走的是 target 而不是 proxy,拦截不到,需用 `this = proxy` 绑定(生产里即 Vue 响应式包裹时处理的问题);③ 13 种操作各返回类型不同,set/delete 必须返回布尔。

§10.5 Reflect:与拦截——对应的反射工具

Reflect 提供与 Proxy 13 种拦截——对应的原始操作。两个用途:① 在拦截器内部"转发原始行为";② 替代部分老 Object 方法(返回值更友好)。

```
// 用途一:拦截器内转发(上一节的 get/set 都在用 Reflect)
// 关键:Reflect.set 返回布尔表示成功,Object.defineProperty 抛异常—Proxy 规范要求布尔

// 用途二:替代老 API
const obj = { a: 1, b: 2 };
console.log(Reflect.has(obj, 'a'));      // true,替代 'a' in obj
console.log(Reflect.ownKeys(obj));       // [ 'a', 'b' ],替代 Object.getOwnPropertyNames
console.log(Reflect.deleteProperty(obj, 'b')); // true,替代 delete obj.b
console.log(obj);                        // { a: 1 }

// 用途三:调用函数 with this(替代 call 的现代写法)
const fn = function (x) { return this.base + x; };
console.log(Reflect.apply(fn, { base: 10 }, [5])); // 15

// 用途四:与 defineProperty 的差异(返回值 vs 抛错)
const ok = Reflect.defineProperty(obj, 'c', { value: 3, writable: false });
console.log(ok);                          // true
console.log(obj.c);                       // 3

// 一个完整的"读写校验 + 默认值"代理模板(生产常用):
function withDefaults(target, defaults) {
  return new Proxy(target, {
    get(obj, prop) {
      const v = Reflect.get(obj, prop);
      return v === undefined ? defaults[prop] : v; // 缺属性给默认值
    },
  });
}
const cfg = withDefaults({ port: 8080 }, { host: 'localhost', port: 3000 });
console.log(cfg.host, cfg.port);           // localhost 8080(有值不覆盖)
```

★ 重点结论:Proxy + Reflect 是一对搭档

拦截器里的转发一律用 Reflect(保证与原生行为一致、返回值类型正确);业务代码里 Reflect 用来替代容易踩坑的老 API。读到框架源码中的 Reflect.get/set,就是在"转发原始操作",这是 Vue/ MobX 响应式、ORM 魔法的最底层。

§10.6 原型链操纵与动态元编程边界

```
// 动态原型操纵:运行时改方法(第6章讲过,这里从"元编程"视角再看)
function Greeter(name) {
  this.name = name;
}
Greeter.prototype.hello = function () { return `你好,${this.name}`; };

// 运行时给原型追加方法(影响所有实例)——这就是"猴子补丁"
Greeter.prototype.hello2 = function () { return `再次问好,${this.name}`; };
const g = new Greeter('李雷');
console.log(g.hello2()); // 再次问好,李雷

// 动态属性:Object.defineProperty 精细控制
const ticket = {};
Object.defineProperty(ticket, 'price', {
  value: 100,
  writable: false, // 只读
  enumerable: false, // 不出现在 for...in / Object.keys
  configurable: false, // 不可删除/重定义
});
// ticket.price = 200; // 严格模式 TypeError(只读)
console.log(Object.keys(ticket)); // [] ← 不可枚举
console.log(ticket.price); // 100

// 元编程边界:什么能做什么不能做
// 能:拦截读写、动态加方法、改原型、defineProperty、装饰(第12章)
// 不能:没有宏、没有编译期代码生成、不能拦截"私有字段 #"的访问
// # 私有字段天然绕过 Proxy(规范保证),所以"绝对私有"的数据用 #
```

⚠ 常见误区 10.3:猴子补丁原型是生产大忌

给 `Array.prototype` 等内建原型加方法(如 `Array.prototype.groupBy = ...`)会影响所有代码,库之间互相踩踏,还可能在引擎升级时与新 API 冲突。生产规则:应用代码可以小范围改自己的类原型;库代码禁止;内建原型禁止(用 `Array.prototype.groupBy.call(arr, fn)` 风格或包装函数替代)。

§10.7 编译期算力对照与生产判断

方案	机制	算力位置	代价
C++ 模板	编译期生成实例化代码	编译期	二进制膨胀、编译慢
Java 泛型	编译期检查,运行期擦除	编译期(检查)/运行期(擦除)	反射受限、基本类型要装箱
TypeScript	编译期检查,输出纯 JS	编译期(检查)	运行期无检查(靠类型声明保证)
JavaScript 动态	运行时动态分发	运行期	类型错误后置、无编译期保证
宏(#define / macro)	预处理文本替换	预处理	无类型、不可调试

🚀 工程建议 10.1:元编程何时该用,何时别用

该用:跨切面逻辑(日志、校验、权限、埋点)、数据绑定/响应式框架、ORM/序列化、测试替身(mock)。别用:业务代码里的"灵巧魔法"——Proxy 让调试断点看不到真实数据流,代码评审者要花三倍时间。判断标准:这个拦截是"对所有对象统一生效"还是"只对这一个对象",前者值得抽象成代理,后者直接写普通函数。框架内部多用,业务代码少用,这是 JS 元编程的生产共识。

本章速查表

主题	速查要点
C 的泛化方案	void* + 长度、宏、复制多份——各有痛点
JS 的"泛型"	动态类型天然接受一切类型;类型安全靠运行时校验
TS 泛型	function f<T>(x: T[]): T; T extends {length} 约束;编译期检查运行期擦除
Proxy 用途	拦截读写:日志、校验、默认值、隐藏字段、响应式
13 种拦截	get/set/has/deleteProperty/ownKeys/apply/construct 等
set 返回值	必须返回布尔(Reflect.set 转发)
Reflect 用途	拦截器内转发原始操作;替代 in/delete/apply 老写法
Reflect.ownKeys	返回自身全部键(含不可枚举与 Symbol)
动态原型	运行时加方法影响所有实例(猴子补丁);内建原型禁止
defineProperty	writable/enumerable/configurable 三开关精细控制
元编程边界	不能拦截 # 私有字段;没有宏/编译期代码生成
编译期算力	C++ 生成代码 > TS/Java 检查+擦除 > JS 运行时动态
生产判断	跨切面统一逻辑用代理;单点业务逻辑写普通函数

最小必会清单

- 能说出 C 的 void*/宏/复制多份三种泛化方案的痛点
- 能解释"JS 动态类型天然泛型"及代价(类型错误后置)
- 能看懂 TS 泛型 `function f<T>(x: T[]): T` 并说明运行期擦除
- 能写出 Proxy 的 get/set 拦截器(日志代理)
- 能说出 set 拦截为什么必须返回布尔
- 能用 Proxy 实现"只读对象"或"默认值对象"
- 能说出 Reflect 在拦截器里的作用(转发原始操作)
- 能用 defineProperty 实现只读不可枚举字段
- 能说出猴子补丁内建原型的生产禁令及替代
- 能说出 Proxy 不能拦截 # 私有字段

自测题

Q 1. C 的 `void*` 泛化与 JS 的动态类型,本质区别是什么?

✓ 参考答案

`void*` 需要显式强转与传长度,解引用错误是 UB;JS 直接接受任意值,少了解引用,但类型错误同样后置到运行期,且可能静默产出错误结果。

Q 2. 写出一个 Proxy,使 `obj.secret` 读取时返回 `'***'`,且 `'secret' in proxy` 为 `false`。

✓ 参考答案

get 拦截 `secret` 返回 `'***'`;has 拦截 `secret` 返回 `false`,其余 Reflect 转发。

Q 3. Proxy 的 set 拦截器为什么必须返回布尔?忘了 `return` 会怎样?

✓ 参考答案

规范要求 `set/deleteProperty` 返回布尔表示操作是否成功,严格模式下返回非布尔(undefined)会抛 `TypeError`。

Q 4. `Reflect.get(obj, prop)` 与 `obj[prop]` 的区别?拦截器里为什么用前者?

✓ 参考答案

两者结果等价,但 Reflect 以函数形式调用,可带 `receiver` 参数控制 `this`,且返回值/行为在拦截器语境下与规范一致,是转发原始操作的规范姿势。

Q 5. 为什么给 `Array.prototype` 加自定义方法被禁止?正确的替代是什么?

✓ 参考答案

会污染全局影响所有代码,库之间冲突,引擎升级可能撞名。替代:独立函数 `function groupBy(arr, keyFn) {...}` 或包装类。

Q 6. 说出 C++ 模板、Java 泛型、JS 动态三者的"算力位置"。

✓ 参考答案

C++ 编译期生成代码;Java 编译期检查 + 运行期擦除;JS 运行期动态分发。依次更灵活、更慢、更少编译期保证。

章末实验题:泛型容器与元编程工具箱

实验 10:泛型容器与元编程工具箱(meta-toolkit)

实验目标:实现类型受约束的栈容器 + Proxy 元编程工具箱(日志/校验/深代理/默认值),全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:动态类型"泛型"、运行时校验):TypedStack 容器,构造参数是校验函数,入栈校验、非法抛 TypeError
- 任务 2(覆盖:Map/Set 内置泛型结构):用 Map 实现"按 id 索引"的缓存,Set 做去重统计
- 任务 3(覆盖:Proxy get/set、Reflect 转发):日志代理记录访问与修改次数,导出统计
- 任务 4(覆盖:校验代理、deleteProperty):账户对象代理,余额非负校验、关键字段禁删
- 任务 5(覆盖:默认值代理):withDefaults 代理,缺属性返回默认值
- 任务 6(覆盖:深代理):递归代理嵌套对象,嵌套属性同样拦截(用 typeof value === 'object' 递归)
- 任务 7(覆盖:defineProperty、原型动态):defineProperty 创建只读字段;运行时给原型加方法并验证所有实例可见
- 任务 8(覆盖:TS 方向):输出一段"TS 泛型对照"注释,说明 Stack<T> 声明如何给任务 1 补静态类型

实验环境:Node.js 20+,运行 `node metatoolkit.js`。

第一步 **写容器:**TypedStack 与 Map 索引缓存,验证"类型约束"与"任意键"

第二步 **写代理工具箱:**makeLogger/makeValidator/withDefaults/deepProxy 四个工厂函数

第三步 **后写演示:**每个工具一个迷你场景,输出拦截日志与统计

✅ 参考答案要点

核心: TypedStack + 四类代理工厂 + defineProperty/原型演示。约 140 行,覆盖全章知识点。

```
// metatoolkit.js — Node.js 20+ 运行:node metatoolkit.js
'use strict';

// 1. 类型受约束的栈(覆盖:动态类型"泛型"、运行时校验)
function createTypedStack(name, validate) {
  const items = [];
  return {
    push(item) {
      if (!validate(item)) {
        throw new TypeError(`${name} 拒绝元素:${item}`);
      }
      items.push(item);
    },
    pop() { return items.pop(); },
    get size() { return items.length; },
  };
}
const numStack = createTypedStack('数字栈', (v) => typeof v === 'number');

// 2. Map/Set 内置结构(覆盖:任意键、去重)
function indexDemo() {
  const byId = new Map();
  byId.set(1001, { name: '李雷' });
  byId.set('k-2', { name: '韩梅梅' }); // 字符串键
  console.log('Map 索引:', byId.get(1001).name, byId.get('k-2').name);

  const seen = new Set(['a', 'b', 'a', 'c', 'b']);
  console.log('Set 去重后数量:', seen.size); // 3
}

// 3. 日志代理(覆盖:Proxy get/set、Reflect 转发、闭包统计)
function makeLogger(target) {
  const stats = { get: 0, set: 0 };
  const proxy = new Proxy(target, {
    get(obj, prop) {
      stats.get++;
      return Reflect.get(obj, prop);
    },
    set(obj, prop, value) {
      stats.set++;
      return Reflect.set(obj, prop, value);
    },
  });
  proxy.__stats = stats; // 附加统计出口
  return proxy;
}

// 4. 校验代理(覆盖:set/deleteProperty 拦截)
function makeValidator(target, rules) {
  return new Proxy(target, {
    set(obj, prop, value) {
      const rule = rules[prop];
      if (rule && !rule(value)) {
        throw new TypeError(`字段 ${prop} 校验失败:${value}`);
      }
    }
  });
}
```

```

        return Reflect.set(obj, prop, value);
    },
    deleteProperty(obj, prop) {
        if (rules[prop]?.locked) return false; // 锁定字段禁删
        return Reflect.deleteProperty(obj, prop);
    },
});
}

// 5. 默认值代理(覆盖:get 拦截、默认值)
function withDefaults(target, defaults) {
    return new Proxy(target, {
        get(obj, prop) {
            const v = Reflect.get(obj, prop);
            return v === undefined ? defaults[prop] : v;
        },
    });
}

// 6. 深代理(覆盖:递归拦截、嵌套对象)
function deepProxy(target, handler) {
    const wrap = (obj) => {
        if (obj !== null && typeof obj === 'object') {
            return new Proxy(obj, {
                ...handler,
                get(t, p) {
                    const v = Reflect.get(t, p);
                    return wrap(v); // 值也是对象就继续包
                },
                set(t, p, val) {
                    if (typeof handler.set === 'function') handler.set(t, p, val);
                    return Reflect.set(t, p, val);
                },
            });
        }
        return obj;
    };
    return wrap(target);
}

// 7. defineProperty 与动态原型(覆盖:属性开关、运行时加方法)
function propertyDemo() {
    const ticket = {};
    Object.defineProperty(ticket, 'price', {
        value: 100, writable: false, enumerable: false, configurable: false,
    });
    console.log('只读且不可枚举:', ticket.price, Object.keys(ticket).length === 0);

    function Greeter(name) { this.name = name; }
    Greeter.prototype.hello = function () { return `你好,${this.name}`; };
    Greeter.prototype.bye = function () { return `${this.name} 再见`; }; // 运行时追加
    console.log(new Greeter('李雷').bye()); // 所有实例立即可用
}

// 8. TS 对照注释(覆盖:TS 泛型方向)

```

```

/*
 * TypeScript 对照(需 .ts):
 * function first(arr: T[]): T | undefined { return arr[0]; }
 * class Stack { private items: T[] = [];
 *   push(item: T): void { this.items.push(item); }
 *   pop(): T | undefined { return this.items.pop(); } }
 * 上面的 numStack 若用 TS 写,校验函数可换成类型参数 T 的静态约束,
 * 运行时校验(assert)与编译期检查(T)互补使用。
 */

// 9. 主流程
function main() {
  indexDemo();

  // 日志代理
  const user = makeLogger({ name: '李雷' });
  user.name = '韩梅梅';
  console.log('日志统计:', user.__stats);    // { get: 0, set: 1 }
  console.log(user.name);                    // 韩梅梅(get 计数 +1)

  // 校验代理
  const acc = makeValidator({ balance: 0 }, {
    balance: (v) => typeof v === 'number' && v >= 0,
  });
  acc.balance = 50;
  // acc.balance = -1;    // ✖ TypeError:校验失败
  // delete acc.balance; // 若规则加 locked:true 则被拒绝

  // 默认值代理
  const cfg = withDefaults({ port: 8080 }, { host: 'localhost', port: 3000 });
  console.log('默认值代理:', cfg.host, cfg.port); // localhost 8080

  // 深代理:嵌套修改也拦截
  const nested = deepProxy({ a: { b: { c: 1 } } }, {
    set(t, p, v) { console.log(`深代理拦截:${String(p)} = ${v}`); },
  });
  nested.a.b.c = 99;                // 触发深代理拦截
  console.log('深代理读取:', nested.a.b.c);

  // TypedStack
  numStack.push(1);
  numStack.push(2);
  console.log('数字栈弹出:', numStack.pop(), '| 栈大小:', numStack.size);

  propertyDemo();
  console.log('实验完成(TS 对照见源码注释)');
}

main();

```

设计说明:TypedStack 用"校验函数注入"模拟泛型约束,呼应 TS 的 `Stack<T>`;代理工具箱四个工厂覆盖 get/set/deleteProperty 三类拦截与深递归;日志代理的统计用闭包附加在 proxy 上;深代理递归包裹嵌套值,直观

展示"响应式框架的雏形";defineProperty 演示只读+不可枚举,动态原型演示运行时加方法。TS 对照注释给出任务 1 的静态版本,为第14章铺路。

下一章预告:第11章 反射与动态特性 —— typeof/instanceof 的坑、Object 反射全家、属性描述符,以及手写 JSON 序列化器。

第11章 反射与动态特性

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解"运行时类型信息":为什么 C 没有反射,JS 为什么天然就有
- 掌握 typeof 的全部结果与三个著名坑(null、数组、函数)
- 掌握 instanceof 的语义与跨 realm 陷阱
- 掌握 Object 反射全家:keys/values/entries/assign/getOwnPropertyDescriptors
- 掌握属性描述符:writable/enumerable/configurable 与 defineProperty
- 掌握 Reflect 全套与动态属性访问(方括号、in、delete)
- 掌握生产判断:何时该用反射,何时不该,替代手段清单

💡 从 C 到 JavaScript:本章主题如何迁移

C 没有反射,运行时"不知道"自己的类型: `sizeof(x)` 是编译期常量,结构体字段名在编译后只是偏移量,不可能"遍历一个结构体的所有字段"。想模拟(如序列化),只能手写字段清单或生成代码。高级语言把这份能力带到运行时: **运行时类型信息(RTTI)**——Java 的反射、Python 的内省、JS 的 Object/Reflect 全家,都能在程序运行时"查看自己的结构"。

JS 的反射尤其彻底,因为对象就是"动态属性表": `Object.keys(obj)` 枚举属性、`obj[key]` 用变量取属性、`'k' in obj` 检查存在——这些在 C 里统统做不到(或要靠宏 hack)。配合属性描述符,你可以精确控制每个字段的读写权与可见性,这比 C 的 struct 成员权限丰富得多。

但反射是双刃剑:它把"编译期该查出的错"推迟到运行期,性能也打折扣(动态属性访问比静态慢一个数量级)。生产判断标准非常清晰: **框架层(序列化、ORM、DI、调试工具)用反射,业务热路径不用**。第11章的实验(手写 JSON 序列化器)正是反射的经典教学用例——你会在 60 行代码里体会到 C 程序员写序列化时"没有反射"的痛苦,以及 JS 的轻松。

📦 前置知识自查

- C 的 sizeof、结构体成员访问、offsetof(可选)
- 第1章 typeof 与八种类型;第6章原型链与 hasOwnProperty
- 第10章 Proxy/Reflect(本章是它的"无拦截版"补充)

🚀 本章学习路线

1. **第一阶段(类型探测)**:读 §11.1–§11.2,把 typeof/instanceof 的坑全部过一遍
2. **第二阶段(Object 反射)**:读 §11.3–§11.4,练习枚举与描述符操作
3. **第三阶段(Reflect 与生产)**:读 §11.5–§11.7,理解反射的代价与边界
4. **第四阶段(实验)**:完成章末"属性遍历打印器 + 手写 JSON 序列化器"
5. **验收标准**:能写出一个通用"对象 → JSON"的递归序列化器,并处理数组/嵌套/循环引用

本章知识导图

- 第11章 反射与动态特性
 - §11.1 typeof:八种结果与三个坑
 - §11.2 instanceof:原型链语义与跨 realm
 - §11.3 Object 反射全家:keys/entries/assign
 - §11.4 属性描述符与 defineProperty
 - §11.5 Reflect 全套与动态属性
 - §11.6 动态特性:方括号、in、delete、原型操纵
 - §11.7 生产判断:何时用反射,替代手段

§11.1 typeof:八种结果与三个坑

```
// 完整结果表(第1章已见,这里做"反射"视角的复习)
console.log(typeof 42);           // 'number'
console.log(typeof 42n);          // 'bigint'
console.log(typeof 's');          // 'string'
console.log(typeof true);         // 'boolean'
console.log(typeof undefined);    // 'undefined'
console.log(typeof Symbol());     // 'symbol'
console.log(typeof {});           // 'object'
console.log(typeof function(){}); // 'function'

// 坑一:null
console.log(typeof null);          // 'object' ← 语言 bug,判断 null 用 === null

// 坑二:数组
console.log(typeof []);            // 'object' ← 不能靠 typeof 区分数组
console.log(Array.isArray([]));    // true ← 正确姿势

// 坑三:函数也是对象,但 typeof 特殊
console.log(typeof async () => {}); // 'function'(async 函数也是 function)

// 生产模式:类型判定工具函数
function typeOf(x) {
  if (x === null) return 'null';
  if (Array.isArray(x)) return 'array';
  const t = typeof x;
  if (t === 'object') return 'object';
  return t;
}
console.log(typeOf(null), typeOf([]), typeOf({}), typeOf(42));
// null array object number
```

⚠ 常见误区 11.1:typeof 判"对象是否为空"

`typeof obj === 'object'` 不能判断"obj 有没有内容"——空对象 `{}` 也是 `'object'`。判断"空对象"要 `Object.keys(obj).length === 0`;判断"对象存在"用 `obj !== null`。`typeof` 只回答"类别",不回答"内容",这是反射类操作的第一课。

§11.2 instanceof:沿原型链的类型检查

```
// 语义:obj instanceof Ctor = Ctor.prototype 是否在 obj 的原型链上
class Animal {}
class Dog extends Animal {}
const d = new Dog();
console.log(d instanceof Dog);      // true
console.log(d instanceof Animal);    // true ← 沿链向上匹配
console.log(d instanceof Object);    // true ← 所有对象都匹配 Object
console.log({} instanceof Object);   // true
console.log({} instanceof Animal);   // false

// 基本类型的坑:number 原始值不是对象
console.log(42 instanceof Number);   // false ← 原始值没有原型链
console.log(new Number(42) instanceof Number); // true ← 包装对象才行
// 生产:基本类型判断用 typeof 或 Object.prototype.toString

// 跨 realm 陷阱(生产必知):
// iframe / worker / vm 里创建的数组,其 Array.prototype 与主 realm 不同
// [] instanceof 主realm的Array 可能为 false
console.log(Array.isArray([]));       // true ← 跨 realm 也安全
// 同理:instanceof 在"包装/代理/跨上下文"场景都不可靠,优先用鸭子检查或
// Object.prototype.toString.call(x) === '[object Array]'

// Object.prototype.toString:万能类型标签
console.log(Object.prototype.toString.call([])); // [object Array]
console.log(Object.prototype.toString.call(null)); // [object Null]
console.log(Object.prototype.toString.call(new Date())); // [object Date]
console.log(Object.prototype.toString.call(/re/)); // [object RegExp]
```

§11.3 Object 反射全家:枚举与复制

```
const user = { id: 1, name: '李雷', score: 92 };

// 三个枚举出口(注意只枚举"自身可枚举"属性)
console.log(Object.keys(user));      // [ 'id', 'name', 'score' ]
console.log(Object.values(user));    // [ 1, '李雷', 92 ]
console.log(Object.entries(user));    // [ ['id',1], ['name','李雷'], ['score',92] ]

// entries 配 for...of 是最常用遍历(ES2017+):
for (const [k, v] of Object.entries(user)) {
  console.log(`${k} = ${v}`);
}

// 从 entries 重建对象:
const copy = Object.fromEntries(Object.entries(user));
console.log(copy);                  // { id: 1, name: '李雷', score: 92 }

// assign:合并/拷贝(浅拷贝!)
const a = { x: 1 };
const b = { y: 2 };
const merged = Object.assign({}, a, b);
console.log(merged);                // { x: 1, y: 2 }
// 现代等价写法:{ ...a, ...b }(第2章展开运算符)

// 所有属性(含不可枚举、Symbol):getOwnPropertyNames / getOwnPropertySymbols
const special = {};
Object.defineProperty(special, 'hidden', { value: 1, enumerable: false });
console.log(Object.keys(special));   // [] ← 不可枚举不出现在 keys
console.log(Object.getOwnPropertyNames(special)); // [ 'hidden' ] ← 全量

// 所有属性的完整描述符:
console.log(Object.getOwnPropertyDescriptors(user));
// { id: { value: 1, writable: true, enumerable: true, configurable: true }, ... }
```

⚠ 常见误区 11.2:keys 不包含继承属性,也不包含 Symbol

`Object.keys` 只返回 自身可枚举的字符串键:继承的属性(原型上的)看不到(用 `for...in` 才看得到);`Symbol` 键看不到(用 `Reflect.ownKeys` 或 `getOwnPropertySymbols`);不可枚举的看不到(用 `getOwnPropertyNames`)。"我想拿到这个对象所有字段"要组合: `Reflect.ownKeys(obj)` 最全。

§11.4 属性描述符:三个开关

每个属性背后有三个开关:**writable**(可改值)、**enumerable**(出现在 `keys`/`for...in`)、**configurable**(可删除/重定义)。C 的 `struct` 成员没有这些概念,这是 JS 属性系统独有的一层"反射能力"。

```

const config = {};
Object.defineProperty(config, 'apiKey', {
  value: 'sk-abc123',
  writable: false,      // 只读
  enumerable: false,    // 不可枚举(日志/序列化时不会泄露)
  configurable: false,  // 不可删除、不可改描述符
});

// 读属性值(反射):
console.log(config.apiKey);      // sk-abc123
console.log(Object.keys(config)); // [] ← 不可枚举,序列化不泄露

// 写与删被拒绝:
'use strict';
try {
  config.apiKey = 'x';          // 严格模式抛 TypeError
} catch (e) {
  console.log('只读拦截:', e.name);
}
console.log(delete config.apiKey); // false ← configurable:false,删除无效

// 查询描述符:
console.log(Object.getOwnPropertyDescriptor(config, 'apiKey'));
// { value: 'sk-abc123', writable: false, enumerable: false, configurable: false }

// 生产用途:内部状态对外只读、敏感字段防泄露、框架字段防篡改
function createInternal(secret) {
  const state = { secret };
  Object.defineProperty(state, 'secret', { enumerable: false });
  return state;
}
const s = createInternal('机密');
console.log(JSON.stringify(s)); // {} ← secret 不会进 JSON!

```

工程建议 11.1:defineProperty 的三个生产场景

① 敏感字段(`enumerable: false`),防止 `JSON.stringify`/日志把密钥、令牌带出去——比"删除字段"更安全(代码里还能用);② 只读配置(`writable: false`),防止运行时被意外改写;③ 常量属性绑定。注意:`Object.freeze` 是"全部属性一键只读+不可配置"的快捷方式,区别是 `freeze` 不改 `enumerable`。

§11.5 Reflect 全套与动态属性

```
// Reflect 与 Object 反射的分工:Reflect 更"函数式"、返回值友好
const obj = { a: 1 };

console.log(Reflect.get(obj, 'a'));           // 1,替代 obj['a']
console.log(Reflect.has(obj, 'a'));           // true,替代 'a' in obj
console.log(Reflect.ownKeys(obj));             // [ 'a' ],最全的键(含 Symbol)
console.log(Reflect.deleteProperty(obj, 'a')); // true,返回布尔而非 undefined
console.log(obj);                             // {}

// 动态属性访问:方括号是反射的日常形态
const key = 'name';
const person = { name: '韩梅梅', age: 17 };
console.log(person[key]);                     // 韩梅梅 ← 变量作键
person[key] = '李雷';                         // 动态写入
console.log(person.name);                     // 李雷

// 动态调用方法:Reflect.apply 或函数引用
const tools = {
  add(a, b) { return a + b; },
};
const method = 'add';
console.log(tools[method](1, 2));              // 3 ← 方法名也可以是变量
console.log(Reflect.apply(tools.add, tools, [3, 4])); // 7

// 构造反射:Reflect.construct 等价 new
class Point {
  constructor(x, y) { this.x = x; this.y = y; }
}
const p = Reflect.construct(Point, [1, 2]);
console.log(p instanceof Point);              // true

// 原型反射:
console.log(Reflect.getPrototypeOf(p) === Point.prototype); // true
```

§11.6 动态特性:in、delete、原型操纵与遍历深度

```
// in 与 hasOwnProperty 的分工
const proto = { inherited: true };
const own = Object.create(proto);
own.self = 1;
console.log('self' in own);           // true(自己的)
console.log('inherited' in own);      // true(in 沿原型链)
console.log(own.hasOwnProperty('inherited')); // false(不是自己的)
console.log(Object.hasOwn(own, 'inherited')); // false ← ES2022 推荐写法

// delete:删除自身属性
delete own.self;
console.log('self' in own);           // false
// delete 不作用于原型属性:own 仍能看到 inherited

// 通用深度遍历:反射 + 递归(生产日志/调试/测试断言的基础)
function deepDump(obj, prefix = '') {
  if (obj === null || typeof obj !== 'object') {
    console.log(prefix + ' = ' + String(obj));
    return;
  }
  for (const [k, v] of Object.entries(obj)) {
    deepDump(v, prefix + '.' + k);
  }
}
deepDump({ a: { b: 1 }, c: [2, 3] });
// .a.b = 1
// .c.0 = 2
// .c.1 = 3

// 循环引用保护:遍历时记录已访问对象
function safeDump(obj, seen = new Set()) {
  if (obj === null || typeof obj !== 'object') return String(obj);
  if (seen.has(obj)) return '[循环]';
  seen.add(obj);
  const out = {};
  for (const [k, v] of Object.entries(obj)) {
    out[k] = safeDump(v, seen);
  }
  return JSON.stringify(out);
}
const cyc = { name: '环' };
cyc.self = cyc;
console.log(safeDump(cyc)); // {"name":"环","self":"[循环]"}

```

§11.7 生产判断:何时用反射,何时不用

场景	推荐	理由
序列化(对象→JSON/存储)	反射(entries/递归)	通用、字段增删免维护
表单校验(动态字段)	反射 + schema 校验	字段集合运行时才知道
ORM/数据层映射	反射 + 元数据(第12章)	框架层必须通用
调试/日志/测试断言	反射遍历打印	对象结构不确定
业务热路径(每秒万次)	静态访问 obj.field	反射慢 1-2 个数量级
固定结构处理	显式代码,不用反射	可读性、类型安全、编辑器补全
对象内部逻辑	this.xxx 直接访问	反射只用于"通用代码"

```
// 热路径对比(简单量化):
function bench() {
  const obj = { a: 1 };
  const start = Date.now();
  for (let i = 0; i < 1e6; i++) obj.a++;           // 静态访问
  const mid = Date.now();
  for (let i = 0; i < 1e6; i++) obj['a']++;        // 动态访问(方括号)
  const end = Date.now();
  console.log(`静态:${mid - start}ms 动态:${end - mid}ms`);
}
// bench(); // 动态通常慢 1.5-3 倍;带字符串拼接/Reflect 更慢

// 替代手段清单(不用反射也能达到目的):
// ① 固定结构 → 显式字段访问 + TypeScript 类型
// ② 需要通用能力 → 生成代码/模板(第12章 go:generate 哲学)
// ③ 需要动态校验 → 用校验库(zod/joi),内部仍是反射但经过优化
console.log('判断口诀:通用性优先用反射,确定性优先用静态');
```

⚠ 常见误区 11.3:用反射遍历"大对象"做热路径序列化

生产事故:把反射序列化器用于每请求一次的响应体(对象几千字段),QPS 直接腰斩。缓解:① 缓存"键列表"(对象形状不变时,字段集合不变, `Object.keys` 结果可缓存);② 手写热点对象的结构化序列化;③ 用 Fastify 的 schema 序列化(编译成代码)。反射可以缓存,不能盲目信任每次调用。

本章速查表

主题	速查要点
typeof 结果	number/bigint/string/boolean/undefined/symbol/object/function 八种
typeof null	'object'(bug);判 null 用 === null
typeof 数组	'object';判数组用 Array.isArray(跨 realm 安全)
instanceof	沿原型链查;Ctor.prototype 在链上即 true;原始值恒 false
Object.keys	自身可枚举字符串键(不含继承/Symbol/不可枚举)
Object.entries	[k,v] 数组;配 for...of 是最常用遍历;fromEntries 可还原
Object.assign	浅拷贝合并;现代用 {...a, ...b}
Reflect.ownKeys	最全键(含 Symbol 与不可枚举)
描述符三开关	writable 可改 / enumerable 可见 / configurable 可删可重定义
defineProperty	生产:敏感字段 enumerable:false 防泄露;配置 writable:false 防改写
Object.hasOwn	ES2022,替代 hasOwnProperty 的安全写法
in 运算符	沿原型链查;hasOwn 只查自身
delete	删自身属性;不删原型属性;configurable:false 时无效
深度遍历	递归 + entries + Set 防循环引用
性能	动态访问比静态慢 1.5–3 倍;热路径别用反射,或缓存键列表
生产判断	框架/序列化/校验用反射;业务固定结构用静态

最小必会清单

- 能写出 typeof 八种结果,并说出 null/数组/函数三个特例
- 能说出 instanceof 的原理(原型链)与跨 realm 陷阱
- 能用 keys/values/entries + for...of 遍历对象
- 能说出 keys 的三种"看不见"(继承/Symbol/不可枚举)及对应 API
- 能用 defineProperty 创建只读 + 不可枚举字段
- 能说出 Object.freeze 与 defineProperty 的关系
- 能用 Reflect.ownKeys 拿到最全键集合
- 能写出带循环引用保护的深度遍历
- 能说出反射慢于静态访问的量级与缓存手段
- 能按"通用性 vs 确定性"判断何时用反射

自测题

Q 1. 写出输出: `console.log(typeof null, typeof [], typeof () => {}, Array.isArray([]))`

✓ 参考答案

'object'、'object'、'function'、true。null 是 bug;数组也是 object;箭头函数是 function。

Q 2. `class A {};` `const a = new A();` 请写出 `a instanceof A`、`a instanceof Object` 的结果并解释。

✓ 参考答案

true、true。instanceof 沿原型链查找:A.prototype 在 a 的链上,Object.prototype 也在(链的终点),所以都匹配。

Q 3. `Object.keys(obj)` 看不到哪三类属性?分别用什么 API?

✓ 参考答案

继承属性(for...in 或沿原型遍历)、Symbol 键(Reflect.ownKeys/getOwnPropertySymbols)、不可枚举属性(getOwnPropertyNames/Reflect.ownKeys)。

Q 4. 如何让一个字段"代码里能读,但 `JSON.stringify` 和日志里不出现"?

✓ 参考答案

defineProperty 设置 enumerable: false,JSON.stringify 与 Object.keys 只序列化可枚举属性。

Q 5. `delete obj.x` 返回什么?在什么情况下返回 false?

✓ 参考答案

成功删除返回 true;configurable: false 的属性、不可删除的(如非严格模式的某些内建)返回 false。

Q 6. 遍历一个可能循环引用的对象树,如何防止死循环?

✓ 参考答案

用 Set 记录已访问对象,递归前检查,命中则标记 '[循环]' 并返回。

章末实验题:对象内省工具箱(通用序列化器)

实验 11:对象内省工具箱(序列化器 + 遍历打印机)

实验目标:实现通用"对象属性遍历打印机"与手写 JSON 序列化器,覆盖全章反射 API。

实验要求:

- 任务 1(覆盖:Object.entries、递归):dumpObject(obj) 深度打印所有可枚举字段,缩进表示层级
- 任务 2(覆盖:循环引用防护、Set):遍历带自引用的对象不死循环,输出 [循环] 标记
- 任务 3(覆盖:typeof、Array.isArray、typeof 工具):serialize(obj) 手写 JSON:支持 null/布尔/数字/字符串/数组/对象,递归序列化
- 任务 4(覆盖:字符串转义、不可枚举):字符串值转义引号与换行;用 defineProperty 制造不可枚举字段,序列化器必须忽略它(与 JSON.stringify 行为一致)
- 任务 5(覆盖:Reflect.ownKeys、描述符):inspectProperty(obj, key) 输出该属性完整描述符(三开关)
- 任务 6(覆盖:getOwnPropertyNames、跨原型):diffKeys(a, b) 用 getOwnPropertyNames 对比两对象自身键差异
- 任务 7(覆盖:Object.fromEntries、动态属性):把序列化结果解析回来(用 fromEntries 重建),验证往返一致性
- 任务 8(覆盖:生产判断):注释说明序列化器与 JSON.stringify 的行为差异,以及何时该用哪个

实验环境:Node.js 20+,运行 `node introspect.js`。

第一步 **写类型工具:**typeof 区分 null/array/object,序列化主函数按类型分发

第二步 **写递归:**对象序列化 = 遍历 entries → 递归每个值 → 拼 JSON 片段;数组同理

第三步 **后做演示:**构造含嵌套/循环/不可枚举/特殊字符的对象,验证与 JSON.stringify 输出对比

✅ 参考答案要点

核心:typeof 分发 + 递归序列化 + 转义 + 防循环 + 描述符内省。约 130 行,覆盖全章反射 API。

```
// introspect.js — Node.js 20+ 运行:node introspect.js
'use strict';

// 1. 类型工具(覆盖:typeof、Array.isArray、null 特判)
function typeOf(x) {
  if (x === null) return 'null';
  if (Array.isArray(x)) return 'array';
  const t = typeof x;
  if (t === 'object') return 'object';
  if (t === 'function') return 'function';
  return t;
}

// 2. 深度遍历打印(覆盖:Object.entries、递归、缩进)
function dumpObject(obj, depth = 0, seen = new Set()) {
  const indent = ' '.repeat(depth);
  if (obj === null || typeof obj !== 'object') {
    console.log(indent + String(obj));
    return;
  }
  if (seen.has(obj)) {
    console.log(indent + '[循环]');
    return;
  }
  seen.add(obj);
  for (const [k, v] of Object.entries(obj)) { // 只遍历可枚举自身属性
    console.log(indent + k + ':');
    dumpObject(v, depth + 1, seen);
  }
}

// 3. 手写 JSON 序列化器(覆盖:typeOf 分发、递归、转义)
function serialize(value) {
  switch (typeOf(value)) {
    case 'null':
      return 'null';
    case 'number':
      if (!Number.isFinite(value)) return 'null'; // NaN/Infinity → null(与内置一致)
      return String(value);
    case 'boolean':
      return String(value);
    case 'string':
      return '"' + escapeString(value) + '"';
    case 'array': {
      const parts = value.map((v) => serialize(v)); // undefined 元素 → null(简化)
      return '[' + parts.join(',') + ']';
    }
    case 'object': {
      const parts = [];
      for (const [k, v] of Object.entries(value)) {
        if (v === undefined) continue; // undefined 属性丢弃(与内置一致)
        parts.push('"' + escapeString(k) + '":' + serialize(v));
      }
      return '{' + parts.join(',') + '}';
    }
  }
}
```

```

    default:
        return 'null'; // function/symbol → null(丢弃语义)
    }
}
// 字符串转义(覆盖:反斜杠、引号、换行、制表符)
function escapeString(s) {
    return s
        .replace(/\\/g, '\\\\')
        .replace(/"/g, '\\"')
        .replace(/\n/g, '\\n')
        .replace(/\t/g, '\\t')
        .replace(/\r/g, '\\r');
}

// 4. 属性描述符内省(覆盖:getOwnPropertyDescriptor、Reflect.ownKeys)
function inspectProperty(obj, key) {
    const desc = Object.getOwnPropertyDescriptor(obj, key);
    if (!desc) return `属性 ${key} 不存在`;
    return {
        key,
        writable: desc.writable,
        enumerable: desc.enumerable,
        configurable: desc.configurable,
    };
}

// 5. 键集合对比(覆盖:getOwnPropertyNames、跨原型)
function diffKeys(a, b) {
    const ka = Object.getOwnPropertyNames(a);
    const kb = new Set(Object.getOwnPropertyNames(b));
    const onlyInA = ka.filter((k) => !kb.has(k));
    return onlyInA;
}

// 6. 演示对象(覆盖:嵌套、循环、不可枚举、特殊字符)
function buildSample() {
    const sample = {
        name: '李雷 "引号"',
        note: '换行\n与制表\t符',
        score: 92.5,
        ok: true,
        tags: ['a', 'b', null],
        nested: { deep: { value: 1 } },
    };
    Object.defineProperty(sample, 'secret', {
        value: '隐藏字段', enumerable: false, writable: false,
    });
    sample.self = sample; // 循环引用
    return sample;
}

// 7. 主流程
function main() {
    const sample = buildSample();

```

```

console.log('== 深度遍历(循环引用防护) ==');
dumpObject(sample);

console.log('== 手写序列化 vs 内置 JSON.stringify ==');
const mine = serialize(sample);
console.log('mine:', mine.slice(0, 120) + '...(截断)');
console.log('内置:', JSON.stringify(sample).slice(0, 120) + '...(截断)');

const clean = { name: '韩梅梅', scores: [1, 2, 3], ok: true };
console.log('往返一致性:', serialize(clean) === JSON.stringify(clean));

console.log('== 属性描述符自省 ==');
console.log(inspectProperty(sample, 'secret')); // writable:false 等
console.log(inspectProperty(sample, 'name'));

console.log('== 键对比 ==');
const other = { name: 'x', extra: 1 };
console.log('sample 独有键:', diffKeys(sample, other).join(', '));
}

main();

```

设计说明:serialize 与内置 JSON.stringify 的行为对齐点:NaN/Infinity→null、undefined 属性丢弃、不可枚举忽略、函数/符号→null;差异点:不支持 toJSON 钩子与缩进参数——注释里注明"生产用 JSON.stringify,手写版用于理解反射"。dumpObject 用 Set 防循环;inspectProperty 输出三开关,呼应 §11.4;diffKeys 演示 getOwnPropertyNames 的全量语义。运行后对比"mine"与"内置"两个输出,可直观看到手写序列化器与标准行为的一致与差异。

下一章预告:第12章 注解与装饰器 —— 高阶函数式装饰、TC39 装饰器提案现状与 TypeScript 装饰器,以及计时/重试/缓存的完整实现。

第12章 注解与装饰器

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解"元数据 vs 行为注入"的概念,与 C 的 `#pragma/_attribute_` 对照
- 掌握高阶函数式"装饰":用组合函数包装函数,加行为不改源码
- 亲手实现四个生产级装饰器:计时、重试、缓存、防抖
- 理解 TC39 装饰器提案的现状(阶段)与浏览器/Node 支持情况
- 了解 TypeScript 装饰器的用法(类/方法/属性/参数),能读懂框架代码
- 理解"装饰 vs 魔法"的生产边界:何时用装饰器,何时用普通函数

💡 从 C 到 JavaScript:本章主题如何迁移

C 里给函数"加行为"的手段很有限:宏包一层(有副作用)、复制粘贴改一份(维护噩梦)、函数指针表(改调用方)。C 编译器的 `#pragma` 与 `__attribute__((deprecated))` 属于**元数据**——告诉工具链"这个函数该警告",但不改变函数行为。**元数据 = 描述代码的代码;行为注入 = 实际改变运行的代码**,这是本章的第一对概念。

JS 的函数式装饰走"行为注入"路线,而且不需要任何新语法:装饰器就是一个高阶函数——接收原函数,返回新函数,新函数在调用原函数前后附加逻辑(计时、重试、缓存、鉴权)。这正是第4章"函数是一等公民"的实战延伸,也是组合优于继承(第6章)在函数域的体现。

真正的 **@decorator** 语法(Java 的注解、Python 的装饰器)在 JS 里仍在提案路上:TC39 装饰器提案经历了 2014 年版(被 TypeScript 采用)与 2022 年版(标准化方向)的迭代,截至本书写作,Node 20 与主流浏览器 原生不执行 **@语法**,生产用法是 TypeScript 编译或 Babel 转译。所以读框架代码(Vue/Angular/NestJS/Express 类型)时,你能看到大量 `@Component`、`@Injectable`——那是 TypeScript 装饰器,不是原生 JS。

本章的立场:函数式装饰是 **生产 JS 的主力** (零依赖、可测试);TS 装饰器是 **阅读框架的能力**;提案语法是 **未来方向**。三个层次都要会,但代码里先用第一个。

📦 前置知识自查

- C 的 `#pragma`、`__attribute__((deprecated/noreturn))`(了解即可)
- 第4章:函数是一等公民、闭包、rest 参数、call/apply
- 第10章:高阶函数与组合思想
- 第9章:Promise 与错误传播(重试装饰器要用)

🚀 本章学习路线

1. **第一阶段(概念与函数式装饰)**:读 §12.1–§12.2,理解"装饰 = 高阶函数"
2. **第二阶段(四件套实现)**:读 §12.3,亲手实现计时/重试/缓存/防抖并测试
3. **第三阶段(提案与 TS)**:读 §12.4–§12.5,能读懂框架里的 @ 语法
4. **第四阶段(实验)**:完成章末"计时 + 重试 + 缓存组合装饰器"
5. **验收标准**:不看资料,能写出一个带参数的装饰器(如 `withRetry(fn, 3)`)并解释组合顺序

本章知识导图

- 第12章 注解与装饰器
 - §12.1 概念:元数据 vs 行为注入、与 C 的对照
 - §12.2 函数式装饰:高阶函数组合
 - §12.3 四个生产级装饰器:计时/重试/缓存/防抖
 - §12.4 TC39 装饰器提案现状
 - §12.5 TypeScript 装饰器:类/方法/属性/参数
 - §12.6 生产边界:装饰 vs 魔法

§12.1 概念:元数据与行为注入

对比项	C	JS 函数式装饰
元数据	<code>#pragma pack</code> 、 <code>__attribute__((deprecated))</code>	注释/JSDoc、Symbol 属性标记
行为注入	宏包一层(危险)、手改源码	高阶函数包装,原函数不变
语法	预处理指令/属性	普通函数调用: <code>withLog(fn)</code>
可测试性	宏难以单测	装饰器本身是纯函数,可单测
@ 语法	无	TC39 提案中;TS 可用

```

// C 的元数据示例(概念对照):
// #pragma pack(1)           ← 编译器指令:结构体紧凑对齐
// __attribute__((deprecated)) ← 编译器指令:调用处发警告
// 它们"描述"代码,但不改变函数逻辑。

// JS 的对应物:JSDoc 注释(元数据)+ Symbol 标记
const HAS_CACHE = Symbol('hasCache');
const api = {
  fetchUser(id) {
    return { id, name: '用户' + id };
  },
};
api[HAS_CACHE] = true;           // 运行时元数据:标记这个 API 可缓存
console.log(api[HAS_CACHE]);    // true

// 而"行为注入"在 JS 里是函数式装饰(本章主角):
function withLog(fn) {
  return function (...args) {
    console.log(`调用 ${fn.name},参数:`, args);
    const result = fn(...args);
    console.log(`返回:`, result);
    return result;
  };
}
const loggedFetch = withLog(api.fetchUser);
loggedFetch(7);
// 调用 fetchUser,参数: [ 7 ]
// 返回: { id: 7, name: '用户7' }
// 原函数 fetchUser 未被修改(纯包装)

```

§12.2 函数式装饰:组合函数的艺术

```
// 装饰器的本质模板:接收函数,返回增强函数
function decorator(fn) {
  return function (...args) {
    // 前置逻辑
    const result = fn(...args);
    // 后置逻辑
    return result;
  };
}

// 带参数的装饰器:外层函数收参数,内层收函数
function withConfig(options) {
  return function (fn) {
    return function (...args) {
      // options 可用
      return fn(...args);
    };
  };
}

// 组合:多个装饰器叠加,从内向外执行
const finalFn = withLog(withTimer(someFn));
// someFn 先被 withTimer 包装,再被 withLog 包装
// 调用顺序:withLog 的前置 → withTimer 的前置 → someFn → withTimer 后置 → withLog 后置

function withTimer(fn) {
  return function (...args) {
    const start = performance.now();
    const result = fn(...args);
    console.log(`${fn.name} 耗时 ${(performance.now() - start).toFixed(2)}ms`);
    return result;
  };
}

function someFn(x) { return x * 2; }
const composed = withLog(withTimer(someFn));
console.log('组合结果:', composed(21));
// 调用 someFn,参数: [ 21 ]
// someFn 耗时 0.02ms
// 返回: 42
```

★ 重点结论:装饰器的三不变原则

好的装饰器必须保持:① 签名不变 (参数与返回值原样透传);② this 不变 (对象方法被装饰后,this 仍指向原调用者——用普通 function + apply 转发);③ 原函数不可变 (永远返回新函数,绝不修改原函数)。违背任一条,装饰器就会在组合中引入隐蔽 bug。

§12.3 四个生产级装饰器

12.3.1 计时装饰器

```
// 生产用法:性能观测、慢接口告警
function withTiming(fn, { thresholdMs = 100, log = console.log } = {}) {
  return function (...args) {
    const start = process.hrtime.bigint();
    try {
      return fn.apply(this, args);          // apply 保持 this
    } finally {
      const ms = Number(process.hrtime.bigint() - start) / 1e6;
      if (ms > thresholdMs) {
        log(`[慢调用] ${fn.name} 耗时 ${ms.toFixed(2)}ms(阈值 ${thresholdMs}ms)`);
      }
    }
  };
}

const slow = withTiming((n) => {
  const t = Date.now();
  while (Date.now() - t < n) {}           // 忙等
  return n;
});

slow(150); // [慢调用] 耗时约 150ms,触发告警
```

12.3.2 重试装饰器(异步版,生产核心)

```
// 用法:网络抖动重试、外部服务临时失败重试
function withRetry(fn, { retries = 3, delayMs = 50, onError = null } = {}) {
  return async function (...args) {
    let lastErr;
    for (let attempt = 1; attempt <= retries; attempt++) {
      try {
        return await fn.apply(this, args);
      } catch (err) {
        lastErr = err;
        onError?.(err, attempt);
        if (attempt < retries) {
          await sleep(delayMs * attempt);    // 线性退避
        }
      }
    }
    throw lastErr;    // 重试耗尽,上抛
  };
}

function sleep(ms) { return new Promise((r) => setTimeout(r, ms)); }

// 模拟:前两次失败第三次成功
let calls = 0;
const flaky = withRetry(async (x) => {
  calls++;
  if (calls < 3) throw new Error(`第${calls}次失败`);
  return x * 10;
}, { retries: 4 });
flaky(5).then((v) => console.log('重试后成功:', v, '| 共调用', calls, '次'));
// 重试后成功: 50 | 共调用 3 次
```

12.3.3 缓存装饰器(同步版)

```
// 用法:纯函数结果缓存(重计算、不可变输入)
function withCache(fn, { maxSize = 100, getKey = JSON.stringify } = {}) {
  const cache = new Map();
  return function (...args) {
    const key = getKey(args);
    if (cache.has(key)) {
      return cache.get(key); // 命中直接返回
    }
    const result = fn.apply(this, args);
    if (cache.size >= maxSize) {
      cache.delete(cache.keys().next().value); // 淘汰最旧(FIFO 简化)
    }
    cache.set(key, result);
    return result;
  };
}

let computeCalls = 0;
const fib = withCache((n) => {
  computeCalls++;
  return n <= 1 ? n : fib(n - 1) + fib(n - 2); // 注意:递归内部也走缓存
}, { maxSize: 100 });
console.log('fib(30) =', fib(30)); // 832040
console.log('实际计算次数:', computeCalls); // 31(每个 n 只算一次,否则爆炸)
console.log('再次调用命中缓存:', fib(30)); // 832040,computeCalls 不变
```

12.3.4 防抖/节流装饰器(第9章复习,装饰器视角)

```
// 装饰器形态的防抖:包装后,连续调用只触发最后一次
function debounce(fn, wait = 300) {
  let timer = null;
  return function (...args) {
    clearTimeout(timer);
    const ctx = this; // 保持 this
    timer = setTimeout(() => fn.apply(ctx, args), wait);
  };
}

const save = debounce((data) => {
  console.log('已保存:', data); // 只应输出最后一次
}, 100);
save('a');
save('ab');
save('abc');
setTimeout(() => console.log('防抖完成'), 200);
// 输出:已保存: abc(仅一次)
```

⚠ 常见误区 12.1:装饰器忘了保持 this,对象方法爆炸

对象方法被装饰后,若包装函数用箭头函数(没有自己的 this)或漏了 apply 转发,方法内的 this 会丢失: `obj.m = withLog(obj.m); obj.m()` 里 this 变 undefined。修复模板: 包装函数用 function 声明 + `fn.apply(this, args)`; 异步装饰器同理 `await fn.apply(this, args)`。

§12.4 TC39 装饰器提案现状

```
// 现状(截至 2026):TC39 装饰器提案处于 Stage 3(候选);
// Node 20/22 与主流浏览器均不原生执行 @ 语法。
// 两种生产用法:
// 1. TypeScript 编译(experimentalDecorators,传统 TS 装饰器)
// 2. Babel @babel/plugin-proposal-decorators(按提案语义转译)
// 提案语法(2022 版,示意,需转译器执行):
// function logged(value, context) {
//   return function (...args) {           // 包装方法
//     console.log('调用', context.name);
//     return value.apply(this, args);
//   };
// }
// class C {
//   @logged
//   method() { return 1; }
// }

// 生产建议:业务代码不要依赖 @ 语法(转译链与版本碎片化);
// 读代码时能认出 "@xxx" 是装饰器即可,框架内部实现与这里的高阶函数是同一思想
console.log('TC39 装饰器:提案 Stage 3,Node/浏览器未原生支持');
```

§12.5 TypeScript 装饰器:读框架代码的能力

```
// 框架代码里最常见的 @ 语法(以 Angular/NestJS 风格为例,示意):
// @Component({ selector: 'app-root' })      ← 类装饰器:注册元数据
// class AppComponent { }
// @Injectable()                             ← 依赖注入标记
// class UserService { }
// @Controller('/users')                     ← 路由映射
// class UserController { }

// TS 装饰器分四种(需要 experimentalDecorators):
// ① 类装饰器:接收构造函数,可替换/扩展类
// ② 方法装饰器:接收(target, propertyKey, descriptor),可改 descriptor.value
// ③ 属性装饰器:接收(target, propertyKey),只读元数据
// ④ 参数装饰器:接收(target, propertyKey, parameterIndex),只读元数据

// 等价思想的 JS 实现:用"注册表"手动完成框架元数据(不需要转译器)
const ROUTES = new Map();
function register(route, handler) {
  ROUTES.set(route, handler);
}
class UserController {
  list() { return ['u1', 'u2']; }
}
// 手动"装饰":给方法挂路由元数据
register('/users', new UserController().list);
console.log('路由注册表:', [...ROUTES.keys()]); // [ '/users' ]

// 方法装饰器等价实现(descriptor 思想):
function methodDecorator(target, key, descriptor) {
  const original = descriptor.value;
  descriptor.value = function (...args) {           // 替换方法体
    console.log(`访问 ${key}`);
    return original.apply(this, args);
  };
  return descriptor;
}
// 上面这段是 TS 编译器在执行 @xxx 时的标准做法;
// 原生 JS 里手动调用等价逻辑即可
console.log('TS 装饰器 = 编译期执行的元编程(descriptor 操作)');
```

⚠ 常见误区 12.2:把 TS 装饰器当原生 JS 语法

在 .js 文件里写 `@Component` 会直接 `SyntaxError`;在 .ts 文件里也依赖 `tsconfig` 的 `experimentalDecorators: true` 与 `emitDecoratorMetadata`。而且 TS 装饰器与 TC39 提案语法 **不是一回事** (参数顺序、返回值语义都不同)。生产判断:能编译、团队统一即可用 TS 装饰器;原生 JS 团队一律用函数式装饰。

§12.6 生产边界:装饰 vs 魔法

🚀 装饰器生产检查清单

- 优先函数式装饰(withX(fn)),零依赖、可单测、可组合
- 装饰器只做"横切关注点":计时/日志/重试/缓存/鉴权/限流——不做业务逻辑
- 装饰器保持幂等:同一函数装饰两次不应产生重复副作用(用 Symbol 标记防重复)
- 重试/防抖等异步装饰器:注意 this、注意并发调用(重试期间再次调用需排队)
- 需要元数据(框架要读取标记)时,用 Symbol 键或装饰器注册表,不用魔法字符串
- @ 语法:TS 项目按团队规范使用;纯 JS 项目等 TC39 Stage 4 落地再评估

```
// 幂等装饰器:防止同一函数被重复包装
const WRAPPED = Symbol('wrapped');
function onceDecorated(fn) {
  if (fn[WRAPPED]) return fn; // 已装饰,直接返回
  const wrapped = function (...args) {
    return fn.apply(this, args);
  };
  wrapped[WRAPPED] = true;
  return wrapped;
}
// 框架内部常用这种技巧避免重复包裹

// 装饰器注册表:把"标记"与"行为"解耦
const DECORATORS = new Map();
function registerDecorator(name, impl) {
  DECORATORS.set(name, impl);
}
function applyDecorators(fn, names) {
  return names.reduceRight((f, name) => {
    const impl = DECORATORS.get(name);
    return impl ? impl(f) : f;
  }, fn);
}
registerDecorator('timing', withTiming);
registerDecorator('retry', (f) => withRetry(f, { retries: 3 }));
const enhanced = applyDecorators(someFn, ['timing', 'retry']);
console.log('注册表装饰后调用:', enhanced(3)); // 6(带计时与重试)
```

🚀 工程建议 12.1:装饰器与中间件的选择

HTTP 服务里,与"请求上下文"绑定的横切逻辑(日志、鉴权、限流)用 **中间件** (Express/Koa/Fastify 的 app.use), 与"单个函数"绑定的用 **装饰器**。判断标准:关心"每个请求"还是"这个函数"?前者中间件(请求生命周期统一管理), 后者装饰器(函数级精确控制)。两种都别混用,否则横切逻辑会重复执行。

本章速查表

主题	速查要点
元数据 vs 行为注入	元数据描述代码(#pragma/JSDoc/Symbol);行为注入改变运行(装饰器)
装饰器本质	高阶函数:fn → 新 fn,前置/后置逻辑
带参数装饰器	外层收配置,内层收函数,再内层收参数
组合顺序	withA(withB(fn)):调用时 A 外层先执行,返回时 A 后执行
三不变	签名不变、this 不变(apply 转发)、原函数不变
计时	process.hrtime.bigint() 或 performance.now();阈值告警
重试	async 循环 + 退避;耗尽上抛 lastErr;onError 钩子
缓存	Map 存键值;JSON.stringify 做键;注意淘汰策略
防抖/节流	clearTimeout 重置 / 时间戳限频;保持 this
TC39 提案	Stage 3;Node/浏览器未原生支持 @;用 TS 或 Babel
TS 装饰器	类/方法/属性/参数四种;方法装饰器改 descriptor;需 experimentalDecorators
与提案区别	TS 装饰器与 TC39 提案语义不同,不要混为一谈
幂等	Symbol 标记防重复包装
注册表	装饰器名 → 实现;按名组合应用(reduceRight)
生产选择	横切逻辑按请求用中间件,按函数用装饰器

最小必会清单

- 能说出元数据与行为注入的区别,各举一个 C 与 JS 的例子
- 能写出装饰器的标准模板(前置/后置逻辑、apply 保 this)
- 能写出带参数的装饰器(如 withRetry(fn, {retries:3}))
- 能说出 withA(withB(fn)) 的执行顺序
- 能写出同步缓存装饰器与"命中/未命中"分支
- 能写出异步重试装饰器的完整循环
- 能说出 TC39 装饰器提案当前阶段与原生支持现状
- 能看懂 @Component/@Injectable 并说出它们在 TS 中的性质
- 能说出 TS 装饰器与 TC39 提案的差异
- 能用 Symbol 实现幂等装饰器

自测题

Q 1. 装饰器与 C 的 #pragma 有何本质区别?

✅ 参考答案

#pragma/_attribute_ 是元数据,描述代码但不改变运行逻辑(编译器处理);JS 装饰器(函数式)是行为注入,在运行时实际包装函数、改变其行为。前者"注释给工具链",后者"包装给运行时"。

Q 2. withA(withB(fn)) 调用时,执行顺序如何?

✅ 参考答案

A 的前置 → B 的前置 → fn → B 的后置 → A 的后置。装饰器从内向外执行,像洋葱剥皮。

Q 3. 装饰器包装对象方法时,为什么必须用 function + apply(this)?箭头函数行吗?

✅ 参考答案

箭头函数没有自己的 this,包装后方法内的 this 会丢失(变外层作用域)。用普通 function + fn.apply(this, args) 才能把调用者的 this 转发给原函数。

Q 4. withRetry 的异步实现里,重试耗尽后应该做什么?为什么不能静默返回?

✅ 参考答案

throw lastErr 上抛最后一次错误。静默返回会让调用方以为成功,错误被吞掉,与"三不变"和错误传播原则(第9章)相悖。

Q 5. TC39 装饰器提案与 TypeScript 装饰器是同一件事吗?

✅ 参考答案

不是。TC39 是标准提案(2022 版语义,Stage 3);TS 装饰器是 TypeScript 自己的实现(experimentalDecorators),参数与返回语义不同,属于早期提案版本的分叉。

Q 6. 如何防止同一函数被装饰器重复包装?

✅ 参考答案

用 Symbol 标记:包装前检查 fn[SYMBOL],已标记直接返回原包装;包装后给新函数打上标记。

章末实验题:装饰器工具箱(decorators.js)

实验 12:装饰器工具箱(计时 + 重试 + 缓存 + 组合)

实验目标:实现完整装饰器库并用组合构建"可缓存 + 可重试 + 可计时"的业务函数,全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:函数式装饰模板、三不变):withLog 装饰器,保持签名/this/原函数不变
- 任务 2(覆盖:计时):withTiming 同步计时,阈值告警,返回耗时统计
- 任务 3(覆盖:重试、异步、错误传播):withRetry 异步重试 + 线性退避 + onError 钩子,耗尽上抛
- 任务 4(覆盖:缓存、纯函数):withCache 同步缓存 + 最大容量淘汰 + 命中统计导出
- 任务 5(覆盖:组合顺序):compose 工具(从左到右应用),验证组合调用顺序
- 任务 6(覆盖:幂等):onceDecorated 防重复包装,验证同一函数两次包装只包一次
- 任务 7(覆盖:注册表):注册表 + applyDecorators,按名组合
- 任务 8(覆盖:TS/提案坐标):注释说明 TC39 现状与 TS 装饰器对应关系;业务函数用"组合式装饰"完成(读框架代码时再遇 @ 语法即懂)

实验环境:Node.js 20+,运行 `node decorators.js`。

第一步 写四个基础装饰器:每个都遵守三不变,单测各自行为

第二步 写组合工具:compose(...decorators)(fn) 与 applyDecorators 注册表

第三步 后组装业务函数:fetchUser 用 cache+retry+timing 组合,验证缓存命中与重试路径

✅ 参考答案要点

核心:四个基础装饰器 + compose/注册表/幂等三个组合工具 + 业务组装。约 150 行,覆盖全章知识点。

```
// decorators.js — Node.js 20+ 运行:node decorators.js
'use strict';

// 1. 基础工具(覆盖:睡眠、符号标记)
function sleep(ms) { return new Promise((r) => setTimeout(r, ms)); }
const WRAPPED = Symbol('wrapped');

// 2. withLog:记录参数与返回值(覆盖:rest、apply 保 this)
function withLog(fn, log = console.log) {
  return function (...args) {
    log(`→ ${fn.name}(${args.join(', ')}))`);
    const result = fn.apply(this, args);
    log(`← 返回 ${JSON.stringify(result)}`);
    return result;
  };
}

// 3. withTiming:计时 + 阈值告警 + 统计(覆盖:前置/后置、finally)
function withTiming(fn, { thresholdMs = 0, stats = null } = {}) {
  return function (...args) {
    const start = process.hrtime.bigint();
    try {
      return fn.apply(this, args);
    } finally {
      const ms = Number(process.hrtime.bigint() - start) / 1e6;
      if (stats) { stats.calls = (stats.calls ?? 0) + 1; stats.totalMs += ms; }
      if (ms > thresholdMs) {
        console.log(`[计时] ${fn.name} 耗时 ${ms.toFixed(2)}ms(阈值 ${thresholdMs}ms)`);
      }
    }
  };
}

// 4. withRetry:异步重试 + 退避 + onError(覆盖:async、错误传播)
function withRetry(fn, { retries = 3, delayMs = 30, onError = null } = {}) {
  return async function (...args) {
    let lastErr;
    for (let attempt = 1; attempt <= retries; attempt++) {
      try {
        return await fn.apply(this, args);
      } catch (err) {
        lastErr = err;
        onError?.(err, attempt);
        if (attempt < retries) await sleep(delayMs * attempt);
      }
    }
    throw lastErr;
  };
}

// 5. withCache:缓存 + 淘汰 + 统计(覆盖:Map、闭包、纯函数)
function withCache(fn, { maxSize = 50 } = {}) {
  const cache = new Map();
  const stats = { hits: 0, misses: 0 };
  const wrapped = function (...args) {
```

```

    const key = JSON.stringify(args);
    if (cache.has(key)) {
        stats.hits++;
        return cache.get(key);
    }
    stats.misses++;
    const result = fn.apply(this, args);
    if (cache.size >= maxSize) cache.delete(cache.keys().next().value);
    cache.set(key, result);
    return result;
};
wrapped.stats = stats;           // 统计挂在函数上
return wrapped;
}

// 6. compose:从右到左组合(覆盖:高阶函数、reduce)
function compose(...decorators) {
    return (fn) => decorators.reduceRight((f, d) => d(f), fn);
}

// 7. 幂等装饰(覆盖:Symbol 标记)
function onceDecorated(decorator) {
    return (fn) => {
        if (fn[WRAPPED]) return fn;
        const wrapped = decorator(fn);
        wrapped[WRAPPED] = true;
        return wrapped;
    };
}

// 8. 注册表(覆盖:按名组合)
const REGISTRY = new Map();
function register(name, impl) { REGISTRY.set(name, impl); }
function applyDecorators(fn, names) {
    return names.reduceRight((f, name) => {
        const impl = REGISTRY.get(name);
        return impl ? impl(f) : f;
    }, fn);
}

// 9. 业务函数:模拟带网络抖动与延迟的接口(覆盖:重试场景构造)
let networkCalls = 0;
async function fetchUserRaw(id) {
    networkCalls++;
    await sleep(10 + Math.random() * 20);
    if (networkCalls % 4 === 0) throw new Error('网络抖动'); // 每 4 次抖一次
    return { id, name: `用户${id}`, at: Date.now() };
}

// 10. 主流程(覆盖:组合使用、统计输出)
async function main() {
    const stats = { totalMs: 0 };

    // 组合:缓存 → 重试 → 计时(从内到外:先缓存,再重试,最外计时)
    const fetchUser = compose(

```

```

    withTiming,
    (f) => withRetry(f, { retries: 3, onError: (e, n) => console.log(`重试 ${n}:${e.message}`) }),
    withCache,
    onceDecorated, // 幂等保护(演示)
  )(fetchUserRaw);
// withTiming 需要 stats 参数 → 单独绑定
const wrapped = withTiming(fetchUserRaw, { stats });
void wrapped;

// 三次调用:第一次缓存 miss,第二次命中,第三次命中
const r1 = await fetchUser(1);
const r2 = await fetchUser(1);
console.log('两次结果相同(缓存):', r1.id === r2.id);

// 验证重试路径:连续调用触发第 4 次抖动
for (let i = 0; i < 3; i++) await fetchUser(i + 10);
await fetchUser(99).then(
  (r) => console.log('抖动后重试成功:', r.id),
  (e) => console.log('重试耗尽:', e.message)
);

// 统计输出
console.log('网络真实调用次数:', networkCalls);
console.log('缓存统计:', JSON.stringify(fetchUser.stats ?? '未导出'));

// 注册表使用
register('log', withLog);
const logged = applyDecorators((x) => x * 2, ['log']);
console.log('注册表组合结果:', logged(21));
}

main().catch((e) => console.error('主流程异常:', e));

```

设计说明:compose 把四个装饰器按"缓存最内、重试中层、计时最外"组合——缓存命中时根本不会走到重试;networkCalls 每 4 次触发一次抖动,演示重试路径与 onError 钩子;缓存统计挂在包装函数上导出;onceDecorated 演示幂等防重复。注释里补充:TS 装饰器实现与函数式装饰是同一思想的两种语法,TC39 提案落地前生产代码用函数式。运行后观察"网络真实调用次数"小于总调用次数(缓存生效)、抖动被重试吸收的输出。

下一章预告:第13章 模块系统 —— CJS 与 ESM 全解、require/import 互操作坑、package.json 与 npm,把"头文件时代"彻底翻篇。

第13章 模块系统:CJS、ESM 与包管理

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 C `#include` 的三个代价:文本复制、宏污染、链接期才报错
- 掌握 CJS 模块:`require` 的同步语义与模块缓存机制
- 掌握 ESM 模块:`import/export` 的静态语义、顶层 `await` 与 `import.meta`
- 掌握 `__dirname` 在两种模块下的差异与 `import.meta.url` 方案
- 掌握 CJS/ESM 互操作:`createRequire`、默认导出 `interop`、双包危害
- 掌握 `package.json` 的 `main/module/exports/type` 四个字段的职责
- 掌握 npm 工作流:`install`、`package-lock.json`、语义化版本

💡 从 C 到 JavaScript:本章主题如何迁移

C 里"把代码拼起来"靠预处理: `#include` 把头文件内容逐字复制进源文件。这套机制有众所周知的痛点:每个编译单元都背着整份头文件、宏污染命名空间、重复包含要靠 `#pragma once` 手动防御,而"符号找不到"要拖到链接期才报错。JavaScript 的模块系统走完了完整的三段进化: `#include` 是文本复制 → CJS 的 `require` 是"运行一次并缓存" → ESM 的 `import` 是"静态链接"。

CJS(CommonJS)诞生于 Node.js 早期:一个文件就是一个模块, `require()` 同步读文件、执行、把导出对象缓存起来,第二次 `require` 直接命中缓存。ESM(ECMAScript Modules)是现代标准: `import` 语句在解析阶段就被静态分析,名字、依赖图全部确定,因此编辑器能给出提示、打包器能做 tree-shaking(摇树优化,把没用的导出删掉)。这就像 C 的"声明与定义分离"升级成了"编译器真的认识你的接口"。

对 C 程序员来说本章最值钱的知识是两件事:一是 `__dirname` 在 ESM 里不存在,必须用 `import.meta.url` 自己换算——这是从 CJS 迁移到 ESM 的第一个坑;二是 `package.json` 的 `type/main/exports` 字段决定了 .js 文件被解析成哪种模块——扩展名不是唯一的判据。第13章的实验是一个"数学库 + 命令行工具"的小项目,你会同时用上 ESM 导出、CJS 兼容层和 npm 依赖,完成从"写一个 .c 文件"到"组织一个 Node 包"的完整过渡。

📦 前置知识自查

- C 预处理:`#include`、`#pragma once`、头文件与源文件分离
- 第2章解构赋值(`import` 语句中常见);第3章作用域与闭包
- 第4章函数一等公民;第7章 `process.argv` 与命令行参数

🚀 本章学习路线

1. **第一阶段(C 的基线)**:读 §13.1,回顾 `#include` 的三个代价
2. **第二阶段(CJS)**:读 §13.2,把 `require/module.exports`/缓存机制过一遍
3. **第三阶段(ESM)**:读 §13.3-§13.4,重点对比差异表与 `__dirname` 方案
4. **第四阶段(工程化)**:读 §13.5-§13.7,理解互操作、`package.json` 与 npm
5. **验收标准**:能独立搭出一个"ESM 库 + CLI 入口 + CJS 兼容"的小包,并解释每个字段的作用

本章知识导图

- 第13章 模块系统:CJS、ESM 与包管理
 - §13.1 C 的 #include:文本复制与它的代价
 - §13.2 CJS:require 与 module.exports
 - §13.3 ESM:import 与 export
 - §13.4 ESM 与 CJS 的关键差异
 - §13.5 互操作陷阱:createRequire 与双包危害
 - §13.6 package.json:模块的身份证
 - §13.7 npm:依赖管理与版本语义

§13.1 C 的 #include:文本复制与它的代价

```
// math.h — 头文件:声明 + 宏,等待被"复制"
#pragma once           // 防重复包含:同一文件只展开一次
int add(int a, int b);
#define SQUARE(x) ((x) * (x)) // 宏:纯文本替换,无类型检查

// main.c — 预处理阶段,math.h 的内容被逐字复制进本文件
#include "math.h"
#include <stdio.h>

int main(void) {
    printf("%d\n", add(2, 3));
    printf("%d\n", SQUARE(3 + 1)); // 展开为 (3+1)*(3+1),结果是 16
    return 0;
}
```

C 的拼接发生在三个不同的阶段,这是它的核心特征:

- **预处理期(文本复制):**#include 只是"把文件内容贴进来"。每个 .c 编译单元各背一份头文件,头文件越大编译越慢;不写 `#pragma once` 就可能重复定义。
- **编译期(翻译单元内):**宏在此时展开。 `SQUARE(x)` 没有类型、没有作用域,还容易出括号事故—— `SQUARE(3+1)` 展开成 `(3+1)*(3+1)`。
- **链接期(符号拼接):**函数声明可以编译,但"函数到底在哪"要到链接期才知道。 `undefined reference` 是链接错误,不是编译错误——错误发现得越晚,修起来越贵。

⚠ 常见误区 13.1:认为 #include 是"接口"而不是"文本"

头文件里任何一行宏、typedef 都可能污染所有包含它的文件,且包含顺序敏感:A 头里定义了 B 头依赖的宏,B 才能展开成功。这就是为什么 C 项目有"头文件洁癖"文化。JS 的模块没有这个问题:每个模块是独立作用域,导入的名字必须显式声明,顺序无关。

§13.2 CJS:require 与 module.exports

```
// lib/math.js — CJS 模块:一个文件就是一个模块,导出对象决定"外可见面"
'use strict';

function add(a, b) { return a + b; }
function mul(a, b) { return a * b; }
const VERSION = '1.0.0';

// 方式一:整体赋值 module.exports(最常用,一个模块导出一个"包")
module.exports = { add, mul, VERSION };

// 方式二:给 exports 挂属性 — 等价,因为 exports 一开始就是
// module.exports 的引用;但注意:不能对 exports 整体重赋值!
// exports.add = add; 这样写也有效
// exports = { add }; 这样写无效(断开了引用,外部拿到的还是老对象)
```

```
// main.cjs — require 的语义:同步读取 + 执行一次 + 结果缓存
const math = require('./lib/math.js');

console.log(math.add(2, 3));      // 5
console.log(math.VERSION);       // '1.0.0'

// 缓存:第二次 require 不会重新执行模块代码
const again = require('./lib/math.js');
console.log(again === math);     // true — 同一个对象,模块只执行一次

// require.cache 里能看到所有已加载模块(生产调试偶尔有用)
console.log(Object.keys(require.cache).length >= 1); // true
```

把 CJS 与 C 对照看: `#include` 是"复制", `require` 是"执行 + 缓存"。

- **加载即执行**: `require` 同步地读取文件、执行模块顶层代码、返回 `module.exports`。模块顶层代码只会执行一次(首次 `require`),之后都命中缓存。
- **作用域隔离**: 模块内声明的变量默认不对外可见,只有挂到 `module.exports` 上的才可见——比头文件干净得多,没有宏污染问题。
- **__dirname/ __filename**: CJS 模块里直接可用,指向当前文件目录与完整路径(对应 C 里"程序自己在哪"——C 没有标准答案,只有 `argv[0]` 的近似)。
- **循环依赖**: `require` 遇到循环时会返回"对方当前已完成的部分导出"——这也是为什么生产建议"尽量让依赖图无环"。

⚠ 常见误区 13.2:exports 整体重赋值

`exports = { add }` 不会生效: `exports` 只是 `module.exports` 的别名,重赋值只是把局部变量换个指向, `module.exports` 还是原来的空对象。规则: 要么一直用 `module.exports`, 要么一直用 `exports.xxx`, 不要混着用,更不要整体重赋值。

§13.3 ESM:import 与 export

```
// lib/math.mjs — ESM 模块:导出声明(语法级别,静态可分析)
export function add(a, b) { return a + b; }
export function mul(a, b) { return a * b; }
export const VERSION = '1.0.0';           // 命名导出:可以导出任何绑定
export default { add, mul };              // 默认导出:一个模块至多一个
```

```
// main.mjs — 导入端
import math, { add, mul, VERSION } from './lib/math.mjs';
//      ↑默认导出      ↑命名导出(名字必须精确匹配)

console.log(add(1, 2));           // 3
console.log(math.mul(3, 4));      // 12 — 默认导出是一个对象

// 重命名:避免命名冲突
import { VERSION as VER } from './lib/math.mjs';
console.log(VER);                 // '1.0.0'
```

ESM 与 CJS 的本质差异是 **静态分析**:import/export 必须出现在模块顶层(不能写在 if 里),依赖关系在解析阶段就完全确定。这带来三个工程红利:

- **编辑器提示**:导入导出名字错了立刻标红,像 C 的“编译期检查”提前到了写代码时。
- **tree-shaking**:打包器可以精确删除没被 import 的导出(比如只用了 add,没用的 mul 不进 bundle)。
- **顶层 await**:ESM 允许在模块顶层写 await(加载依赖时等一等),CJS 不行。

```
// config.mjs — 顶层 await:模块加载阶段异步初始化
const resp = await fetch('https://example.com/api/config');
export const config = await resp.json();
// 别的模块 import config 时,已经拿到了完整配置
```

⚠ 常见误区 13.3:import 里想用变量拼路径

import path from './' + name + '.mjs' 是语法错误——import 必须静态可分析,路径不能动态拼接。动态加载要用 import() 函数(见 §13.5),它返回 Promise。

§13.4 ESM 与 CJS 的关键差异

差异点	CJS	ESM
加载方式	require 同步执行	import 异步加载(顶层 await 可用)
__dirname/ __filename	直接可用	不存在,用 import.meta.url 换算
顶层 this	module.exports	undefined
严格模式	非默认(需 'use strict')	默认严格
语法位置	require 可写在任意位置	import 必须顶层;动态用 import()
扩展名判据	.cjs 或 type=commonjs	.mjs 或 type=module

最大的迁移坑是 `__dirname` 的消失:

```
// main.mjs — ESM 里没有 __dirname,需要自己从 import.meta.url 换算
import { fileURLToPath } from 'node:url';
import path from 'node:path';

console.log(import.meta.url);
// file:///C:/app/main.mjs — 这是"模块自身的 URL"

const __dirname = path.dirname(fileURLToPath(import.meta.url));
console.log(__dirname);
// C:\app — 与 CJS 的 __dirname 对齐

// 读同目录文件的老姿势在 ESM 里会踩坑:
// fs.readFileSync('./data.txt') 是相对"进程工作目录"的,不是模块目录
// 正确姿势:path.join(__dirname, 'data.txt')
```

⚠ 常见误区 13.4:ESM 里直接写 `__dirname`

在 `.mjs`(或 `type=module` 的 `.js`)里写 `__dirname` 会直接 `ReferenceError` ——它不是关键字,只是 CJS 注入的变量。生产包常用一个小工具函数封装换算逻辑,或直接用 `import.meta.dirname` (Node 20.11+ 提供,等价封装)。

§13.5 互操作陷阱:createRequire 与双包危害

```
// tool.mjs — ESM 里要用 require?用 createRequire 造一个
import { createRequire } from 'node:module';

const require = createRequire(import.meta.url);
// 以"当前模块"为基准的 require:内建模块、CJS 包都能加载

const path = require('node:path');
const fs = require('node:fs');
console.log(fs.readFileSync('data.txt', 'utf8'));
```

互操作三条铁律(生产面试高频):

- **ESM import CJS:**可以。CJS 的 `module.exports` 会整体作为 default 导出;命名导出由静态分析尽力推断(同步赋值才猜得到),猜不到时只能 `import pkg from 'pkg'` 再解构——所以 CJS 包的文档往往建议 `const { f } = require('pkg')` 风格。
- **CJS require ESM:**不行。require 是同步的,而 ESM 的加载是异步的,require 会直接抛 `ERR_REQUIRE_ESM`。替代:在 CJS 里用动态 `import('pkg.mjs')` (返回 Promise)。
- **双包危害:**同一个包被 CJS 与 ESM 各加载一份,内存里出现两份模块实例——两份状态、两套类, `instanceof` 全部失效。根治靠 `package.json` 的 `exports` 条件导出(§13.6),让包只以一种模块形式暴露。

```
// legacy.cjs — 模拟一个"老旧第三方 CJS 依赖"
module.exports = { greet: () => 'legacy hello' };

// main.mjs — CJS 里没有 require 时,用动态 import 加载 ESM;反向同理
const legacy = await import('./legacy.cjs');
console.log(legacy.default.greet()); // 'legacy hello'
// 注意:跨模块类型拿 default — CJS 的 module.exports 整体是 default
```

§13.6 package.json:模块的身份证

```
{
  "name": "calc-tools",
  "version": "1.4.2",
  "description": "命令行计算器工具包",
  "type": "module",
  "main": "./bin/cli.js",
  "exports": {
    ".": "./lib/index.js",
    "./math": "./lib/math.js",
    "./package.json": "./package.json"
  },
  "bin": { "calc": "./bin/cli.js" },
  "scripts": { "start": "node bin/cli.js" },
  "dependencies": { "commander": "^12.0.0" },
  "devDependencies": { "eslint": "^9.0.0" }
}
```

字段	作用	说明
type	决定 .js 按哪种模块解析	"module" = ESM;"commonjs"(默认) = CJS;.mjs/.cjs 扩展名强制覆盖
main	老式入口(require 与老工具)	向后兼容字段;新包以 exports 为准
exports	现代入口 + 子路径白名单	"." 是主入口,其余是允许外部 import 的子路径;未列出的子路径一律不可导入
bin	命令行工具注册名	npm install 后,注册名直接可用(如 calc)
scripts	任务脚本	npm start / npm test 等,可复用本机 PATH

exports 还能做条件导出,一举解决双包危害——同一个包,让 CJS 消费者拿到 CJS 副本、ESM 消费者拿到 ESM 副本:

```
{
  "exports": {
    ".": {
      "import": "./lib/index.mjs", // ESM 消费者走这里
      "require": "./lib/index.cjs" // CJS 消费者走这里
    }
  }
}
```

§13.7 npm:依赖管理与版本语义

```
npm init -y          # 生成 package.json(对话式初始化)
npm install lodash    # 安装并写入 dependencies(运行时依赖)
npm install -D eslint # -D = 写入 devDependencies(仅开发期依赖)
npm ci               # 严格按 package-lock.json 安装,CI 环境必用
npm outdated         # 查看哪些依赖有新版
```

- **node_modules**: 依赖实际落盘目录, 体积巨大, 永不提交版本库, 由 package.json + lock 文件重建。
- **package-lock.json**: 把"版本范围"钉死为"确切版本 + 完整依赖树"。有了它, 任何人在任何机器 `npm ci` 得到完全相同的依赖——这是"我这边能跑"问题的标准解药。
- **语义化版本(SemVer)**: 主.次.补丁。`^1.4.2` 允许 1.x 内更新(不跨主版本); `~1.4.2` 只允许补丁更新; `1.4.2` 精确锁定。主版本 0 意味着不稳定。
- **打包器一句话**: webpack/vite/rollup 把多个 ESM 源文件(含 node_modules 里的依赖)分析、摇树、合并成浏览器可直接运行的单个 bundle 文件——浏览器本身没有 Node 的模块加载器, 打包器就是"预链接器"。

⚠ 常见误区 13.5: 不提交 package-lock.json

锁文件不提交, 团队里就会出现"我这边能跑、你那边报错"——因为 `^1.4.2` 允许各自装到不同版本。规则: 应用项目必须提交 lock 文件并用 `npm ci`; 库项目才可以选择性忽略。

本章速查表

主题	速查要点
C #include	预处理期文本复制; 顺序敏感; 宏无类型; 错误拖到链接期
require	CJS 同步加载; 模块顶层代码只执行一次; 结果缓存(require.cache)
module.exports	CJS 导出对象; exports 是它的别名, 整体重赋值会断开
__dirname(CJS)	当前文件目录, 直接可用
import/export	ESM 静态分析; 必须顶层; 命名导出 + 至多一个默认导出
import.meta.url	ESM 模块自身 URL; 配合 fileURLToPath 还原 __dirname
顶层 await	仅 ESM; CJS 不支持
createRequire	ESM 里造 require: createRequire(import.meta.url)
ESM 导入 CJS	default = module.exports; 命名导出靠静态推断, 不可靠
CJS require ESM	不允许(同步 vs 异步); 改用动态 import()
双包危害	同一包被两种模块各加载一份; exports 条件导出根治
扩展名/type	.cjs/.mjs 强制; js 看 type 字段(module/commonjs)
exports 字段	现代入口 + 子路径白名单 + import/require 条件分支
语义化版本	^1.4.2 主版本内浮动; ~1.4.2 仅补丁; lock 文件钉死
npm ci	严格按 lock 安装; CI 必用; 应用项目必须提交 lock

最小必会清单

- 能说出 `#include`、`require`、`import` 三者的本质差异(复制/执行缓存/静态链接)
- 能解释 `require` 的缓存语义与循环依赖的部分导出行为
- 能解释 `exports` 整体重赋值为什么无效
- 能说出 ESM 的三个静态分析红利(提示/tree-shaking/顶层 `await`)
- 能在 ESM 里还原 `__dirname(fileURLToPath + import.meta.url)`
- 能说出 CJS/ESM 互操作的三个规则与双包危害
- 能解释 `type/main/exports` 三个字段的职责与优先级
- 能写出带条件导出的 `exports` 配置解决双包危害
- 能解释 `^` 与 `~` 的区别、`lock` 文件的作用、`npm ci` 的适用场景
- 能一句话解释打包器在浏览器场景下的角色

自测题

Q 1. C 的 `#include` 与 Node 的 `require` 本质区别是什么?

✓ 参考答案

`#include` 是预处理期的文本复制(每次编译各自展开,无缓存);`require` 是运行期的"执行一次 + 缓存结果",模块代码只跑一遍,第二次 `require` 直接返回缓存对象。

Q 2. 为什么 `exports = { add }` 无效,而 `module.exports = { add }` 有效?

✓ 参考答案

`exports` 只是 `module.exports` 的引用别名;整体重赋值只是让局部变量指向新对象,`module.exports` 本身不变,外部拿到的仍是原对象。必须直接操作 `module.exports`。

Q 3. 在 `.mjs` 文件里写 `__dirname` 会发生什么?正确做法是什么?

✓ 参考答案

`ReferenceError: __dirname 未定义`——它只是 CJS 注入的变量,不是语言关键字。正确做法:`fileURLToPath(import.meta.url) + path.dirname` 换算,或使用 `import.meta.dirname(Node 20.11+)`。

Q 4. 一个 CJS 包被"ESM 消费者"和"CJS 消费者"同时加载,会发生什么?如何根治?

✓ 参考答案

双包危害:两份模块实例、两套状态与类,`instanceof` 失效。根治:`package.json exports` 配置条件导出(`import/require` 分支),让每个消费者只拿到一种形态。

Q 5. `package.json` 里 `"type": "module"` 时,一个 `.cjs` 文件按哪种模块解析?为什么?

✅ 参考答案

按 CJS。扩展名优先于 type 字段: .cjs 强制 CJS、.mjs 强制 ESM, type 字段只决定"无扩展名标记的 .js"的解析方式。

Q 6. 团队里出现"我这边能跑,你那边报错",最可能的原因与标准解法是什么?

✅ 参考答案

依赖版本漂移: ^ 允许范围浮动,各人装到不同版本。标准解法:提交 package-lock.json,统一用 npm ci 按锁文件精确安装。

章末实验题:数学库 + CLI 工具包

📄 实验 13:数学库 + CLI 工具包(ESM/CJS 双模)

实验目标:搭建一个"库 + 命令行"双入口小包,覆盖 ESM 导出、CJS 兼容、package.json 配置与互操作。

实验要求:

- 任务 1(覆盖:ESM 命名导出):lib/math.js 导出 sum/avg/max/min 四个纯函数,支持任意个数的数值参数
- 任务 2(覆盖:CLI 入口、process.argv):bin/cli.js 支持 `node cli.js sum 1 2 3` 与 `node cli.js avg 4 5 6`, 参数不足时打印用法并退出码非 0
- 任务 3(覆盖:createRequire/互操作):cli.js 里用 createRequire 加载一个 CJS 兼容层 legacy.cjs(module.exports 导出版本号),并在输出中展示
- 任务 4(覆盖:package.json):配置 type=module、exports(".")与"./math"子路径)、bin 与 scripts
- 任务 5(覆盖:缓存语义):注释说明"为什么 cli.js 里 import 两次 math.js 只执行一次"
- 任务 6(覆盖:语义化版本):dependencies 里用 ^ 版本装一个内建无关的第三方包,注释说明版本范围语义(不必真实安装)

实验环境:Node.js 20+,运行 `node bin/cli.js sum 1 2 3`。

第一步 写纯函数库:lib/math.js 只做导出,不做任何 I/O——库与入口分离是模块设计第一课

第二步 写 CLI:解析 process.argv.slice(2),按子命令分发,统一处理"参数不是数字"的错误

第三步 后接互操作与配置:补 legacy.cjs 与 package.json,验证两种加载路径都能工作

✅ 参考答案要点

核心:纯函数库 + 入口分发 + createRequire 互操作 + package.json 双入口。约 120 行,覆盖全章模块知识点。

```
// lib/math.js — ESM 数学库(纯函数,无状态、无 I/O)
export function sum(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}
export function avg(...nums) {
  if (nums.length === 0) throw new Error('avg: 参数不能为空');
  return sum(...nums) / nums.length;
}
export function max(...nums) {
  if (nums.length === 0) throw new Error('max: 参数不能为空');
  return Math.max(...nums);
}
export function min(...nums) {
  if (nums.length === 0) throw new Error('min: 参数不能为空');
  return Math.min(...nums);
}
```

```
// legacy.cjs — CJS 兼容层(模拟"老旧第三方依赖")
// 任务 3:ESM 里用 createRequire 加载它,验证 CJS 的导出形态
module.exports = {
  libName: 'calc-tools',
  version: '1.0.0',
  loadedBy: 'createRequire(cjs)',
};
```

```

// bin/cli.js — CLI 入口(ESM)
import { createRequire } from 'node:module';
import { sum, avg, max, min } from '../lib/math.js';

const require = createRequire(import.meta.url);
const legacy = require('../legacy.cjs'); // 互操作:CJS 模块整体是 default

function usage() {
  console.log('用法: node cli.js <sum|avg|max|min> 数字...');
  console.log('示例: node cli.js sum 1 2 3 → 6');
  process.exitCode = 1;
}

function parseNums(list) {
  const nums = list.map(Number);
  if (nums.some((n) => Number.isNaN(n))) {
    throw new Error('存在无法解析的数字: ' + list.join(' '));
  }
  return nums;
}

function main() {
  console.log('== ' + legacy.libName + ' v' + legacy.version + ' ==');

  const [cmd, ...rest] = process.argv.slice(2);
  if (!cmd || rest.length === 0) return usage();

  try {
    const nums = parseNums(rest);
    switch (cmd) {
      case 'sum': console.log('sum =', sum(...nums)); break;
      case 'avg': console.log('avg =', avg(...nums)); break;
      case 'max': console.log('max =', max(...nums)); break;
      case 'min': console.log('min =', min(...nums)); break;
      default: usage();
    }
  } catch (err) {
    console.error('错误: ' + err.message);
    process.exitCode = 1;
  }
}

// 任务 5:下面这行执行后,math.js 的顶层代码只跑一次(缓存语义);
// 再 import 一次拿到的还是同一份导出,不会重复执行模块代码。
main();

```

```
// package.json — 任务 4/6:双入口 + 子路径 + 版本语义
{
  "name": "calc-tools",
  "version": "1.0.0",
  "type": "module",
  "main": "./bin/cli.js",
  "exports": {
    ".": "./lib/math.js",
    "./math": "./lib/math.js"
  },
  "bin": { "calc": "./bin/cli.js" },
  "scripts": { "start": "node bin/cli.js sum 1 2 3" },
  "dependencies": { "commander": "^12.0.0" }
}
// ^12.0.0:主版本 12 内可浮动更新(12.x);~12.0.0 则只允许补丁(12.0.x)
```

设计说明:sum 用 rest 参数收集任意个数;parseNums 把 argv 里的字符串统一转数字并显式报错——模拟真实 CLI 的输入校验;legacy.cjs 演示 CJS 与 ESM 双模共存的场景,呼应 §13.5 互操作规则。运行 `node bin/cli.js sum 1 2 3` 输出 6;运行 `node bin/cli.js` 打印用法并以退出码 1 结束。

下一章预告:第14章 类型系统进阶 —— 结构类型与鸭子类型、JSON 边界、深浅拷贝、空安全,以及 TypeScript 与 JSDoc 渐进类型的方向。

第14章 类型系统进阶:结构类型、TS 与领域建模

建议学时:3-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解结构类型(鸭子类型)与 C 标称类型的本质差异
- 掌握运行时类型检查的三层防线:typeof / instanceof / shape 校验
- 掌握判别联合(tag 字段 + switch)的建模方法
- 掌握 JSON 作为"类型边界"的六种类型与四个转换坑
- 掌握深浅拷贝的边界:展开符、structuredClone 与手写递归
- 掌握空安全:?? 与 ?. 的组合规则
- 理解 TypeScript 与 JSDoc 渐进类型的方向,形成生产判断

💡 从 C 到 JavaScript:本章主题如何迁移

C 是标称类型(nominal typing): `struct Point` 与 `struct Circle` 就算布局一模一样也是两种类型,传参要显式强转;类型检查发生在编译期,运行期没有"类型"这个实体。JS 是结构类型(structural typing): 运行时只看对象的形状——"凡是长着 Point 样子的对象都能当 Point 用"。这把 C 里"编译期说了算"的权力移交给了运行时,代价是错误发现变晚,收益是极大的灵活性与可组合性(鸭子类型、动态 JSON、无模板的数据管道)。

类型不是"有或没有"的问题,而是三道防线如何分工:typeof 挡原始值,instanceof 挡类实例,手写 shape 校验挡"形状不符"的结构体。再加上判别联合(给对象一个 status 标签字段,用 switch 精确分支),C 程序员用 union + enum 才能表达的"一种数据多种形态",在 JS 里用普通对象就能优雅建模。

生产环境的真实图景是"渐进类型":核心边界(输入/输出/配置文件)要么用运行时校验兜底,要么引入 TypeScript 或 JSDoc 注解获得编译期检查;业务内部保持动态自由。第14章的实验(订单 + 商品 + 用户的领域模型)把本章所有技术串成一套真实系统:你会看到"校验函数 + 判别联合 + JSON 往返 + 深拷贝"如何组装出一个可运行、可维护的订单引擎——这是从"会写 C 程序"到"会设计 JS 系统"的最后一跃。

📦 前置知识自查

- C 的 struct/union/enum、强转、NULL 指针检查
- 第1章八种类型与 typeof;第6章原型链与 instanceof
- 第7章 JSON 序列化基础;第11章反射与 Object 反射全家

🚀 本章学习路线

1. **第一阶段(类型观)**:读 §14.1-§14.2,建立"结构类型 + 三层防线"的心智模型
2. **第二阶段(边界)**:读 §14.3-§14.5,吃透 JSON 转换坑、深浅拷贝与空安全
3. **第三阶段(工具)**:读 §14.6,理解 TS 与 JSDoc 渐进类型的定位
4. **第四阶段(建模)**:读 §14.7 并完成章末领域模型实验
5. **验收标准**:能用"校验函数 + 判别联合 + JSON 往返"独立建模一个带状态的业务实体

本章知识导图

- 第14章 类型系统进阶:结构类型、TS 与领域建模
 - §14.1 C 的标称类型 vs JS 的结构类型
 - §14.2 运行时类型检查的三层防线
 - §14.3 JSON:类型系统的外部边界
 - §14.4 深浅拷贝:边界与工具
 - §14.5 空安全:?? 与 ?. 的组合规则
 - §14.6 TypeScript 与 JSDoc:渐进类型的方向
 - §14.7 生产建议:运行时校验与领域建模

§14.1 C 的标称类型 vs JS 的结构类型

```
// C:标称类型 — 类型名即类型,布局相同也互不兼容
struct Point { int x, y; };
struct Circle { int x, y, r; }; // 布局包含 Point,但不是 Point

void draw(struct Point p) { /* ... */ }

struct Circle c = {1, 2, 5};
// draw(c); // 编译警告:不兼容的类型
draw((struct Point)c); // 强转:程序员负责,编译器闭嘴
```

```
// JS:结构类型 — 只看形状,不认名字
function draw(p) {
  // 不检查 p 是什么"类",只要求它有 x 和 y
  console.log('draw at (' + p.x + ', ' + p.y + ')');
}

draw({ x: 1, y: 2 }); // 普通对象,OK
draw({ x: 3, y: 4, r: 5 }); // 多一个 r,OK — 形状满足即可
draw({ x: 1 }); // 缺 y:运行时不报错,打印 (1, undefined)
```

结构类型的本质是 **"接口隐含于形状"**:调用方与被调方之间没有显式契约,契约就是"你需要哪些字段"。这带来两面的结果:

- **好的一面**:松耦合。测试时传一个假对象(测试替身)不需要继承任何类;函数天然多态——任何"长得像"的输入都能用。第6章的鸭子类型在这里有了理论依据。
- **代价**:拼错字段名不会报错,只会运行时产生 undefined 与神秘 bug。C 编译器替你把关,现在要自己把——这就是 §14.2 三层防线存在的理由。

⚠ 常见误区 14.1:字段名拼错是"小事"

`user.name` 拼成 `user.nane`:不报错,拿到 undefined,后续拼接字符串变成 "undefined",存进数据库才发现。C 里这是编译错误,JS 里是"迟到的数据事故"。对策:关键字段在入口处做 shape 校验(§14.2),或引入 TS/JSDoc(§14.6)。

§14.2 运行时类型检查的三层防线

三层防线各管一段,生产代码按"成本从低到高"使用:

```
// 第一层:typeof — 管原始值(成本最低)
function toStr(x) {
  if (typeof x === 'string') return x;           // 已经是字符串
  if (typeof x === 'number') return String(x);   // 数字转字符串
  return '';                                     // 其余给空串
}

// 第二层:instanceof — 管类实例
class Account { /* ... */ }
function deposit(a, amount) {
  if (!(a instanceof Account)) throw new TypeError('需要 Account 实例');
  // 跨 realm 场景(第11章)注意 instanceof 会失效,用品牌字段替代
}

// 第三层:shape 校验 — 管"结构体形状"(成本最高,用在边界)
function isProduct(obj) {
  return (
    obj !== null &&
    typeof obj === 'object' &&
    typeof obj.id === 'string' &&           // 字段存在且类型正确
    typeof obj.name === 'string' &&
    typeof obj.price === 'number' &&
    Number.isFinite(obj.price) &&           // 拒绝 NaN/Infinity
    obj.price >= 0                           // 值域约束
  );
}
const raw = JSON.parse(await fetch('/api/products/1').then((r) => r.json()));
if (!isProduct(raw)) throw new Error('商品数据格式错误: ' + JSON.stringify(raw));
```

判别联合:当同一份数据有多种形态时(C 的 union + enum 组合),用一个"标签字段"区分:

```
// 订单三态:tag 字段 status + switch 精确分支
function formatOrder(order) {
  switch (order.status) {
    case 'pending': return '待支付: ' + order.id;
    case 'paid':     return '已支付: ' + order.id + ', 金额 ¥' + order.total;
    case 'cancelled': return '已取消: ' + order.id + ' (原因: ' + order.reason + ')';
    default:
      // 未知分支必须抛错:宁可让系统亮红灯,也不要静默返回错误数据
      throw new Error('未知订单状态: ' + order.status);
  }
}
```

判别联合三原则: 标签字段放显眼位置(status)、default 分支必抛错、每个分支只访问自己该有的字段(cancelled 才有 reason)。这正是 C 里 enum + union + switch 的运行时版,且更安全——没有"谁都能乱读 union"的问题。

§14.3 JSON:类型系统的外部边界

JSON 只有六种类型:null、boolean、number、string、array、object。JS 的世界比它宽,跨边界时会发生隐式转换:

```
const order = {
  id: 1,
  createdAt: new Date(),          // Date 不在 JSON 类型里
  note: undefined,               // undefined 没有 JSON 表示
  score: NaN,                    // NaN 不是 JSON number
  big: 9007199254740993n,        // BigInt 不支持序列化
};

const json = JSON.stringify(order);
console.log(json);
// {"id":1,"createdAt":"2026-08-26T08:00:00.000Z","score":null}
// Date → ISO 字符串;undefined 属性整个被丢弃;NaN → null;BigInt 直接抛错
```

- **Date → 字符串**:序列化后是 ISO 字符串,反序列化回来仍是字符串,不是 Date。JSON 往返改变类型——领域模型里要显式转换(`new Date(obj.createdAt)`)。
- **undefined/NaN**:undefined 属性被丢弃,NaN/Infinity 变成 null。数据静默变形,是“看起来没丢,用起来错了”的经典来源。
- **大整数**:超过 $2^{53} - 1$ 的整数 id(如雪花 ID)精度丢失。生产规则:大 ID 用字符串承载。
- **循环引用**:JSON.stringify 抛 TypeError。深度遍历要先做防环(第11章的 Set 技巧)。

⚠ 常见误区 14.2:JSON 是“无损的”

JSON 只是序列化格式,不是 JS 的镜像。Date/undefined/NaN/BigInt 四个类型跨边界都会变形或报错。规则:跨 JSON 边界前,先把类型显式转换成 JSON 六型之一(日期 → ISO 字符串、大整数 → 字符串、undefined → 省略),反序列化后再显式还原。

§14.4 深浅拷贝:边界与工具

```
const a = { user: { name: '李雷', tags: ['a', 'b'] } };

// 浅拷贝:{...a} 只复制第一层,内层对象仍然共享
const shallow = { ...a };
shallow.user.name = '韩梅梅';
console.log(a.user.name);          // '韩梅梅' ← 原对象被污染了!

// 深拷贝:structuredClone(ES2022 内置,支持 Date/Map/Set/循环引用)
const deep = structuredClone(a);
deep.user.name = '张三';
console.log(a.user.name);          // '韩梅梅' — 原对象不受影响

// 浅拷贝的另一写法(老代码常见)
const old = Object.assign({}, a);
```

深浅拷贝的边界要记牢:

- **{...obj} / Object.assign**:只复制可枚举自身属性,且只复制“引用”本身——内层对象/数组仍是同一份。90% 的“拷贝 bug”都是把它当深拷贝用。
- **structuredClone**:现代深拷贝首选:支持循环引用、Date、Map、Set、TypedArray。限制:函数与类实例(非结构化数据)会抛 DataCloneError,Symbol 键会丢失。

- **手写递归 clone:**教学与特殊场景(过滤字段、转换类型)才需要;必须带 WeakMap 防循环引用:

```
function deepClone(obj, seen = new WeakMap()) {
  if (obj === null || typeof obj !== 'object') return obj; // 原始值原样返回
  if (seen.has(obj)) return seen.get(obj); // 防循环引用
  const clone = Array.isArray(obj) ? [] : {};
  seen.set(obj, clone);
  for (const [k, v] of Object.entries(obj)) {
    clone[k] = deepClone(v, seen); // 递归
  }
  return clone;
}
```

⚠ 常见误区 14.3:改"拷贝"影响了原数据

排查顺序:先确认是不是浅拷贝共享了内层引用。生产规则:默认用 structuredClone;只在"确信内层会被替换而不会被修改"时用浅拷贝(如 React 的不可更新惯例)。

§14.5 空安全:?? 与 ?. 的组合规则

```
// C:NULL 指针解引用 → 段错误,程序当场崩溃
// JS:null/undefined 上访问属性 → TypeError(同理,但可防御)

// 空值合并 ?? :只在 null/undefined 时取默认值
const port = process.env.PORT ?? 8080;
const name = user.name ?? '匿名';
const zero = 0 ?? 100; // 0 — 空值合并不吞 0
const empty = '' ?? '默认'; // '' — 空值合并不吞空串

// 对比 || :把 falsy 值一视同仁(0/''/NaN 都会被替换)
const a = 0 || 100; // 100 ← 逻辑或吞掉了 0
const b = 0 ?? 100; // 0 ← 空值合并保留 0

// 可选链 ?. :路径上任何一环为空,整个表达式短路为 undefined
const city = user?.address?.city ?? '未知';
// 等价于:
// const city = (user != null && user.address != null)
//   ? (user.address.city ?? '未知')
//   : '未知';

// 组合调用:函数可能不存在时
const result = config?.parser?.call(config, raw) ?? null;
```

空安全的黄金组合:?. 负责"路",?? 负责"默认值",两者总是成对出现。对照 C:JS 的空访问不会崩溃进程,但会静默传播 undefined——所以"默认值"必须显式给出,否则 undefined 会一路流到 UI 层变成 "undefined" 字样。

表达式	x = 0	x = ''	x = null	x = undefined
x '默认'	'默认'	'默认'	'默认'	'默认'
x ?? '默认'	0	''	'默认'	'默认'

§14.6 TypeScript 与 JSDoc:渐进类型的方向

```
// TypeScript — 带类型的 JS 超集:编译期检查,编译产物就是 JS
interface Product {
  id: string;
  price: number;
}

function total(products: Product[]): number {
  return products.reduce((s, p) => s + p.price, 0);
}

total([{ id: 'a', price: 9.9 }]); // 编译通过
// total([{ id: 'a' }]); // 编译错误:缺少 price — 错误在写代码时发现
```

TS 对 C 程序员非常友好:它把"标称感"带回来了——interface 声明形状,函数签名约束参数,编辑器即时报错,像 C 的编译期检查提前到输入阶段。但它不改变运行行为:类型在编译时被擦除,运行时仍是动态 JS——所以 TS 检查通过不意味着数据合法(网络 JSON 依然要校验)。

联合类型是 TS 对判别联合的静态化:

```
type OrderStatus = 'pending' | 'paid' | 'cancelled';

interface Order {
  id: string;
  status: OrderStatus;
  total: number;
}

// status 写错成 'penging' 会在编译期被拦下 — 这就是 §14.2 判别联合的静态版
```

JSDoc 渐进类型:不想引入构建步骤时,用 JSDoc 注释获得同样的编辑器检查(零依赖,VS Code 原生支持):

```
// @ts-check ← 文件顶部的注释:开启本文件类型检查
/** @param {{ id: string, price: number }[]} products */
function total(products) {
  return products.reduce((s, p) => s + p.price, 0);
}

/** @typedef {{ id: string, price: number }} Product */
/** @type {Product[]} */
const list = [];
```

⚠ 常见误区 14.4:TS 检查通过 = 数据安全

TS 的类型在运行时被擦除,JSON.parse 出来的数据它管不着。生产规则:TS/JSDoc 管"代码内部",运行时校验管"系统边界"——网络响应、文件内容、用户输入,一律在入口用 shape 校验(§14.2)把关,再信任内部。

§14.7 生产建议:运行时校验与领域建模

把全章收敛成三条工程原则:

- **原则一(边界校验):**在"不可信数据"进入系统的四个入口做校验——HTTP 请求体、JSON 文件、环境变量/配置、用户输入。校验失败立即抛错,错误信息要带原始数据片段,方便排查。
- **原则二(判别联合优先):**一个实体有多种状态/形态时,用 status 标签 + switch,default 抛错。不要用"一堆可选字段 + 到处 if"——那会让非法状态静默存在。
- **原则三(默认值显式):**所有"可能为空"的读取都用 ?? 给出显式默认;绝不把 undefined 留在数据里流向 UI 或数据库。C 程序员把它想成"每次解引用前先判 NULL"的纪律,只是更优雅。

 **常见误区 14.5:校验函数到处复制粘贴**

isProduct 这种 shape 校验是纯函数、无状态——把它放进专门的 domain/validators 文件,统一导出;校验失败抛出的错误统一包装成带字段名的 Error,方便上层日志聚合。生产包(如 zod)提供声明式 schema,一行顶十行,值得了解。

本章速查表

主题	速查要点
标称类型	C:类型名即类型;布局相同也不兼容;强转是程序员承诺
结构类型	JS:运行时只看形状;缺字段不报错,产生 undefined
三层防线	typeof 管原始值;instanceof 管类实例;shape 校验管形状与值域
判别联合	status 标签 + switch;default 必须抛错;分支只碰自己的字段
JSON 六型	null/boolean/number/string/array/object;其余类型跨界会变形
JSON 转换坑	Date→字符串;undefined 属性丢弃;NaN→null;BigInt 抛错
大整数	超过 2 ⁵³ -1 精度丢失;生产用字符串承载大 ID
浅拷贝	{...a}/Object.assign:只复制第一层,内层共享引用
深拷贝	structuredClone:循环引用/Date/Map/Set 安全;函数不可克隆
手写 clone	递归 + WeakMap 防环;只在特殊场景(过滤/转换)需要
?? 空值合并	只在 null/undefined 时取默认;0 与 "" 保留
? 可选链	路径上任一环为空即短路;与 ?? 成对使用
TypeScript	编译期检查、编译后擦除;interface/联合类型;不替代运行时校验
JSDoc 渐进	// @ts-check + /** @type */ 零依赖获得编辑器检查
生产三原则	边界校验 / 判别联合优先 / 默认值显式

最小必会清单

- 能解释标称类型与结构类型的差异及各自的代价
- 能说出三层防线的分工与使用顺序
- 能写出 isProduct 风格的 shape 校验函数(含值域检查)
- 能用 status + switch 建模多态实体,并坚持 default 抛错

- 能说出 JSON 六种类型与四个转换坑(Date/undefined/NaN/BigInt)
- 能说出 {...a} 与 structuredClone 的边界,并给出排查思路
- 能写出带 WeakMap 防环的手写深拷贝
- 能解释 ?? 与 || 的区别,并正确使用 ?. 与 ?? 组合
- 能说出 TS 与 JSDoc 的定位(编译期/编辑器检查,运行期擦除)
- 能按"边界校验、判别联合、显式默认值"三原则设计一个领域模型

自测题

Q 1. 结构类型与标称类型的本质区别是什么?JS 为此付出的代价是什么?

✓ 参考答案

标称类型看"名字/声明",布局相同也互不兼容;结构类型只看形状,有需要的字段就能用。代价:拼错字段不报编译错,运行时产生 undefined;错误发现变晚,需要自行加强校验。

Q 2. {...obj} 是深拷贝吗?请举一个它"污染原对象"的具体例子。

✓ 参考答案

不是,只复制第一层。例:const b = { ...a }; b.user.name = 'x' 会修改 a.user.name,因为内层 user 是共享引用。要深拷贝用 structuredClone。

Q 3. 写出 0 ?? 5 与 0 || 5 的结果,并解释差异。

✓ 参考答案

0 ?? 5 是 0(空值合并只在 null/undefined 时取默认,0 是合法值被保留);0 || 5 是 5(逻辑或把 0 当 falsy 替换)。配置默认值场景优先用 ??。

Q 4. JSON.stringify({ d: new Date(), u: undefined, n: NaN }) 输出什么?

✓ 参考答案

{"d":"2026-08-26T...(ISO 字符串)","n":null}:Date 变字符串、undefined 属性被丢弃、NaN 变 null。这就是"JSON 往返改变类型"的三个例子。

Q 5. structuredClone 不能克隆什么?遇到循环引用会怎样?

✓ 参考答案

不能克隆函数与类实例(抛 DataCloneError),Symbol 键丢失;循环引用是支持的(内置处理),不需要手动防环——这正是它相对 JSON.stringify 的优势。

Q 6. 判别联合的 default 分支为什么必须抛错而不是返回一个兜底值?

✅ 参考答案

返回兜底值会掩盖非法状态——数据从“脏”变成“静默的错误结果”,后续在业务深处爆炸,极难定位。抛错让错误在边界处立刻暴露,带着上下文日志,符合“失败要早”原则。

章末实验题:订单领域模型

实验 14:订单引擎(商品 + 用户 + 订单领域模型)

实验目标:用“校验函数 + 判别联合 + JSON 往返 + 深拷贝”建模一个可运行的订单系统,覆盖全章知识点。

实验要求:

- 任务 1(覆盖:shape 校验):createProduct/createUser/createOrder 三个工厂,入参校验失败抛带字段名的 Error
- 任务 2(覆盖:判别联合):订单状态 pending/paid/cancelled 用 status 标签建模;describe(order) 按状态输出不同描述,default 抛错
- 任务 3(覆盖:金额与精度):金额用“整数分”存储,展示 priceInCents 计算,避免浮点误差;大 ID 用字符串
- 任务 4(覆盖:JSON 往返):订单序列化保存、反序列化还原;Date 字段显式转换;undefined 属性显式省略
- 任务 5(覆盖:深拷贝):用 structuredClone 复制订单并修改副本,验证原订单不受影响
- 任务 6(覆盖:空安全):读取可能缺失的用户地址,用 ?. + ?? 输出“未知城市”
- 任务 7(覆盖:JSDoc):为每个工厂函数加 /** @param */ 与 /** @returns */ 注解(用 // @ts-check 验证编辑器提示)

实验环境:Node.js 20+,运行 `node order.js`。

第一步 建校验:把 isProduct/isUser 写成独立纯函数,工厂复用它们——边界校验与业务逻辑分离

第二步 建模:订单 = 用户 + 商品数组 + 状态机;金额一律用分(整数),输出时再转元

第三步 后串流程:创建订单 → 支付 → 序列化 → 反序列化 → 深拷贝实验,输出完整演示日志

✅ 参考答案要点

核心:纯校验函数 + status 判别联合 + 分单位金额 + JSON 往返 + structuredClone 深拷贝。约 130 行,覆盖全章知识点。

```

// order.js — Node.js 20+ 运行:node order.js
// @ts-check
'use strict';

// ===== 校验层:纯函数,无副作用,可复用 =====
/** @param {unknown} v @returns {v is { id: string, name: string, priceCents: number }} */
function isProduct(v) {
  return (
    v !== null && typeof v === 'object' &&
    typeof v.id === 'string' && v.id.length > 0 &&
    typeof v.name === 'string' &&
    typeof v.priceCents === 'number' &&
    Number.isInteger(v.priceCents) && v.priceCents >= 0
  );
}

/** @param {unknown} v @returns {v is { id: string, name: string, city: string }} */
function isUser(v) {
  return (
    v !== null && typeof v === 'object' &&
    typeof v.id === 'string' &&
    typeof v.name === 'string' &&
    (v.city === undefined || typeof v.city === 'string')
  );
}

// ===== 工厂层:入参校验,失败立即抛错 =====
/** @param {string} id @param {string} name @param {number} priceCents */
function createProduct(id, name, priceCents) {
  const p = { id, name, priceCents };
  if (!isProduct(p)) throw new Error('商品参数非法: ' + JSON.stringify(p));
  return p;
}

/** @param {string} id @param {string} name @param {string} [city] */
function createUser(id, name, city) {
  const u = { id, name, city };
  if (!isUser(u)) throw new Error('用户参数非法: ' + JSON.stringify(u));
  return u;
}

/** @param {{ id: string, user: object, items: object[] }} args */
function createOrder(args) {
  if (!isUser(args.user)) throw new Error('订单用户非法');
  if (!Array.isArray(args.items) || args.items.some((it) => !isProduct(it))) {
    throw new Error('订单商品列表非法');
  }
  const totalCents = args.items.reduce((s, it) => s + it.priceCents, 0);
  return {
    id: args.id,
    status: 'pending', // 判别联合:初始状态唯一合法值
    user: args.user,
    items: args.items,
    totalCents,
    createdAt: new Date(),
  };
}

```

```

    paidAt: null,                // 显式 null:表示"尚不存在",而非 undefined
  };
}

// ===== 状态机:判别联合的 switch,default 必须抛错 =====
function pay(order) {
  if (order.status !== 'pending') throw new Error('只有 pending 可支付: ' + order.status);
  order.status = 'paid';
  order.paidAt = new Date();
}

function cancel(order, reason) {
  if (order.status === 'paid') throw new Error('已支付订单不能取消');
  order.status = 'cancelled';
  order.reason = reason;
}

function describe(order) {
  const yuan = (order.totalCents / 100).toFixed(2);
  switch (order.status) {
    case 'pending': return '待支付 ' + order.id + ', 金额 ¥' + yuan;
    case 'paid':    return '已支付 ' + order.id + ', 金额 ¥' + yuan;
    case 'cancelled': return '已取消 ' + order.id + ': ' + order.reason;
    default: throw new Error('未知订单状态: ' + order.status);
  }
}

// ===== JSON 往返:日期显式转换,undefined 显式省略 =====
function toJSON(order) {
  return {
    id: order.id,
    status: order.status,
    user: order.user,
    items: order.items,
    totalCents: order.totalCents,
    createdAt: order.createdAt.toISOString(), // Date → 字符串
    paidAt: order.paidAt ? order.paidAt.toISOString() : null,
  };
}

function fromJSON(json) {
  const order = createOrder({
    id: json.id,
    user: json.user,
    items: json.items,
  });
  order.status = json.status; // 还原状态
  order.createdAt = new Date(json.createdAt); // 字符串 → Date
  order.paidAt = json.paidAt ? new Date(json.paidAt) : null;
  order.totalCents = json.totalCents;
  return order;
}

// ===== 主流程 =====
function main() {
  const apple = createProduct('P-001', '苹果', 550); // ¥5.50,分单位
  const milk = createProduct('P-002', '牛奶', 1200); // ¥12.00

```

```

const user = createUser('U-9001', '李雷', '北京');

const order = createOrder({ id: 'ORD-9001', user, items: [apple, milk] });
console.log(describe(order));           // 待支付 ORD-9001, 金额 ¥17.50
pay(order);
console.log(describe(order));           // 已支付 ...
cancel(order, '用户取消');              // 抛错:已支付不能取消

// 深拷贝:改副本不影响原件
const copy = structuredClone(order);
copy.user.name = '韩梅梅';
console.log('原件用户:', order.user.name); // 李雷
console.log('副本用户:', copy.user.name);   // 韩梅梅

// JSON 往返
const json = JSON.stringify(toJSON(order));
const restored = fromJSON(JSON.parse(json));
console.log('往返一致:', restored.id === order.id &&
  restored.totalCents === order.totalCents);
console.log('还原的日期类型:', restored.createdAt instanceof Date); // true

// 空安全:地址可能缺失
const guest = createUser('U-9002', '韩梅梅'); // 没有 city
console.log('城市:', guest?.city ?? '未知');    // 未知
}
main();

```

设计说明:金额用"分"整数存储,totalCents 由 items 归约计算,输出时再转元——规避浮点误差;paidAt 用显式 null 而非 undefined,让 JSON 往返行为可预测;createOrder 在构造时即校验,保证系统内不存在非法订单;copy 演示 structuredClone 深拷贝隔离;guest?.city ?? '未知' 是空安全黄金组合。运行后应看到:支付成功描述、取消抛错、深拷贝互不影响、往返一致为 true、日期还原为 Date 实例。

全书写到这里。从第1章的数字与类型,到第14章的领域建模,你已经走完了"从 C 到 JavaScript"的完整进阶之路。祝你在 Node.js 与浏览器两个世界都写出清晰、健壮、可维护的代码。